



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO De Tuxtepec

Materia: CIENCIA DE DATOS

DOCENTE:

Doctor Julio Aguilar Carmona.

Actividad:

Curso KAGGLE.

Alumno:

Reyes Angel Alexis.

No. Control: 22350372

Correo: alexisreyesangel7@gmail.com

CARRERA:

INGENIERIA EN SISTEMAS
COMPUTACIONALES

Especialidad:

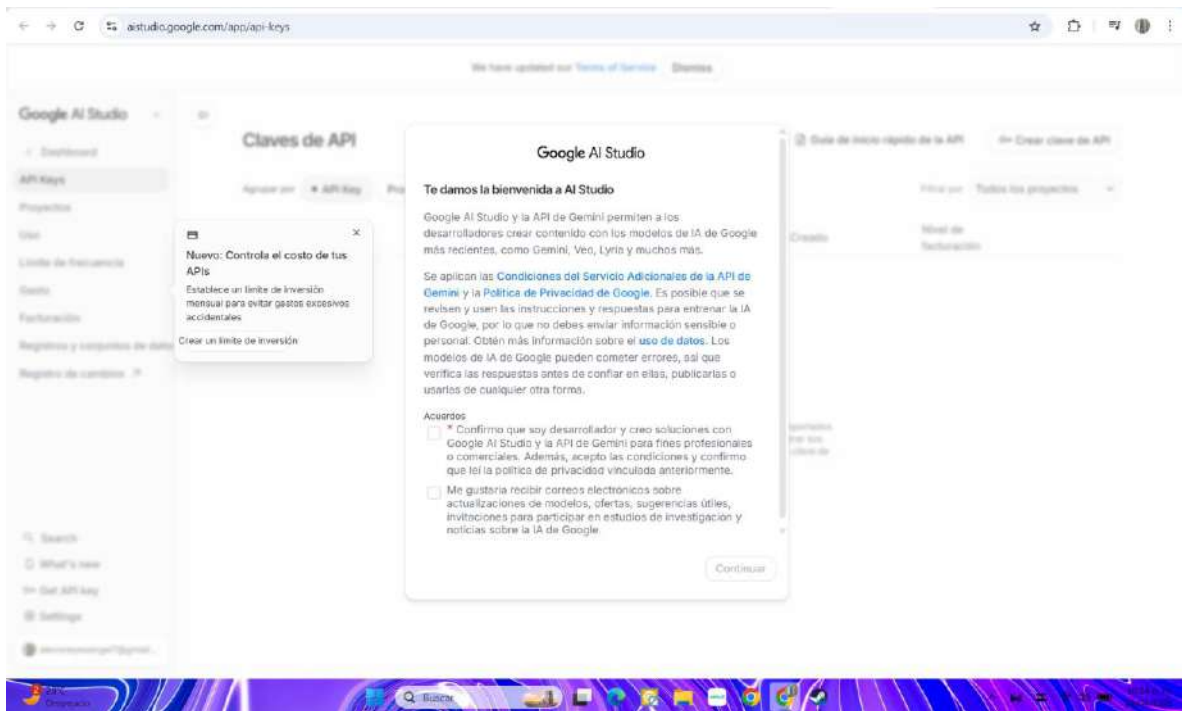
INTELIGENCIA DE NEGOCIOS Y CIENCIA DE DATOS.

Semestre/Grupo: 8° "A"

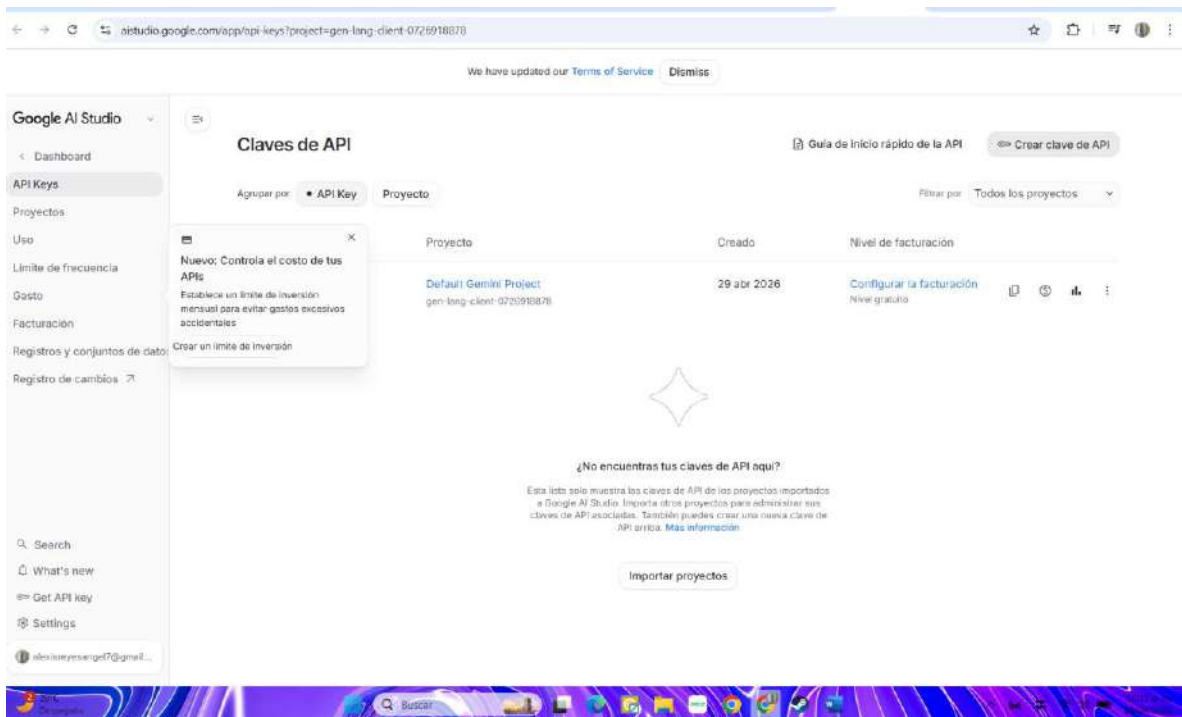
San Juan Bautista Tuxtepec Oaxaca,
02 Mayo de 2026.



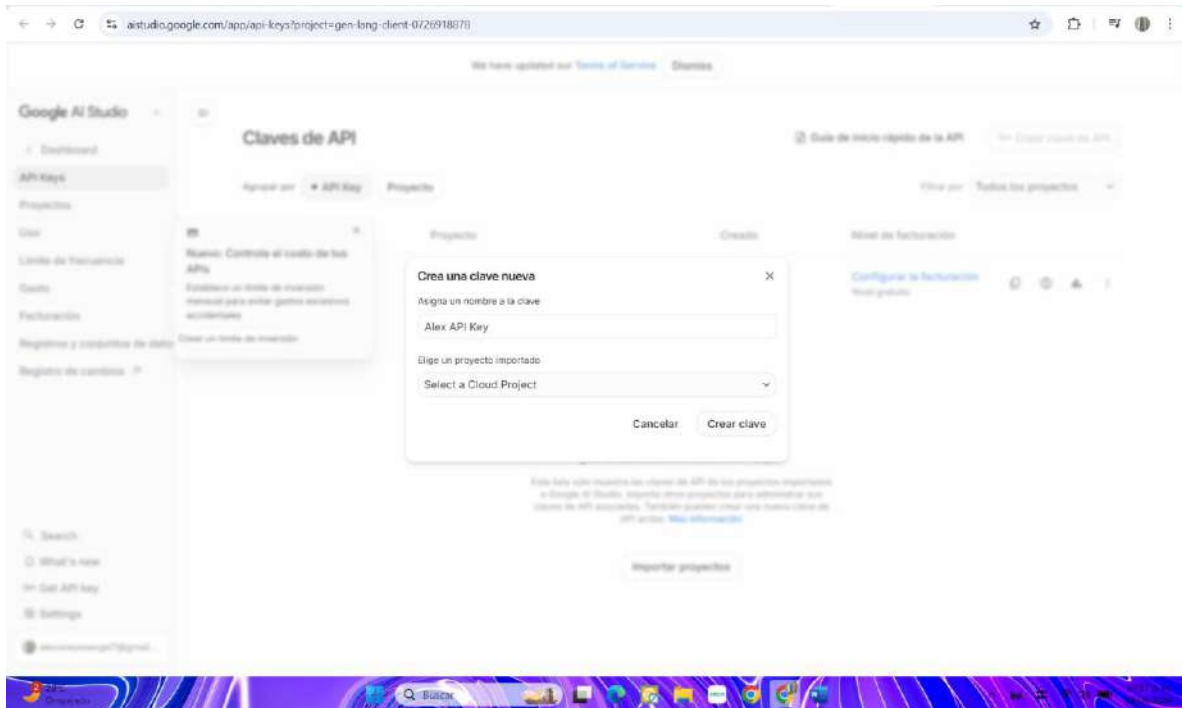
Curso KAGGLE – DÍA 1



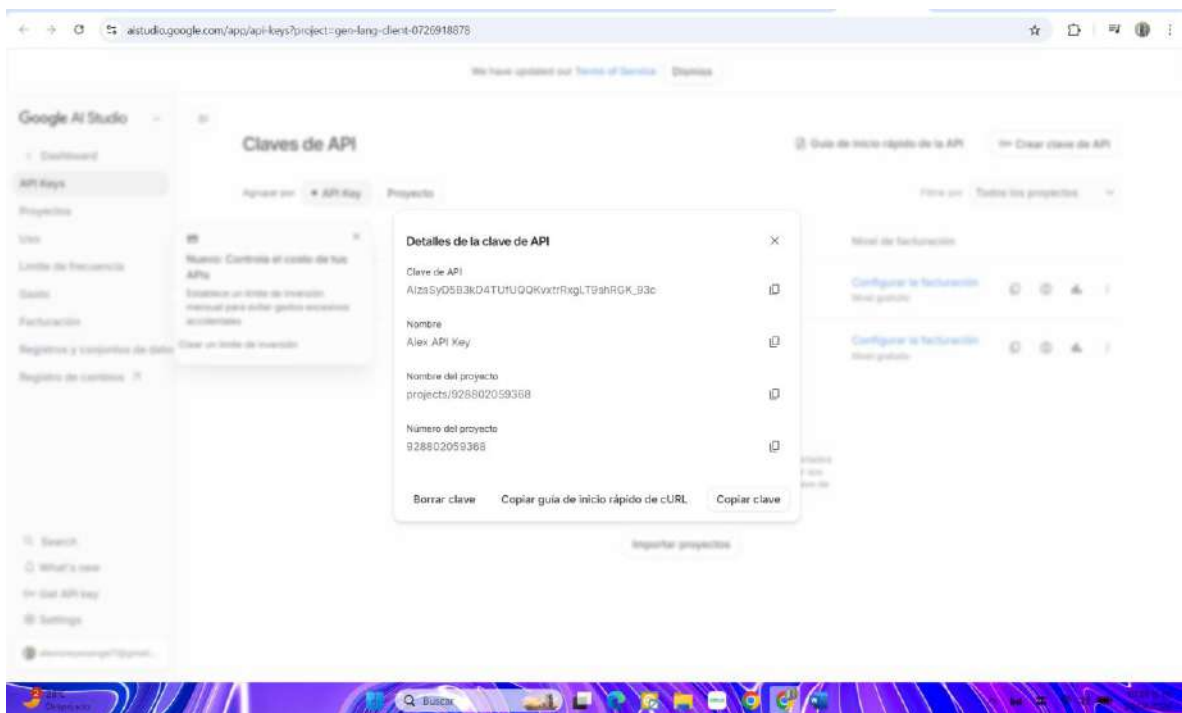
Creamos nuestra llave API



Elegimos el nombre de nuestra clave y le damos en crear proyecto.



Ahora tenemos nuestra clave creada



Añadimos nuestra clave API creada anteriormente

The screenshot shows the Kaggle Notebook editor for 'Day 1a - From Prompt to Action'. The notebook content includes instructions for configuring the Gemini API key and a code cell for authentication. The 'Add Secrets' dialog box is open on the right, showing a new secret with the label 'GOOGLE_API_KEY' and the value 'AlzaSyCjDnoP5nR_PMvicH1c5_vVHnRB'. The dialog has 'Save' and 'Cancel' buttons.

Day 1a - From Prompt to Action Draft saved

File Edit View Run Settings Add-ons Help

Share Save Version 0

1.2: Configure your Gemini API Key

This notebook uses the Gemini API, which requires authentication.

1. Get your API key

If you don't have one already, create an API key in Google AI Studio.

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select **Add-ons**, then **Secrets**.

2. Create a new secret with the label `GOOGLE_API_KEY`.

3. Paste your API key into the "Value" field and click "Save".

4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

```
[ ]: import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(e)
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Añadimos la etiqueta que nos menciona el documento "GOOGLE_API_KEY" y después le damos en guardar. Listo tenemos añadida la clave API.

The screenshot shows the same Kaggle Notebook editor. The 'Add Secrets' dialog box is now closed, and the 'GOOGLE_API_KEY' secret is listed in the 'Secrets' panel on the right. The secret is checked, indicating it is attached to the notebook. The code cell for authentication is still present.

Day 1a - From Prompt to Action Draft saved

File Edit View Run Settings Add-ons Help

Share Save Version 0

1.2: Configure your Gemini API Key

This notebook uses the Gemini API, which requires authentication.

1. Get your API key

If you don't have one already, create an API key in Google AI Studio.

2. Add the key to Kaggle Secrets

Next, you will need to add your API key to your Kaggle Notebook as a Kaggle User Secret.

1. In the top menu bar of the notebook editor, select **Add-ons**, then **Secrets**.

2. Create a new secret with the label `GOOGLE_API_KEY`.

3. Paste your API key into the "Value" field and click "Save".

4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

+ Code + Markdown

```
[ ]: import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(e)
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Code Snippet

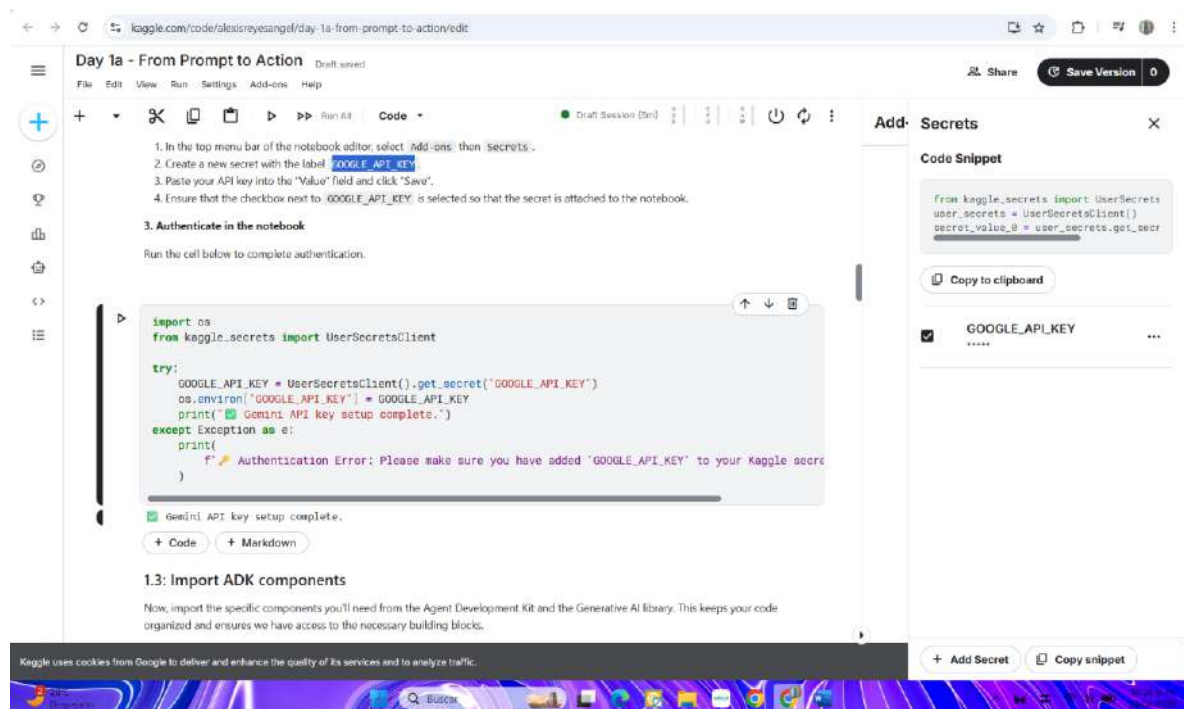
```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_B = user_secrets.get_secret_value("GOOGLE_API_KEY")
```

Copy to clipboard

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

Ahora ejecutamos la clave API de salida y esperamos hasta que nos aparezca que este completada la acción que acabamos de realizar.



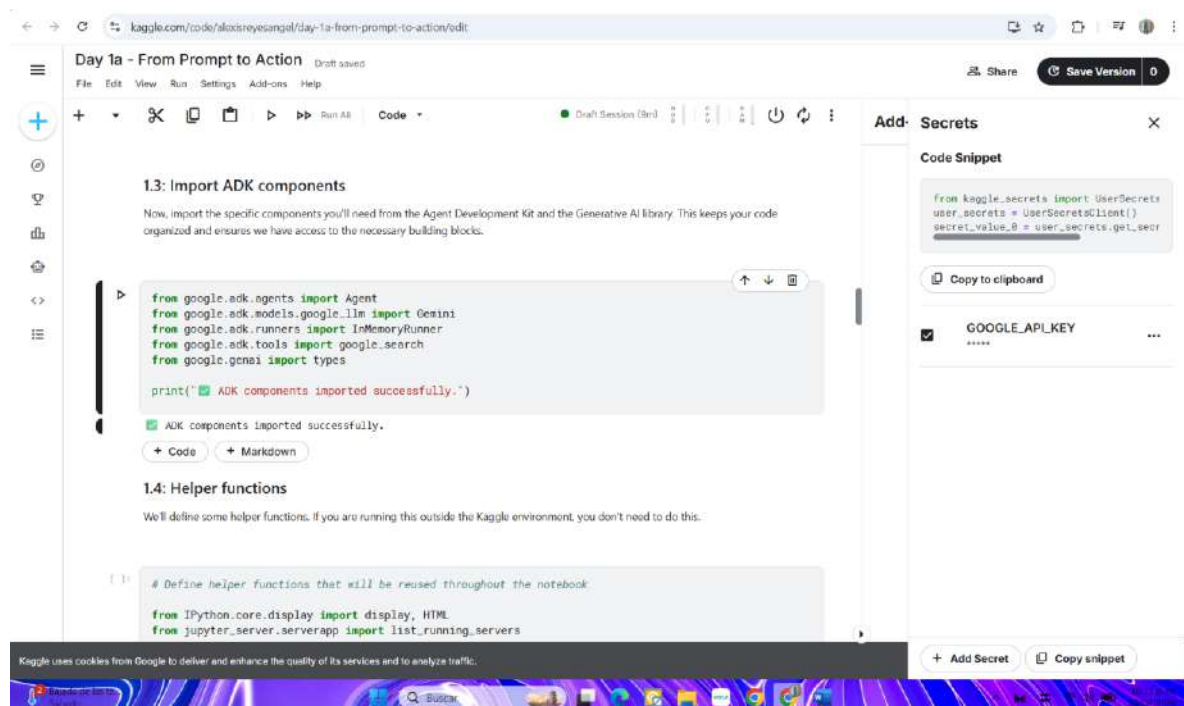
The screenshot shows a Jupyter Notebook interface on Kaggle. The notebook is titled "Day 1a - From Prompt to Action" and is in "Draft saved" state. The left sidebar shows the notebook's structure with sections for "1. In the top menu bar of the notebook editor, select 'Add-ons', then 'Secrets'." and "3. Authenticate in the notebook". The main content area shows a code cell with the following Python code:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secret
    )
```

The output of the code cell shows "✅ Gemini API key setup complete." Below the code cell, there are buttons for "+ Code" and "+ Markdown". The right sidebar shows the "Add Secrets" panel with a "Code Snippet" section containing the same code as the cell. Below the snippet, there is a "Copy to clipboard" button and a "GOOGLE_API_KEY" secret entry with a "Copy snippet" button.

Ahora ejecutaremos los componentes de ADK, y esperaremos hasta que finalice la ejecución.



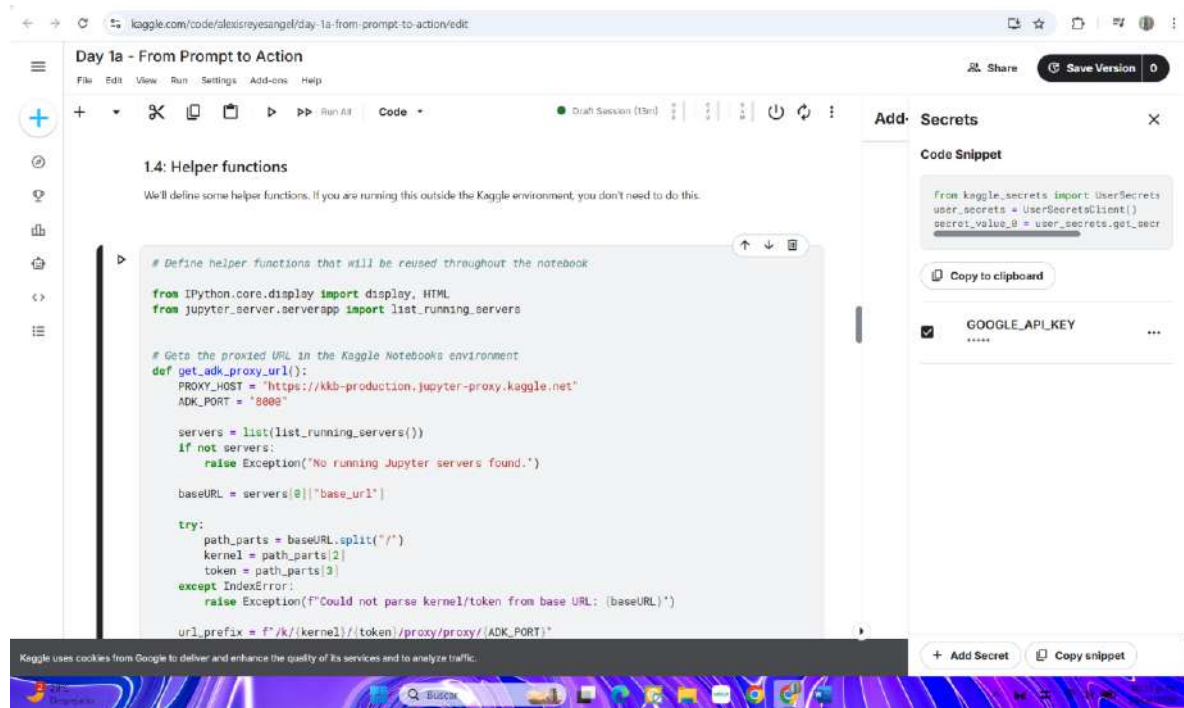
The screenshot shows a Jupyter Notebook interface on Kaggle. The notebook is titled "Day 1a - From Prompt to Action" and is in "Draft saved" state. The left sidebar shows the notebook's structure with sections for "1.3: Import ADK components" and "1.4: Helper functions". The main content area shows a code cell with the following Python code:

```
from google.adk.agents import Agent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types

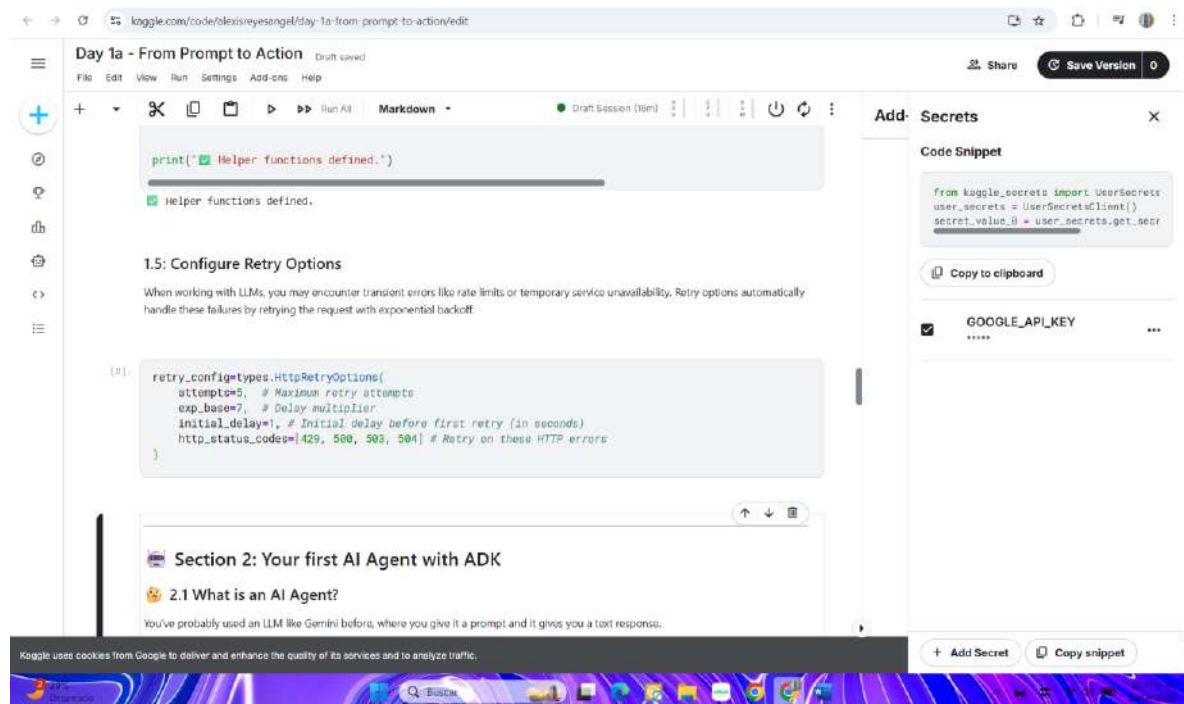
print("✅ ADK components imported successfully.")
```

The output of the code cell shows "✅ ADK components imported successfully." Below the code cell, there are buttons for "+ Code" and "+ Markdown". The right sidebar shows the "Add Secrets" panel with a "Code Snippet" section containing the same code as the cell. Below the snippet, there is a "Copy to clipboard" button and a "GOOGLE_API_KEY" secret entry with a "Copy snippet" button.

Posteriormente definimos las funciones auxiliares.



Límites de velocidad.



Ahora definiremos nuestro primer agente de IA: Ejecutamos nuestro agente raíz definido

The screenshot shows a Jupyter Notebook interface with the title "Day 1a - From Prompt to Action". The notebook content includes:

- instruction:** The agent's guiding prompt. This tells the agent what its goal is and how to behave.
- tools:** A list of tools that the agent can use. To start, we'll give it the `google_search` tool, which lets it find up-to-date information online.

```
root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    descriptions="A simple agent that can answer general questions.",
    instructions="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("Root Agent defined.")
```

Below the code, a green checkmark indicates "Root Agent defined." and there are buttons for "+ Code" and "+ Markdown".

2.3 Run your agent

Now it's time to bring your agent to life and send it a query. To do this, you need a `Runner`, which is the central component within ADK that acts as the orchestrator. It manages the conversation, sends our messages to the agent, and handles its responses.

a. Create an `InMemoryRunner` and tell it to use our `root_agent`:

On the right side, there is a "Secrets" panel with a "Code Snippet" section containing the following code:

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Below the snippet is a "Copy to clipboard" button. In the "Secrets" list, "GOOGLE_API_KEY" is visible with a masked value.

Ahora ejecutamos nuestro agente.

This screenshot shows the same Jupyter Notebook at a later stage. The code cell below the previous one is now executed:

```
runner = InMemoryRunner(agent=root_agent)

print("Runner created.")
```

A green checkmark indicates "Runner created." and there are buttons for "+ Code" and "+ Markdown".

2.3 Run your agent

Now it's time to bring your agent to life and send it a query. To do this, you need a `Runner`, which is the central component within ADK that acts as the orchestrator. It manages the conversation, sends our messages to the agent, and handles its responses.

a. Create an `InMemoryRunner` and tell it to use our `root_agent`:

b. Now you can call the `.run_debug()` method to send our prompt and get an answer.

c. This method abstracts the process of session creation and maintenance and is used in prototyping. We'll explore "what sessions are and how to create them" on Day 3.

The "Secrets" panel on the right remains the same as in the previous screenshot.

Creamos nuestra sesión.

The screenshot shows a Jupyter Notebook interface with the title "Day 1a - From Prompt to Action". The notebook contains a code cell with the following content:

```
response = await runner.run_debug(  
    "What is Agent Development Kit from Google? What languages is the SDK available in?"  
)  
  
## Created new session: debug_session_id
```

Below the code cell, there is a text block that reads: "User > What is Agent Development Kit from Google? What languages is the SDK available in? helpful_assistant > The Agent Development Kit (ADK) from Google is an open-source framework designed to simplify the creation, debugging, and large-scale deployment of AI agents and multi-agent systems. It allows developers to build applications ranging from personal AI assistants to critical business workflows. ADK provides a comprehensive set of tools for developing, running, evaluating, and scaling AI architectures. It supports multi-agent architectures, enabling the composition of specialized agent teams that can collaborate and delegate tasks. The framework also offers a rich tool ecosystem for integrating third-party applications and custom code."

Below the text block, there is a list of languages: "The ADK is available in the following programming languages: Python, TypeScript, Go, Java".

At the bottom of the notebook, there is a text block that reads: "You can see a summary of ADK and its available languages in the response."

On the right side of the notebook, there is a "Secrets" panel with a "Code Snippet" section containing the following code:

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Below the code snippet, there is a "Copy to clipboard" button. In the "Secrets" panel, there is a checkbox labeled "GOOGLE_API_KEY" which is checked.

Creamos nuestro agente.

The screenshot shows a Jupyter Notebook interface with the title "Day 1a - From Prompt to Action". The notebook contains a code cell with the following content:

```
!adk create sample-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

Below the code cell, there is a text block that reads: "Agent created in /kaggle/working/sample-agent: __init__.py, __main__.py, agent.py".

Below the text block, there is a text block that reads: "Get your custom URL to access the ADK web UI in the Kaggle Notebooks environment:".

Below the text block, there is a code cell with the following content:

```
url_prefix = get_adk_proxy_url()
```

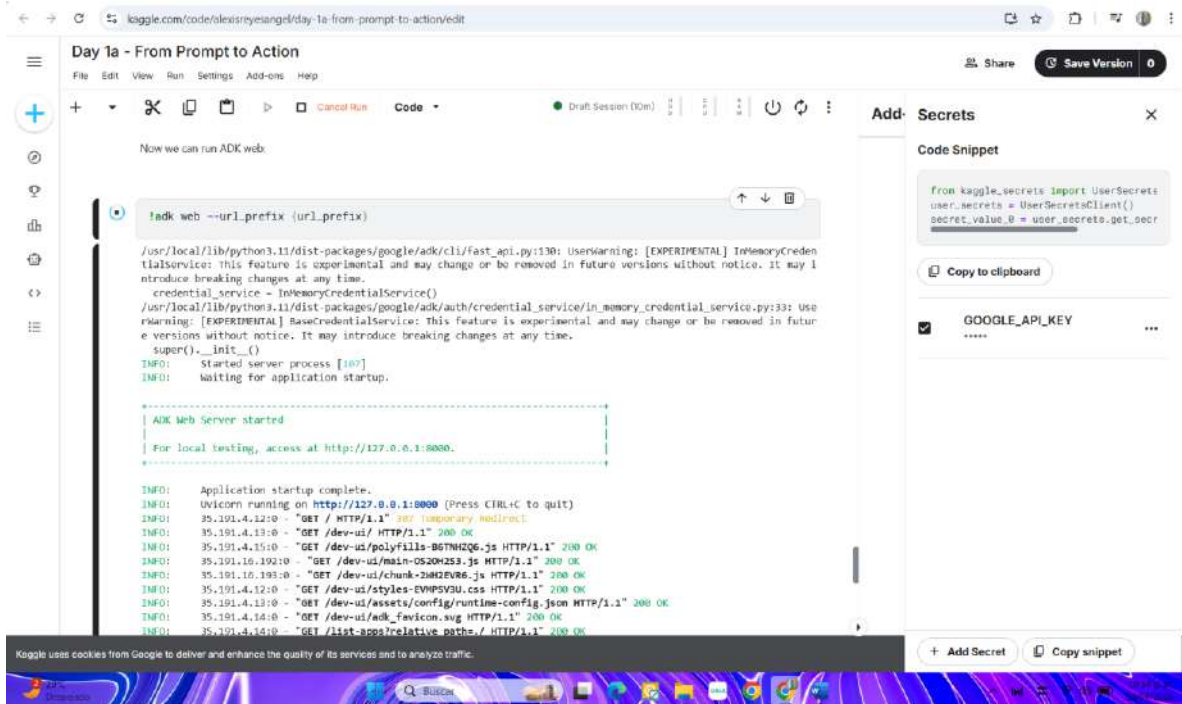
At the bottom of the notebook, there is a text block that reads: "Now we can run ADK web:".

On the right side of the notebook, there is a "Secrets" panel with a "Code Snippet" section containing the following code:

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Below the code snippet, there is a "Copy to clipboard" button. In the "Secrets" panel, there is a checkbox labeled "GOOGLE_API_KEY" which is checked.

Activamos nuestro agente para poder iniciar con el URL.



Day 1a - From Prompt to Action

Now we can run ADK web:

```
!adk web --url_prefix (url_prefix)
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UseWarning: [EXPERIMENTAL] SaveCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [167]
INFO: Waiting for application startup.

[ ADK Web Server started ]
[ For local testing, access at http://127.0.0.1:8080. ]

INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: 35.191.4.12:0 - "GET / HTTP/1.1" 302 [temporary redirect]
INFO: 35.191.4.13:0 - "GET /dev-ui/ HTTP/1.1" 200 OK
INFO: 35.191.4.13:0 - "GET /dev-ui/polyfills-B67MHQ6.js HTTP/1.1" 200 OK
INFO: 35.191.16.192:0 - "GET /dev-ui/main-OS20K23.js HTTP/1.1" 200 OK
INFO: 35.191.16.193:0 - "GET /dev-ui/chunk-2MH2EVR6.js HTTP/1.1" 200 OK
INFO: 35.191.4.12:0 - "GET /dev-ui/styles-EVHPSV3U.css HTTP/1.1" 200 OK
INFO: 35.191.4.13:0 - "GET /dev-ui/assets/config/runtime-config.json HTTP/1.1" 200 OK
INFO: 35.191.4.14:0 - "GET /dev-ui/adk_favicon.svg HTTP/1.1" 200 OK
INFO: 35.191.4.14:0 - "GET /list-apps?relative_paths=/ HTTP/1.1" 200 OK
```

Code Snippet

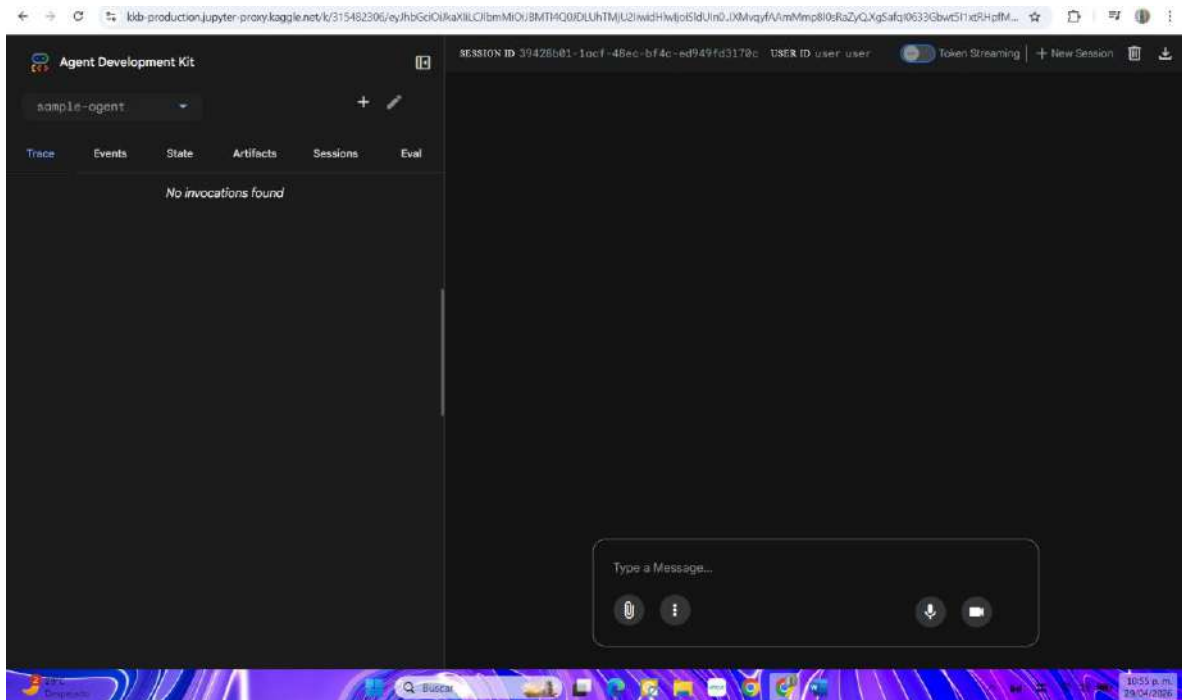
```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
```

Copy to clipboard

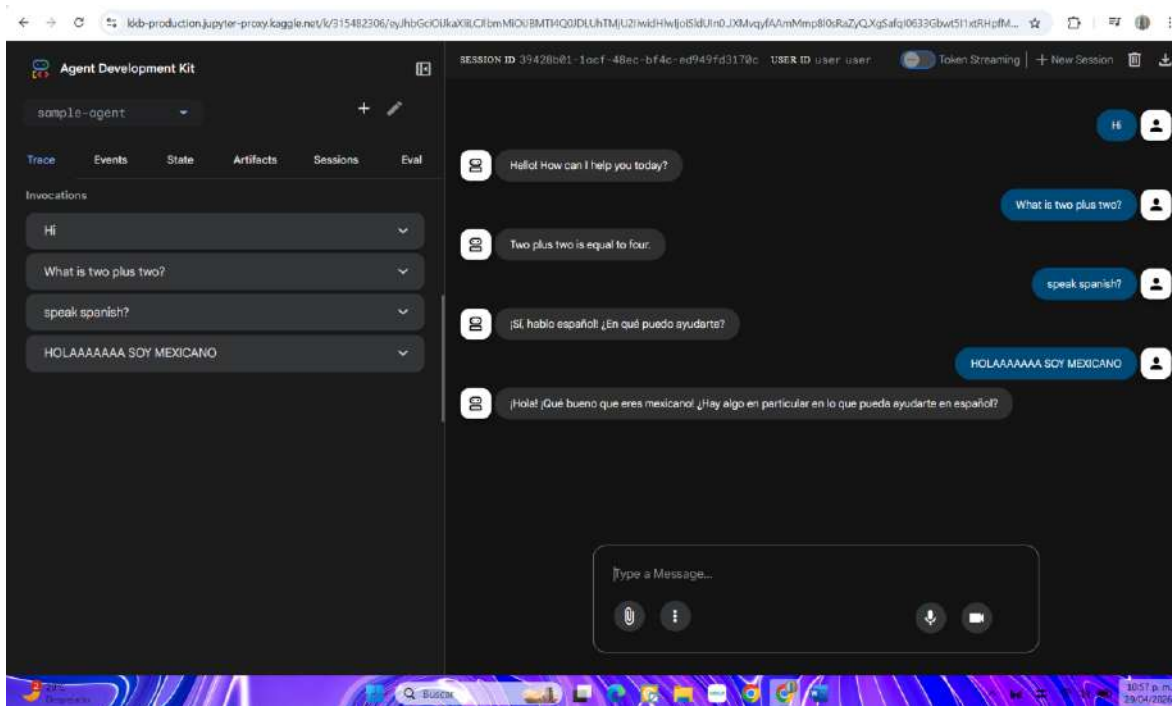
GOOGLE_API_KEY

Add Secret Copy snippet

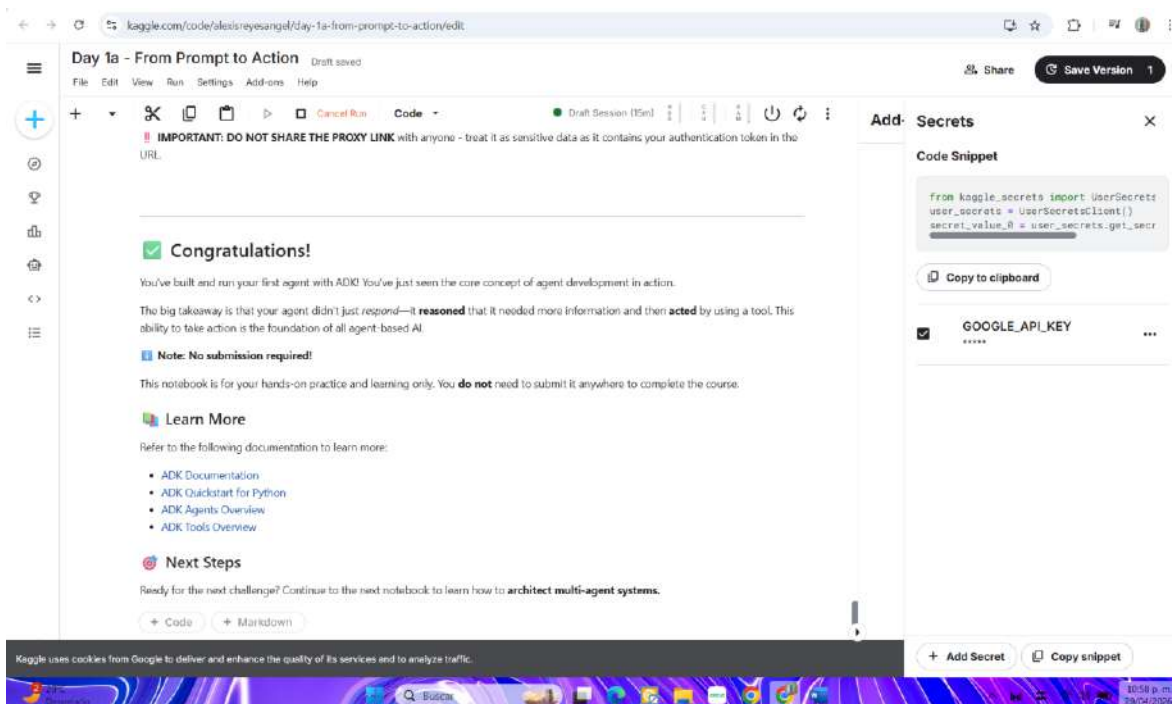
Una vez iniciado el servidor el agente de desarrollo estará en funcionamiento.



Y podemos verificar que este funcionando (entiende el español)



Hemos terminado la sesión 1 en donde el trabajo final fue haber creado nuestro agente IA. Podemos guardar nuestro progreso.



Para la sesión 2 activamos nuestra clave API nuevamente que ya habíamos realizado anteriormente.

The screenshot shows the Kaggle notebook interface for 'Day 1b - Agent Architectures'. The top bar includes the notebook title, a 'Draft saved' status, and buttons for 'Share' and 'Save Version'. The left sidebar contains navigation icons. The main content area displays the Apache License 2.0 text. Below the license, the notebook title 'Multi-Agent Systems & Workflow Patterns' is shown, followed by a welcome message to the Kaggle 5-day Agents course. The text explains that the user will learn to scale up by building agent teams, contrasting a single agent with a multi-agent system. A right-hand sidebar titled 'Add Secrets' shows a 'Code Snippet' with Python code for importing secrets and a list of secrets, including 'GOOGLE_API_KEY'.

Importamos nuestra clave API nuevamente.

This screenshot shows the same Kaggle notebook at a later stage. The main content area now displays a code cell for authentication. The code imports the 'os' module and 'UserSecretsClient' from 'kaggle.secrets'. It uses a try-except block to attempt retrieving the 'GOOGLE_API_KEY' secret and setting it as an environment variable. A message 'Gemini API key setup complete.' is printed upon success. Below the code cell, the notebook section '1.3: Import ADK components' is visible, with text explaining the purpose of importing specific components from the Agent Development Kit (ADK) and the Generative AI library. The right-hand sidebar remains the same, showing the 'Add Secrets' panel with the 'GOOGLE_API_KEY' secret.

Importamos los componentes ADK

The screenshot shows a Kaggle notebook interface. The main content area displays the following text:

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import Agent, SequentialAgent, ParallelAgent, LoopAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import AgentTool, FunctionTool, google_search
from google.genai import types

print("ADK components imported successfully.")
```

Below the code, it says "ADK components imported successfully."

1.4: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

On the right side, there is a "Secrets" panel with a "Code Snippet" section containing the following code:

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Below the snippet is a "Copy to clipboard" button. In the "Secrets" list, "GOOGLE_API_KEY" is selected.

Ejecutamos el agente de resumen.

The screenshot shows a Kaggle notebook interface. The main content area displays the following text:

2.1 Example: Research & Summarization System

Let's build a system with two specialized agents:

1. **Research Agent** - Searches for information using Google Search
2. **Summarizer Agent** - Creates concise summaries from research findings

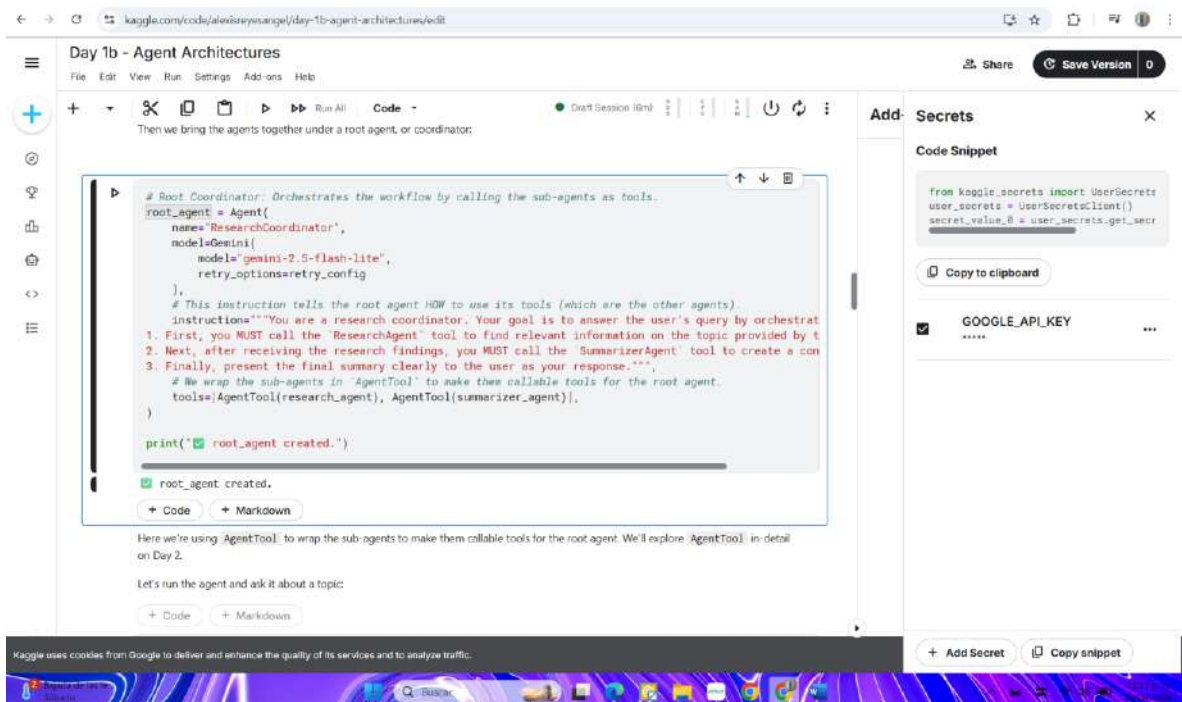
```
# Research Agent: Its job is to use the google_search tool and present findings:
research_agent = Agent(
    name="ResearchAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instructions="You are a specialized research agent. Your only job is to use the google_search tool to find 2-3 pieces of relevant information on the given topic and present the findings.",
    tools=[google_search],
    output_key="research_findings", # The result of this agent will be stored in the session state with this key
)

print("research_agent created.")
```

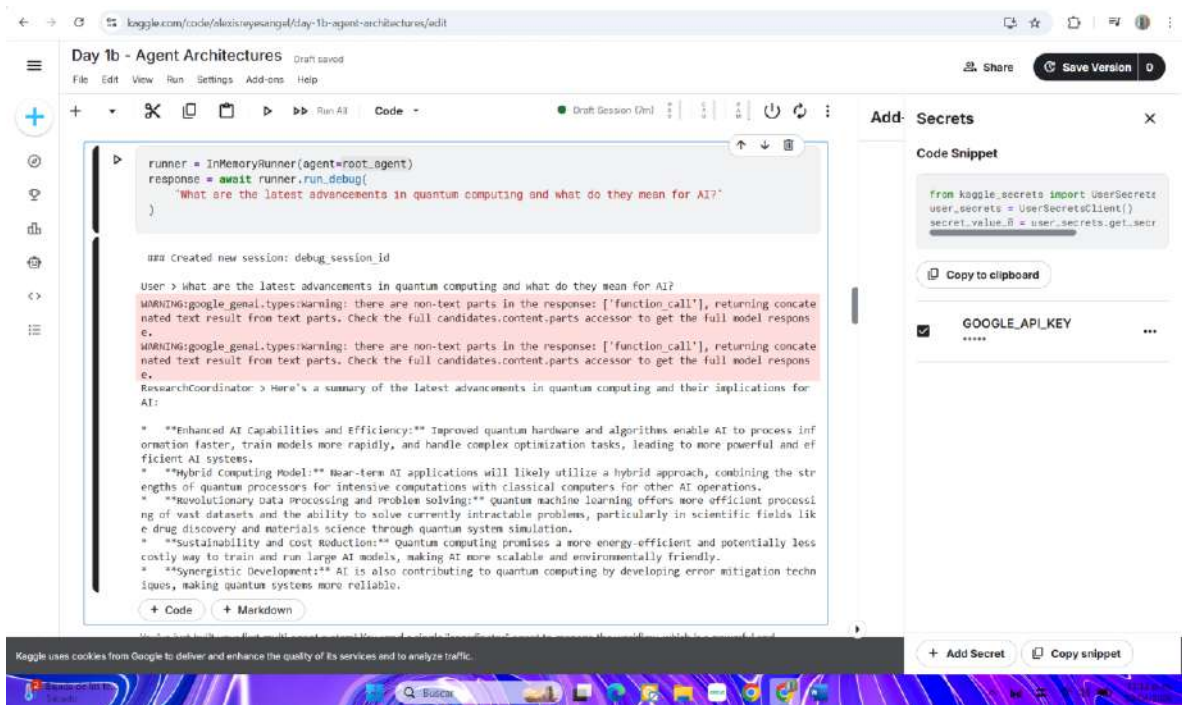
Below the code, it says "research_agent created."

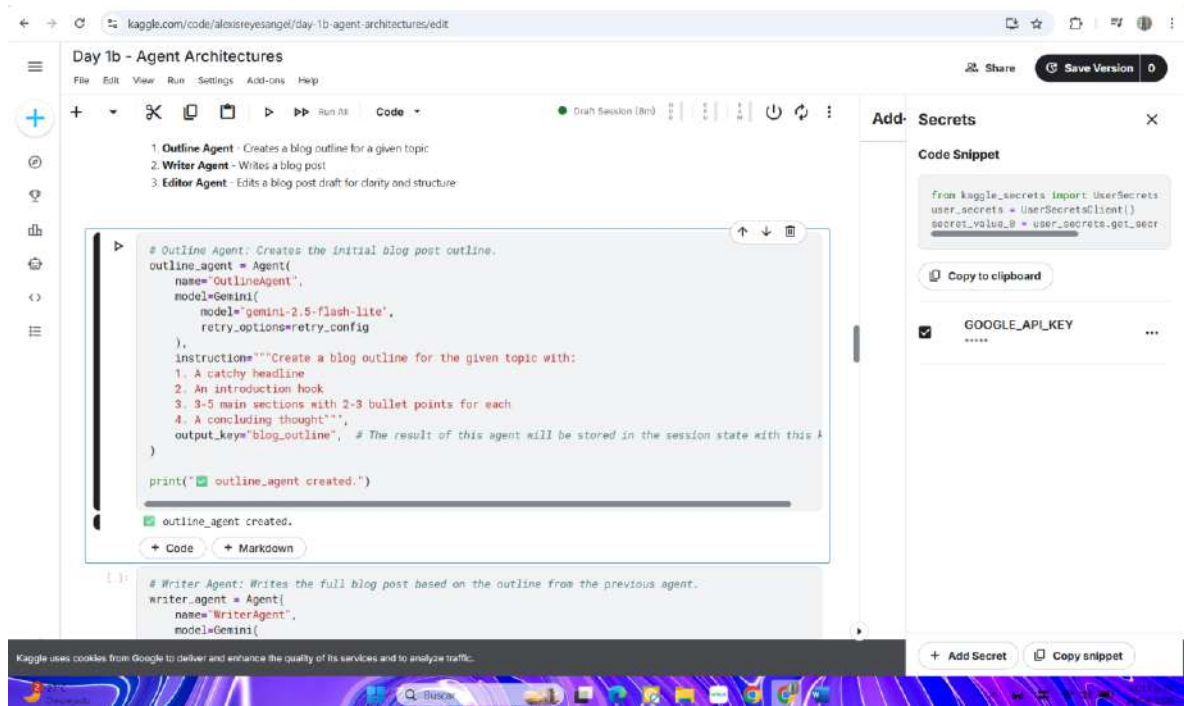
Below that, there is a code block for the Summarizer Agent:

```
# Summarizer Agent: Its job is to summarize the text it receives.
summarizer_agent = Agent(
    name="SummarizerAgent",
    model=Gemini(
```

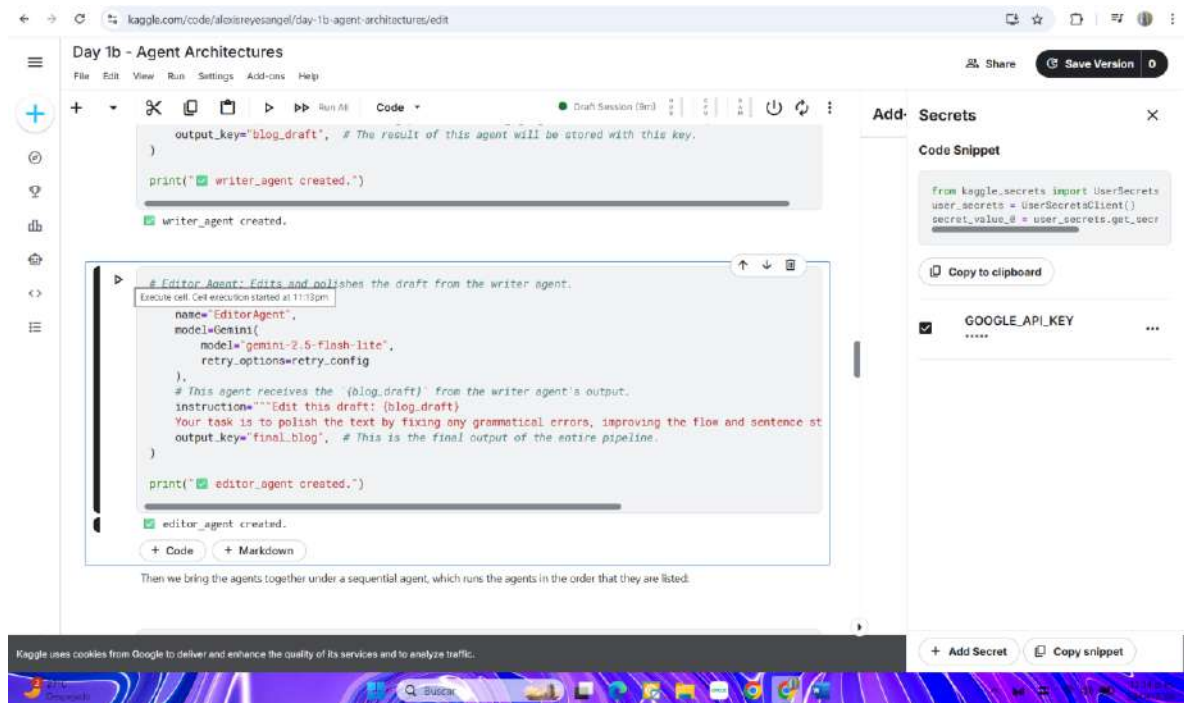


Ejecutamos el coordinador de investigación.





Ejecutamos nuestro agente Editor.



Creamos nuestra sesión.

The screenshot shows the Kaggle interface for the 'Day 1b - Agent Architectures' notebook. The code cell [27] defines a `SequentialAgent` named `BlogPipeline` with sub-agents `outline_agent`, `writer_agent`, and `editor_agent`. It prints 'Sequential Agent created.' and the output shows 'Sequential Agent created.'

Below the code, a text prompt says: 'Let's run the agent and give it a topic to write a blog post about:'. A code cell [28] creates an `InMemoryRunner` for the `root_agent` and runs it with the prompt: 'Write a blog post about the benefits of multi-agent systems for software developers'. The output shows the session ID and the user prompt.

At the bottom, a message states: 'Great job! You've now created a reliable "assembly line" using a sequential agent, where each step runs in a predictable order.'

The right sidebar shows the 'Add Secrets' panel with a 'Code Snippet' and a 'GOOGLE_API_KEY' secret.

The screenshot shows the Kaggle interface for the 'Day 1b - Agent Architectures' notebook. The code cell [29] creates a `Gemini` agent named `tech_researcher` with the instruction: 'Research the latest AI/ML trends. Include 3 key developments, the main companies involved, and the potential impact. Keep the report very concise (100 words)'. It uses the `google_search` tool and stores the result in the session state with the key `tech_research`. The output shows 'tech_researcher created.'

Below the code, a text prompt says: 'Let's run the agent and give it a topic to write a blog post about:'. A code cell [30] creates a `Gemini` agent named `health_researcher` with the instruction: 'Research recent medical breakthroughs. Include 3 significant advances, their practical applications, and estimated timelines. Keep the report concise (100 words)'. It uses the `google_search` tool and stores the result in the session state with the key `health_research`. The output shows 'health_researcher created.'

The right sidebar shows the 'Add Secrets' panel with a 'Code Snippet' and a 'GOOGLE_API_KEY' secret.

Day 1b - Agent Architectures

Then we bring the agents together under a parallel agent, which is itself nested inside of a sequential agent.

This design ensures that the research agents run first in parallel, then once all of their research is complete, the aggregator agent brings together all of the research findings into a single report:

```
# The ParallelAgent runs all its sub-agents simultaneously.
parallel_research_team = ParallelAgent(
    name="ParallelResearchTeam",
    sub_agents=[tech_researcher, health_researcher, finance_researcher],
)

# This SequentialAgent defines the high-level workflow: run the parallel team first, then run the aggregator
root_agent = SequentialAgent(
    name="ResearchSystem",
    sub_agents=[parallel_research_team, aggregator_agent],
)

print("✅ Parallel and Sequential Agents created.")
```

Parallel and Sequential Agents created.

Let's run the agent and give it a prompt to research the given topics:

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug()
```

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

Share Save Version 0

Day 1b - Agent Architectures

Parallel and Sequential Agents created.

Let's run the agent and give it a prompt to research the given topics:

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Run the daily executive briefing on Tech, Health, and Finance"
)
```

Created new session: debug_session_id

User > Run the daily executive briefing on Tech, Health, and Finance

HealthResearcher > This report summarizes recent breakthroughs across Health, Technology, and Finance:

Health:

- CRISPR Gene Editing: This technology allows for precise DNA alteration, with potential to correct genetic diseases like sickle cell anemia and muscular dystrophy.
- mRNA Vaccine Technology: Proven effective with COVID-19, mRNA platforms are being explored for a wider range of diseases, offering rapid development and adaptability.
- Non-Invasive Glucose Monitoring: Emerging biosensors aim to replace invasive blood sampling for diabetics, providing real-time insulin level data.

Technology:

- AI Advancement (Generative AI & Edge AI): AI's capabilities have expanded with generative models like ChatGPT, impacting various industries. Edge AI is enabling on-device processing for faster, more private operations.
- Quantum Computing: Significant strides are being made toward practical quantum systems with new chip announcements. Potential applications span complex problem-solving in various fields.
- 5G Expansion: Increased rollout of 5G is enabling advancements in areas like autonomous vehicles, smart ci

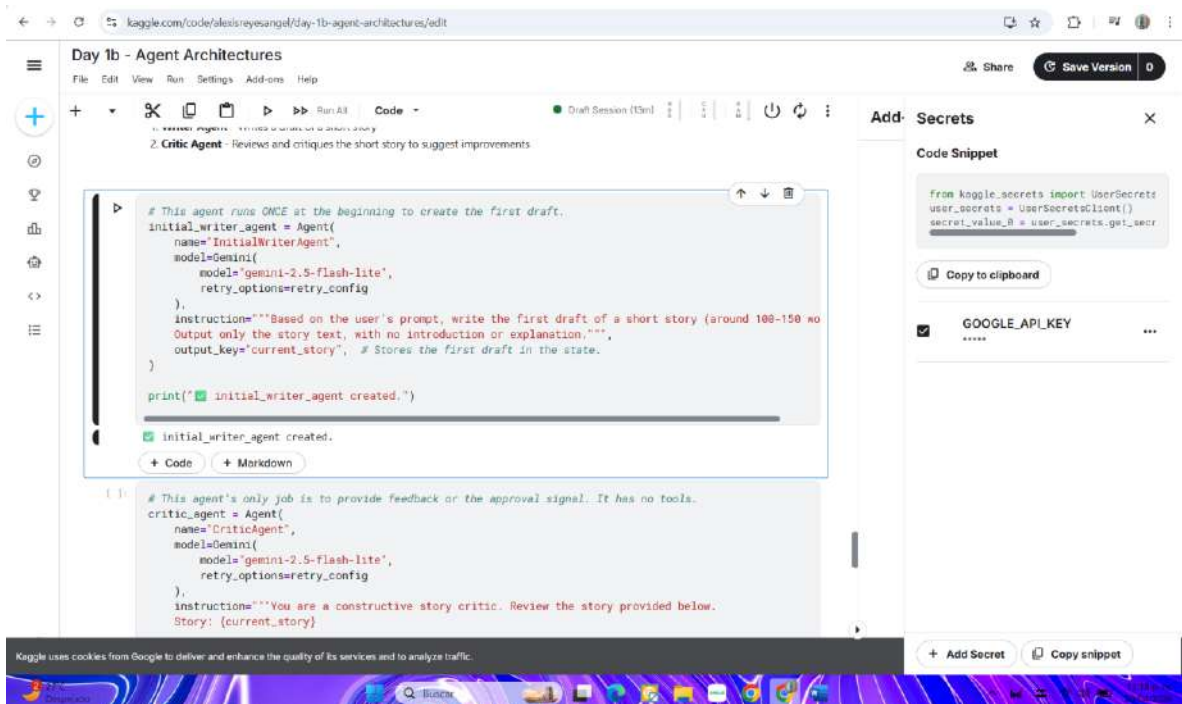
Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

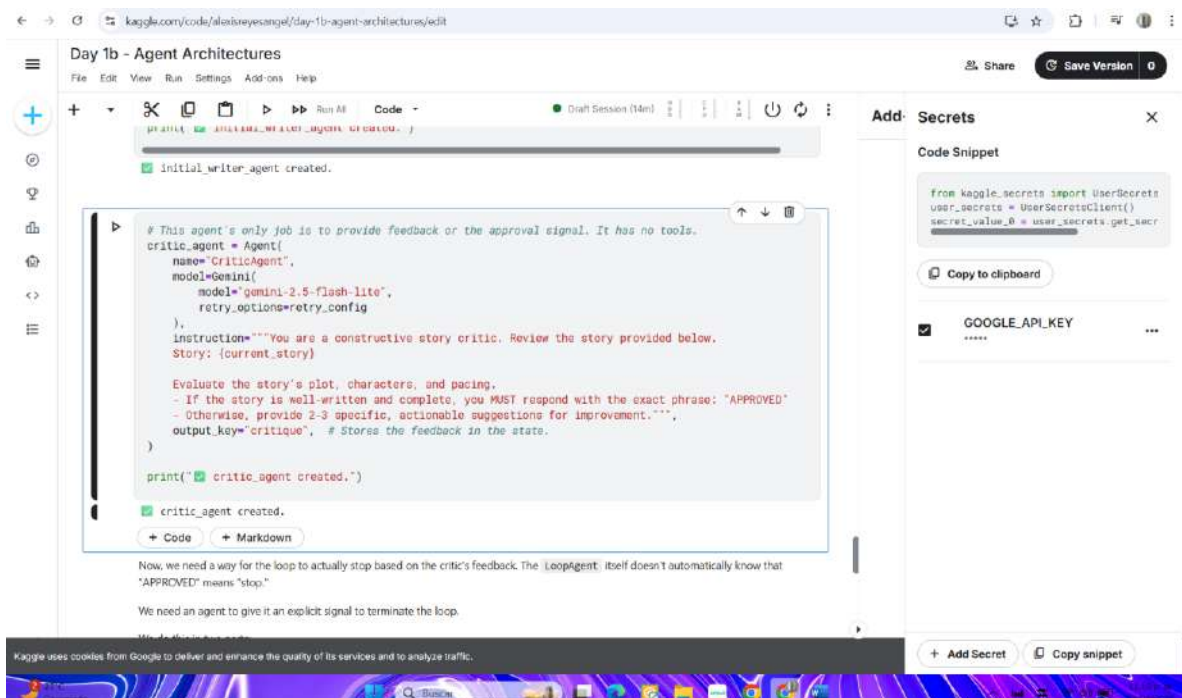
Copy to clipboard

GOOGLE_API_KEY

Share Save Version 0



Ejecutamos el segundo agente.



Day 1b - Agent Architectures Draft saved

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (15m)

Share Save Version 0

We do this in two parts:

1. A simple Python function that the `LoopAgent` understands as an "exit" signal.
2. An agent that can call that function when the right condition is met.

First, you'll define the `exit_loop` function:

```
# This is the function that the RefinerAgent will call to exit the loop.
def exit_loop():
    """Call this function ONLY when the critique is 'APPROVED', indicating the story is finished and no
    return ("status": "approved", "message": "Story approved. Exiting refinement loop.")

print('exit_loop function created.')
```

exit_loop function created.

+ Code + Markdown

To let an agent call this Python function, we wrap it in a `FunctionTool`. Then, we create a `RefinerAgent` that has this tool.

Notice its instructions: this agent is the "brain" of the loop. It reads the `[critique]` from the `CriticAgent` and decides whether to (1) call the `exit_loop` tool or (2) rewrite the story.

```
[ ] # This agent refines the story based on critique OR calls the exit_loop function.
refiner_agent = Agent(
    name="RefinerAgent",
    model=Gemini(
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_8 = user_secrets.get_secr
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 1b - Agent Architectures Draft saved

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (15m)

Share Save Version 0

```
# This agent refines the story based on critique OR calls the exit_loop function.
refiner_agent = Agent(
    name="RefinerAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="You are a story refiner. You have a story draft and critique.

    Story Draft: {current_story}
    Critique: {critique}

    Your task is to analyze the critique.
    - IF the critique is EXACTLY 'APPROVED', you MUST call the 'exit_loop' function and nothing else.
    - OTHERWISE, rewrite the story draft to fully incorporate the feedback from the critique.",
    output_key="current_story", # It overwrites the story with the new, refined version.
    tools=[
        FunctionTool(exit_loop)
    ], # The tool is now correctly initialized with the function reference.
)

print('refiner_agent created.')
```

refiner_agent created.

+ Code + Markdown

Then we bring the agents together under a loop agent, which is itself nested inside of a sequential agent.

This design ensures that the system first produces an initial story draft, then the refinement loop runs up to the specified number of `max_iterations`:

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_8 = user_secrets.get_secr
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 1b - Agent Architectures

Section 6: Summary - Choosing the Right Pattern

Decision Tree: Which Workflow Pattern?

```

graph TD
    Root{What kind of workflow do you need?} --> Fixed[Fixed Pipeline  
(A → B → C)]
    Root --> Concurrent[Concurrent Tasks  
(Run A, B, C all at once)]
    Root --> Iterative[Iterative Refinement  
(A ↔ B)]
    Root --> Dynamic[Dynamic Decisions  
(Let the LLM decide what to do)]
    Fixed --> Sequential[Use SequentialAgent]
    Concurrent --> Parallel[Use ParallelAgent]
    Iterative --> Loop[Use LoopAgent]
    Dynamic --> LLM[Use LLM Orchestrator  
(Agent with other agents as tools)]
  
```

Code Snippet:

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

Con esto finalizamos el día número 1.

Day 1b - Agent Architectures

Quick Reference Table

Pattern	When to Use	Example	Key Features
LLM-based (sub_agents)	Dynamic orchestration needed	Research + Summarize	LLM decides what to call
Sequential	Order matters, linear pipeline	Outline → Write → Edit	Deterministic order
Parallel	Independent tasks, speed matters	Multi-topic research	Concurrent execution
Loop	Iterative improvement needed	Writer + Critic refinement	Repeated cycles

🎉 Congratulations! You're Now an Agent Orchestrator

In this notebook, you made the leap from a single agent to a **multi-agent system**.

You saw **why** a team of specialists is easier to build and debug than one "do-it-all" agent. Most importantly, you learned how to be the **director** of that team.

You used `SequentialAgent`, `ParallelAgent`, and `LoopAgent` to create deterministic workflows, and you even used an LLM as a "manager" to make dynamic decisions. You also mastered the "plumbing" by using `output_key` to pass state between agents and make them collaborative.

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

[Learn More](#)

Code Snippet:

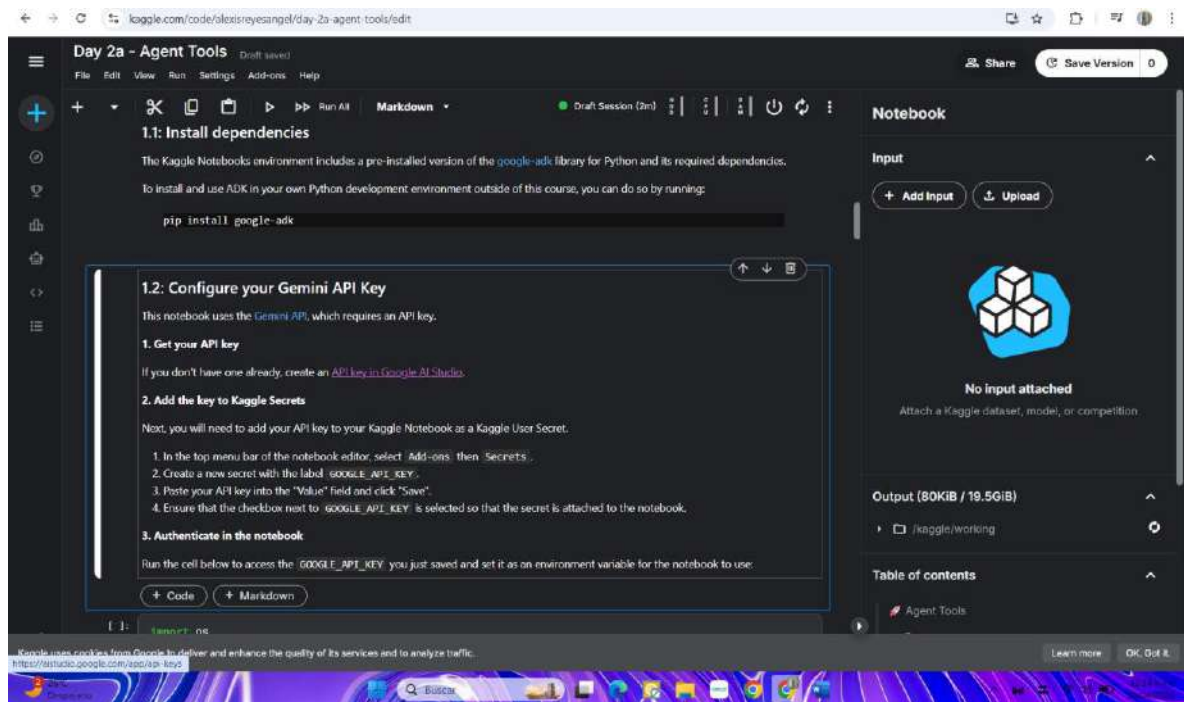
```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

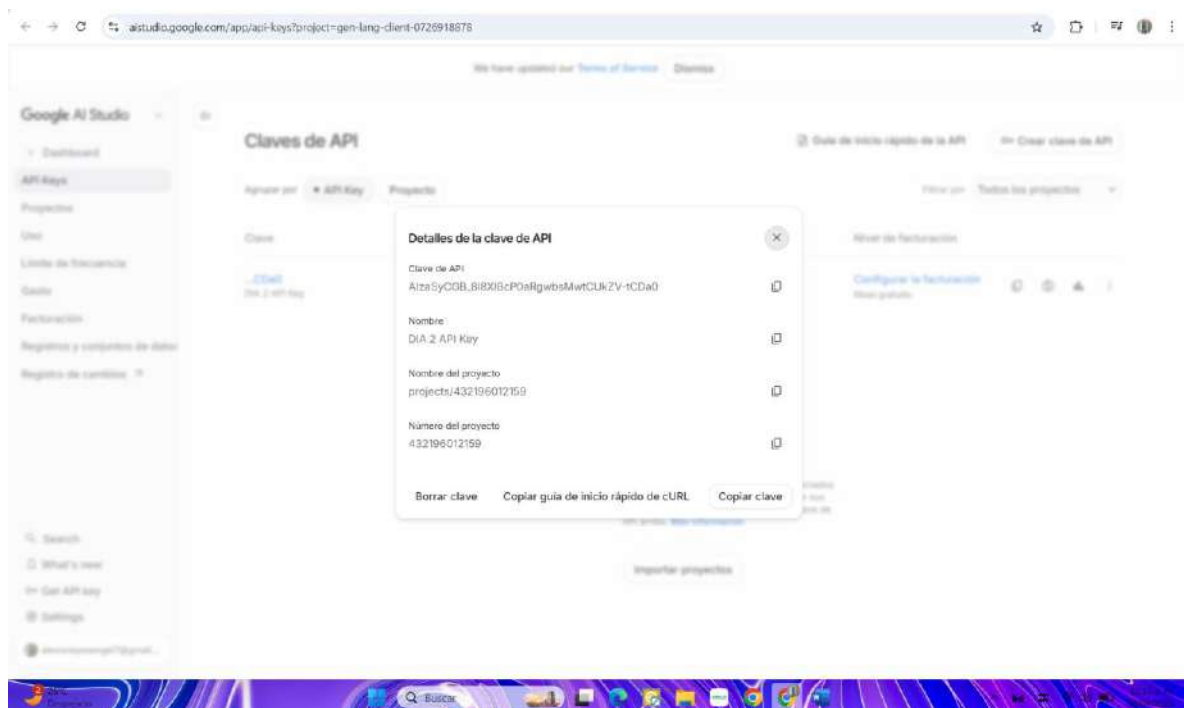
GOOGLE_API_KEY

Curso KAGGLE – DÍA 2

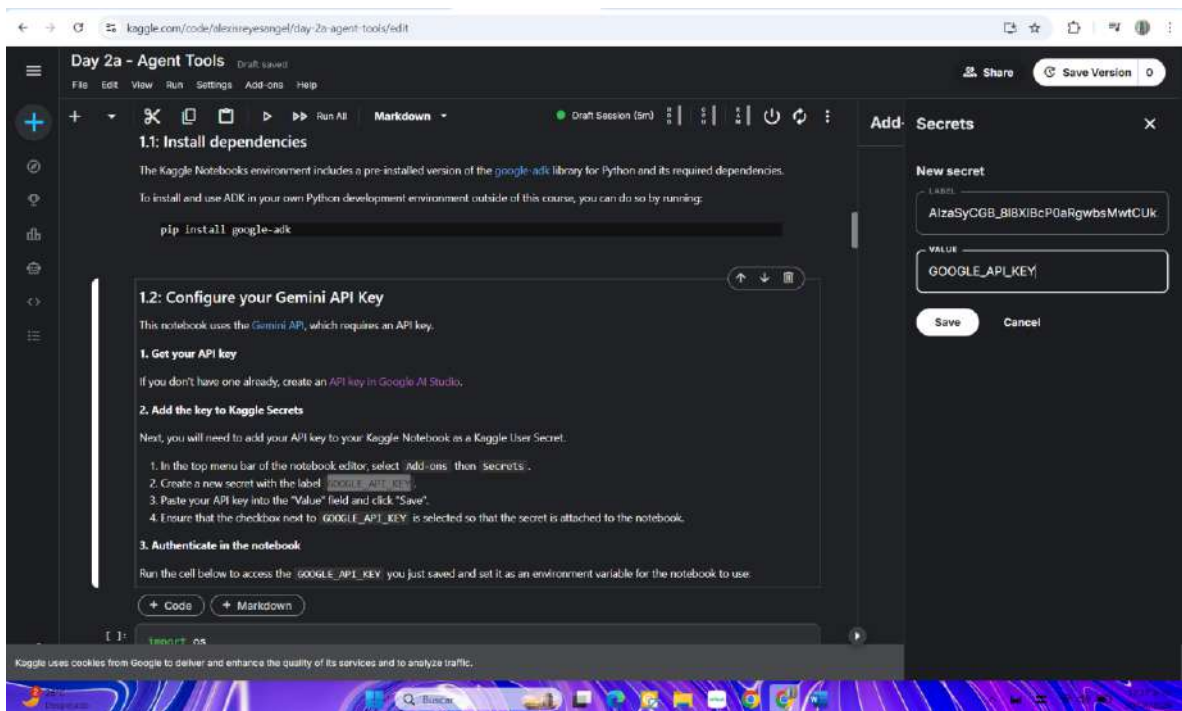
Crearemos una nueva clave API ya que es un nuevo día del curso.



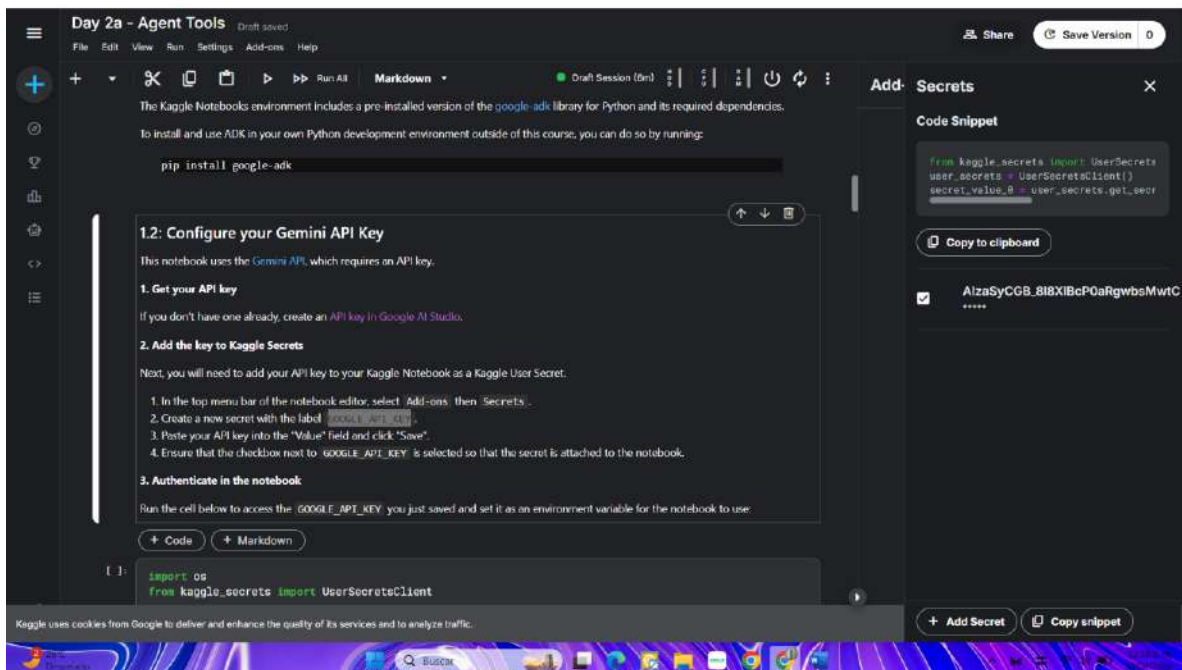
Respondemos lo que nos pide y ahora sí, hemos creado nuestra nueva clave API para este nuevo proyecto del día 2.



Al igual que el día 1, añadiremos nuestra clave API creada a nuestro nuevo proyecto.



Listo, ya hemos añadido nuestra clave API al proyecto nuevo.



Day 2a - Agent Tools

3. Paste your API key into the "Value" field and click "Save".

4. Ensure that the checkbox next to "GOOGLE_API_KEY" is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to access the "GOOGLE_API_KEY" you just saved and set it as an environment variable for the notebook to use.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print('✅ Setup and authentication complete.')
except Exception as e:
    print(f'❌ Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets')
    print(f'Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Unexpected response from the service. Response: {'error': ['No user secrets exist for kernel id 117348688 and label GOOGLE_API_KEY.'], 'error': {'code': 5}, 'wasSuccessful': False}.
```

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

Añadimos los componentes.

Day 2a - Agent Tools

```
except Exception as e:
    print(f'❌ Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets')
    print(f'Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Unexpected response from the service. Response: {'error': ['No user secrets exist for kernel id 117348688 and label GOOGLE_API_KEY.'], 'error': {'code': 5}, 'wasSuccessful': False}.
```

✅ Setup and authentication complete.

```
from google.genai import types

from google.adk.agents import LiteAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search, AgentTool, ToolContext
from google.adk.code_executors import BuiltInCodeExecutor

print('✅ ADK components imported successfully.')

ADK components imported successfully.
```

1.4: Helper functions

Helper function that prints the generated Python code and results from the code execution tool

```
def show_python_code_and_result(response):
    for i in range(len(response)):
```

GOOGLE_API_KEY

Definimos las funciones

The screenshot shows the Kaggle Day 2a - Agent Tools interface. The main editor displays a Python function named `show_python_code_and_result` that processes a list of responses. The function checks if a response contains a valid function call and prints the generated Python code and the response. Below the code editor, there is a section titled "1.5: Configure Retry Options" with a brief explanation of retry options. On the right side, there is a "Secrets" panel with a "Code Snippet" and a list of secrets, including "GOOGLE_API_KEY".

```
def show_python_code_and_result(response):
    for i in range(len(response)):
        # Check if the response contains a valid function call result from the code executor
        if (
            (response[i].content.parts
             and (response[i].content.parts[0]
                  and (response[i].content.parts[0].function_response
                       and (response[i].content.parts[0].function_response.response)
                      )
                )
        ):
            response_code = response[i].content.parts[0].function_response.response
            if "result" in response_code and response_code["result"] != "":
                if "tool_code" in response_code["result"]:
                    print(
                        "Generated Python Code >> ",
                        response_code["result"].replace("tool_code", ""),
                    )
                else:
                    print("Generated Python Response >> ", response_code["result"])

    print("Helper functions defined.")
```

Helper functions defined.

1.5: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

Ejecutamos las opciones.

The screenshot shows the Kaggle Day 2a - Agent Tools interface after executing the helper function. The main editor displays the output "Helper functions defined." and the "1.5: Configure Retry Options" section. Below this, there is a code snippet for configuring retry options. On the right side, the "Secrets" panel is visible, showing the "GOOGLE_API_KEY" secret.

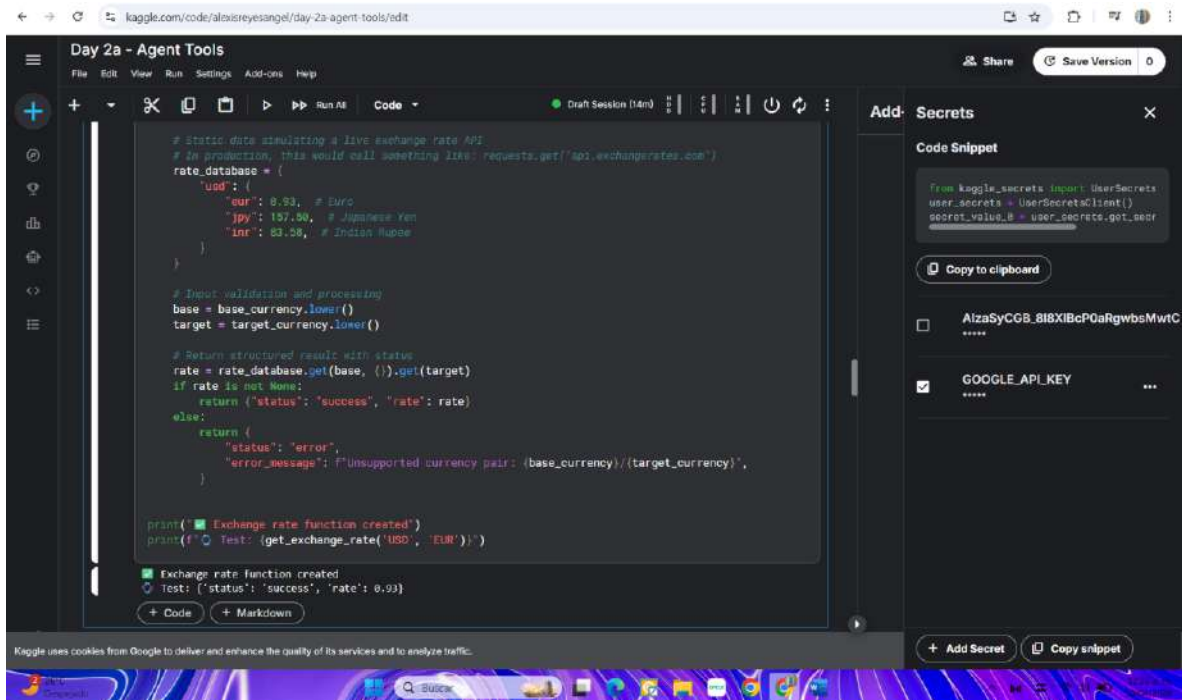
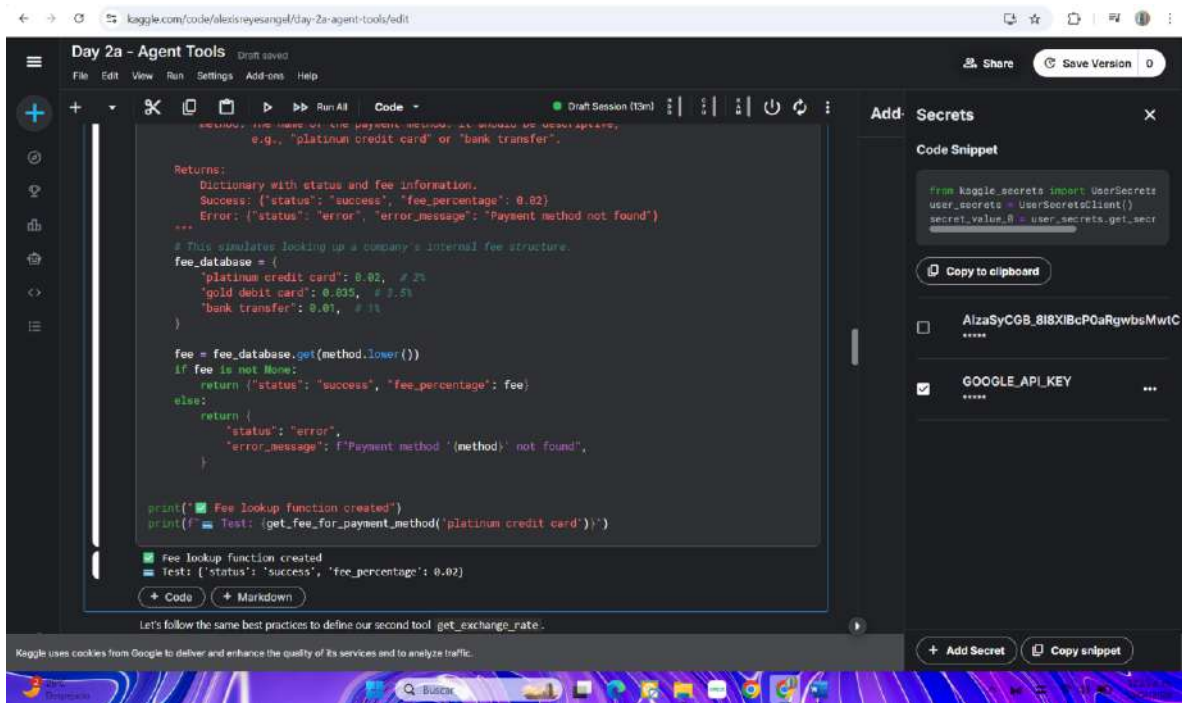
```
retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=2, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

Section 2: What are Custom Tools?

Custom Tools are tools you build yourself using your own code and business logic. Unlike built-in tools that come ready-made with ADK, custom tools give you complete control over functionality.

When to use Custom Tools?

Built-in tools like Google Search are powerful, but **every business has unique requirements** that generic tools can't handle. Custom tools let you implement your specific business logic, connect to your systems, and solve domain-specific problems. ADK provides multiple custom tool hooks to handle these scenarios.



loggle.com/code/alexisreyesangel/day-2a-agent-tools/edit

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (14m)

```
# Currency agent with custom function tools
currency_agent = LLMAgent(
    name='currency_agent',
    model=Gemini(model='gemini-2.5-flash-lite', retry_options=retry_config),
    instruction="You are a smart currency conversion assistant.

    For currency conversion requests:
    1. Use 'get_fee_for_payment_method()' to find transaction fees
    2. Use 'get_exchange_rate()' to get currency conversion rates
    3. Check the 'status' field in each tool's response for errors
    4. Calculate the final amount after fees based on the output from 'get_fee_for_payment_method' and
    5. First, state the final converted amount.
    Then, explain how you got that result by showing the intermediate amounts. Your explanation must
    value in the original currency, the amount remaining after the fee, and the exchange rate used f

    If any tool returns status 'error', explain the issue to the user clearly.
    ...
    tools=[get_fee_for_payment_method, get_exchange_rate],
)

print('Currency agent created with custom function tools')
print('Available tools:')
print('  - get_fee_for_payment_method - Looks up company fee structure')
print('  - get_exchange_rate - Gets current exchange rates')
```

Code Markdown

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

loggle.com/code/alexisreyesangel/day-2a-agent-tools/edit

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (14m)

```
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run.debug(
    "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

## Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?

WARNING:google.gemini.types.warning: there are non-text parts in the response: ['function_call', 'function_call'],
returning concatenated text result from text parts. check the full candidates.content.parts accessor to get the full
model response.
WARNING:google.gemini.types.warning: there are non-text parts in the response: ['function_call'], returning concate
nated text result from text parts. check the full candidates.content.parts accessor to get the full model respons
e.
WARNING:google.gemini.types.warning: there are non-text parts in the response: ['function_call'], returning concate
nated text result from text parts. check the full candidates.content.parts accessor to get the full model respons
e.
currency_agent > You will receive 455.7 EUR.

Here's how that's calculated:

1. **Fee:** A 2.0% fee is charged for using a platinum credit card. This amounts to 10.0 USD (500 USD * 0.02).
2. **Amount after fee:** After the fee, you have 490.0 USD remaining (500 USD - 10.0 USD).
3. **Exchange:** With an exchange rate of 0.93, 490.0 USD is converted to 455.7 EUR (490.0 USD * 0.93).
```

Code Markdown

Excellent! Our agent now uses custom business logic with structured responses.

Section 3: Improving Agent Reliability with Code

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

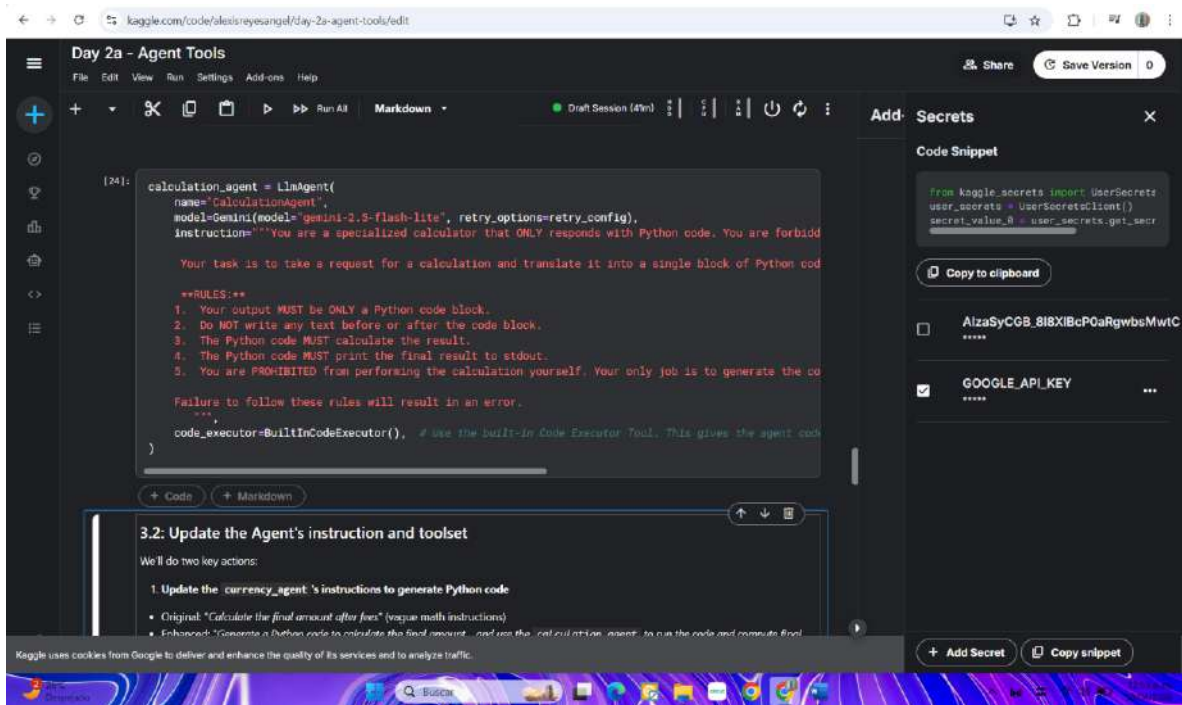
Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

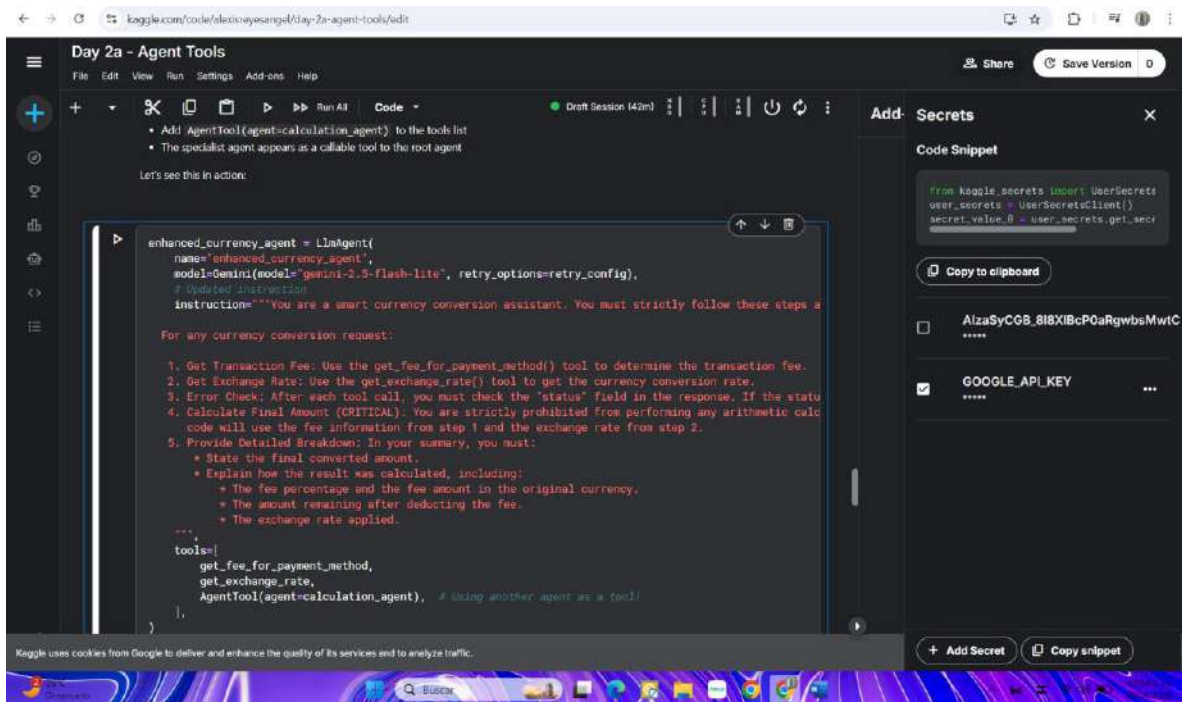
GOOGLE_API_KEY

Add Secret Copy snippet

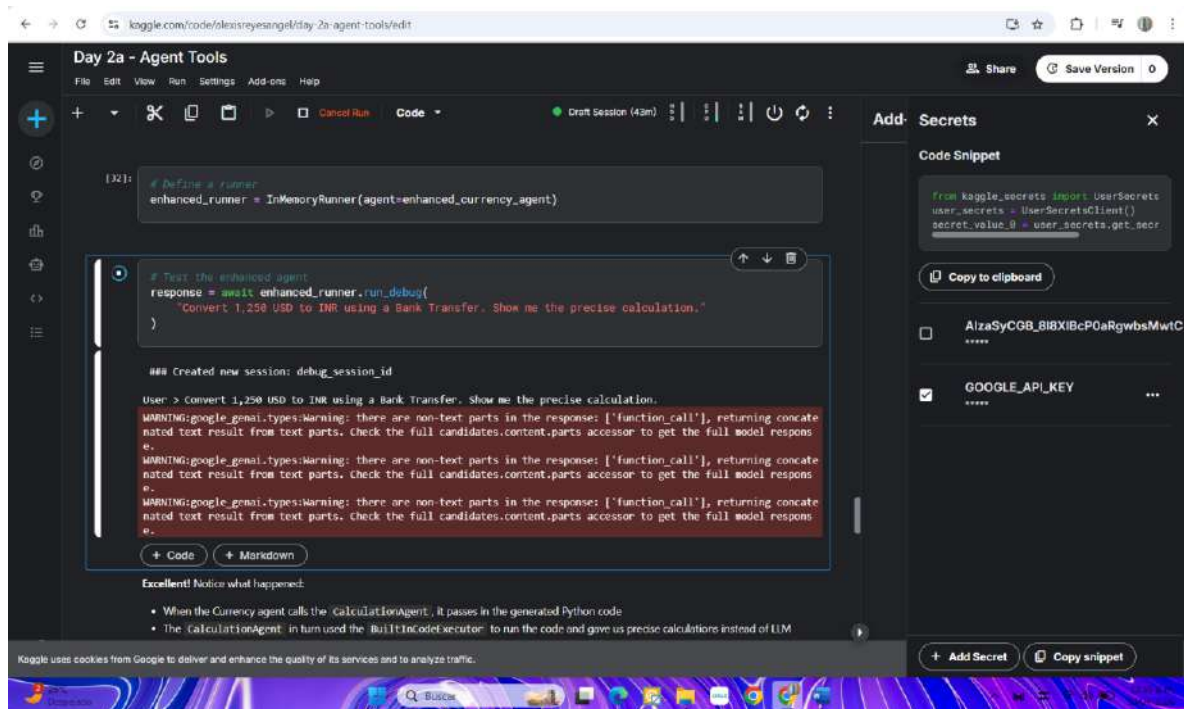
Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.



Definimos el ejecutor



Ejecutamos el segundo ejecutor.



Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Code

Draft Session (43m)

1321

```
# Define a runner
enhanced_runner = InMemoryRunner(agent=enhanced_currency_agent)
```

```
# Test the enhanced agent
response = await enhanced_runner.run_debug(
    "Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation."
)
```

```
### Created new session: debug_session_id
```

User > Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.

WARNING:google.gemini.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenate text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

WARNING:google.gemini.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenate text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

WARNING:google.gemini.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenate text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

+ Code + Markdown

Excellent! Notice what happened:

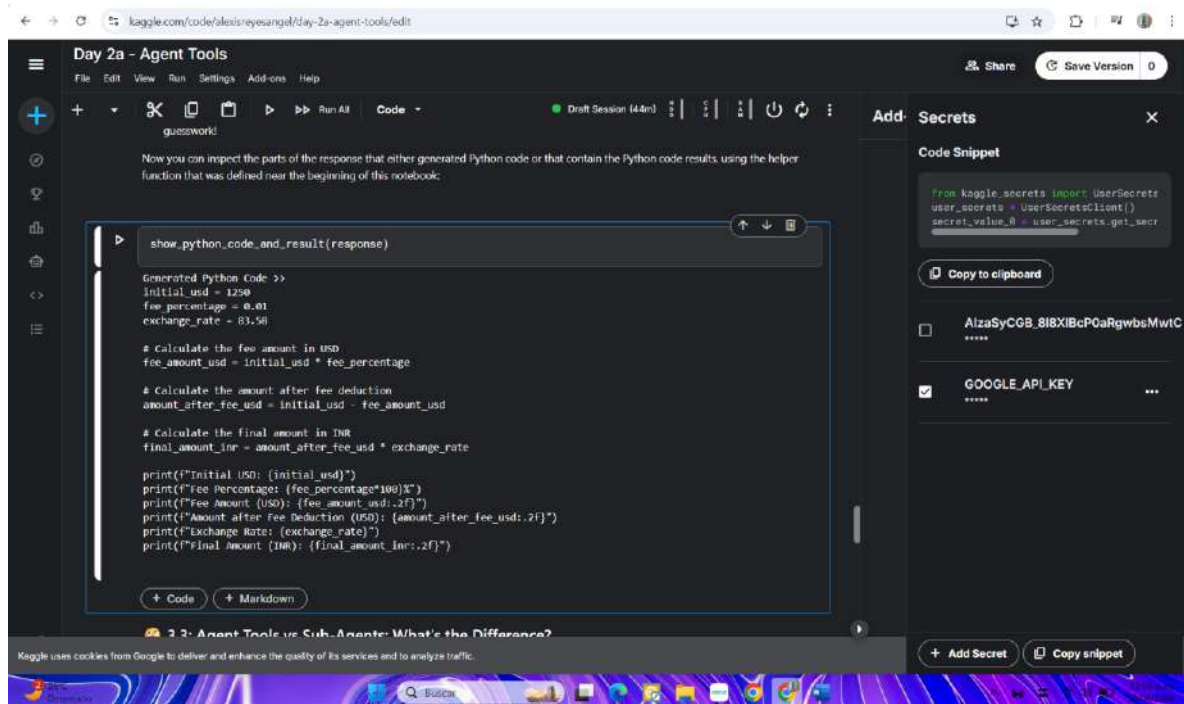
- When the Currency agent calls the CalculationAgent, it passes in the generated Python code
- The CalculationAgent in turn used the BuiltInCodeExecutor to run the code and give us precise calculations instead of LLM

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret + Copy snippet



Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Code

Draft Session (44m)

guesswork!

Now you can inspect the parts of the response that either generated Python code or that contain the Python code results using the helper function that was defined near the beginning of this notebook:

```
show_python_code_and_result(response)
```

```
Generated Python Code >>
initial_usd = 1250
fee_percentage = 0.01
exchange_rate = 83.58

# Calculate the fee amount in USD
fee_amount_usd = initial_usd * fee_percentage

# Calculate the amount after fee deduction
amount_after_fee_usd = initial_usd - fee_amount_usd

# Calculate the final amount in INR
final_amount_inr = amount_after_fee_usd * exchange_rate

print(f"Initial USD: {initial_usd}")
print(f"Fee percentage: {fee_percentage*100}%")
print(f"Fee Amount (USD): {fee_amount_usd:.2f}")
print(f"Amount after Fee Deduction (USD): {amount_after_fee_usd:.2f}")
print(f"Exchange Rate: {exchange_rate}")
print(f"Final Amount (INR): {final_amount_inr:.2f}")
```

+ Code + Markdown

2.2: Agent Tools vs Sub-Agents: What's the Difference?

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret + Copy snippet

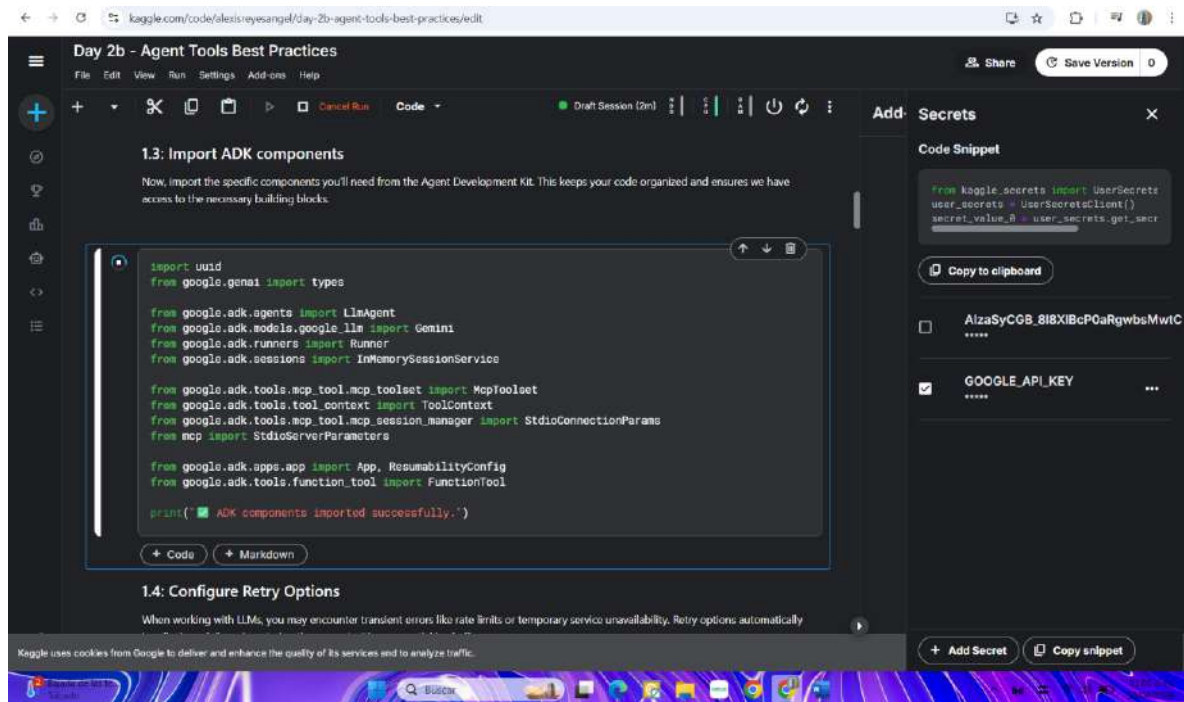
Terminamos la tarea 1 del día 2.

The screenshot shows the Kaggle notebook interface for 'Day 2a - Agent Tools'. The notebook is in 'Draft' mode and has been saved. The main content area displays a 'Congratulations!' message, indicating successful completion of the task. It lists the topics covered: Function Tools, Agent Tools, and Complete Toolkit. A note states that no submission is required. The 'Learn More' section provides links to ADK documentation. The 'Next Steps' section encourages further learning. On the right, the 'Add Secrets' panel is open, showing a code snippet for importing secrets and a list of secrets, including 'GOOGLE_API_KEY' which is checked. The bottom of the screen shows a Windows taskbar with various application icons.

Día 2B

The screenshot shows the Kaggle notebook interface for 'Day 2b - Agent Tools Best Practices'. The notebook is in 'Draft' mode and has been saved. The main content area displays a code cell with Python code for setting up the environment and authenticating with the Google API Key. The code includes imports for 'os' and 'kaggle_secrets', and a function to set the 'GOOGLE_API_KEY' as an environment variable. The output of the code cell shows 'Setup and authentication complete.' The 'Add Secrets' panel on the right is open, showing the same code snippet as in the previous screenshot, and the 'GOOGLE_API_KEY' secret is checked. The bottom of the screen shows a Windows taskbar with various application icons.

Importamos los componentes ADK



Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit. This keeps your code organized and ensures we have access to the necessary building blocks.

```
import uuid
from google.genai import types

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

from google.adk.tools.mcp_tool.mcp_toolset import McpToolset
from google.adk.tools.tool_context import ToolContext
from google.adk.tools.mcp_tool.mcp_session_manager import StudioConnectionParams
from mcp import StudioServerParameters

from google.adk.apps.app import App, ResumabilityConfig
from google.adk.tools.function_tool import FunctionTool

print('✅ ADK components imported successfully.')
```

+ Code + Markdown

1.4: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

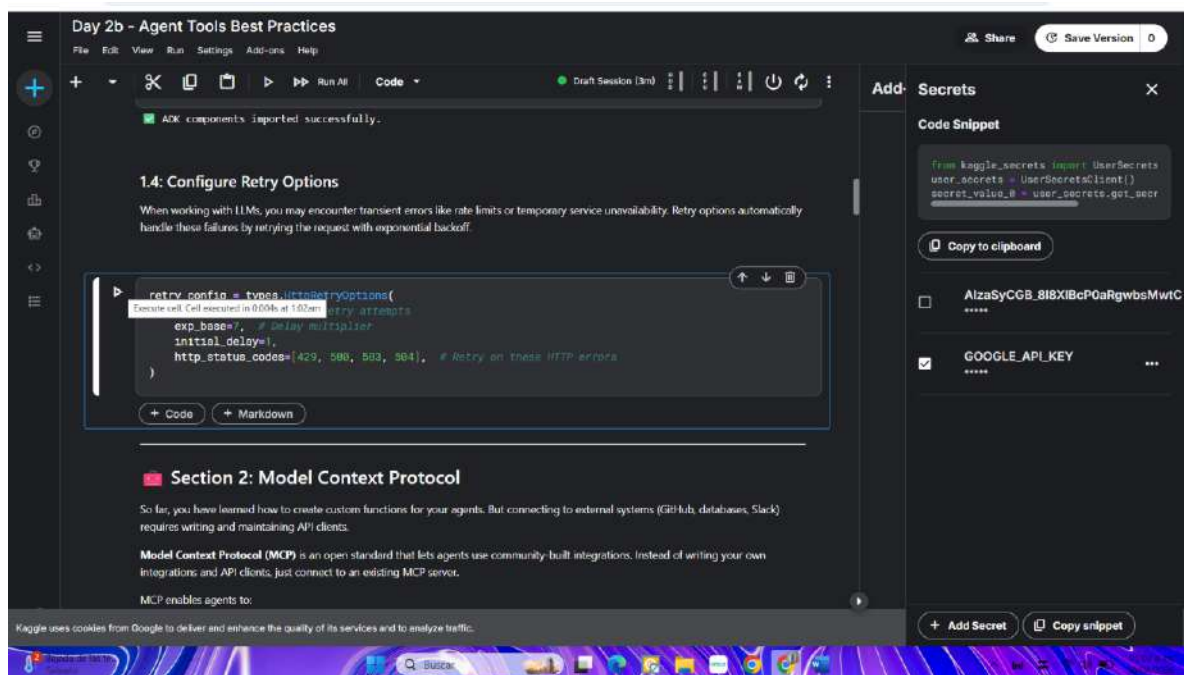
```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet



Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

1.4: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

```
retry_config = types.HttpRetryOptions(
    # Retry attempts
    exp_backoff = 2,
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

+ Code + Markdown

Section 2: Model Context Protocol

So far, you have learned how to create custom functions for your agents. But connecting to external systems (GitHub, databases, Slack) requires writing and maintaining API clients.

Model Context Protocol (MCP) is an open standard that lets agents use community-built integrations. Instead of writing your own integrations and API clients, just connect to an existing MCP service.

MCP enables agents to:

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

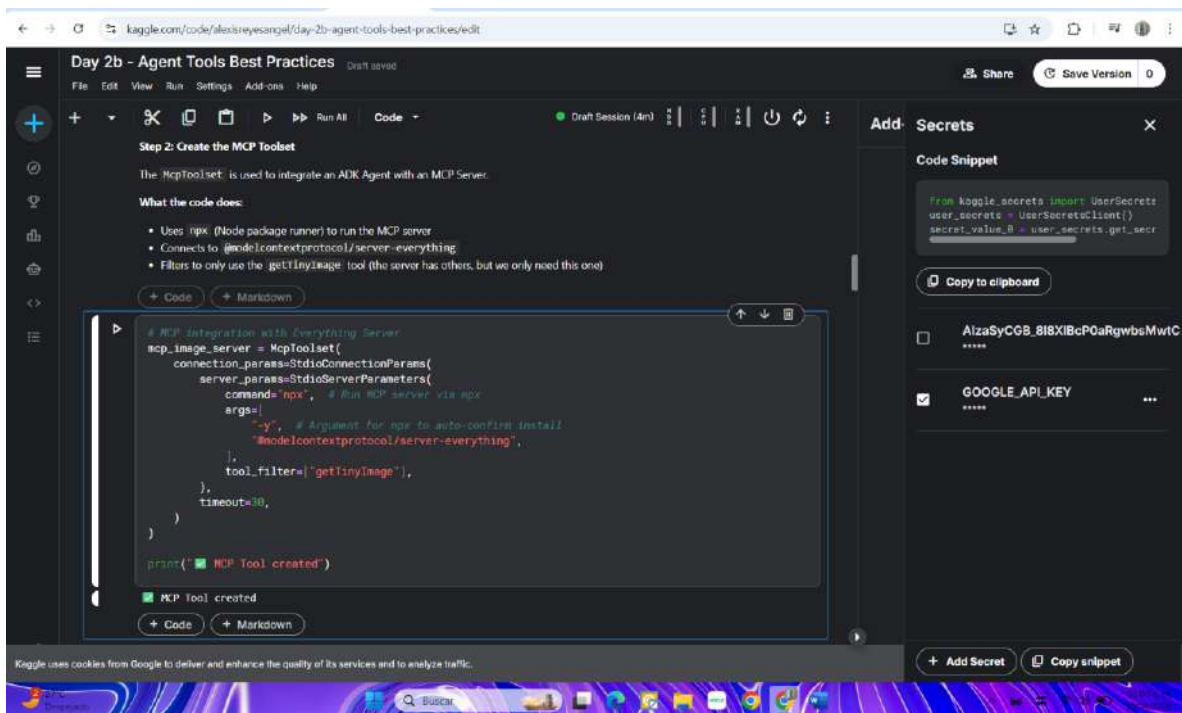
Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Creamos la consulta MCP



The screenshot shows a Kaggle notebook interface. The title is "Day 2b - Agent Tools Best Practices". The code cell is titled "Step 2: Create the MCP Toolset". It contains a Python code snippet that defines an MCP toolset for an image server. The code uses the `McpToolset` class from the `mcp` library. It sets the connection parameters to a local server, the server parameters to a specific command, and the tool filter to "getTinyImage". The code also sets a timeout of 30 seconds. After running the code, a message "MCP Tool created" is displayed.

```
# MCP Integration with Everything Server
mcp_image_server = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command="npx", # Run MCP server via npx
            args=[
                "-y", # Argument for npx to auto-confirm install
                "@modelcontextprotocol/server-everything",
            ],
        ),
        tool_filter="getTinyImage",
    ),
    timeout=30,
)

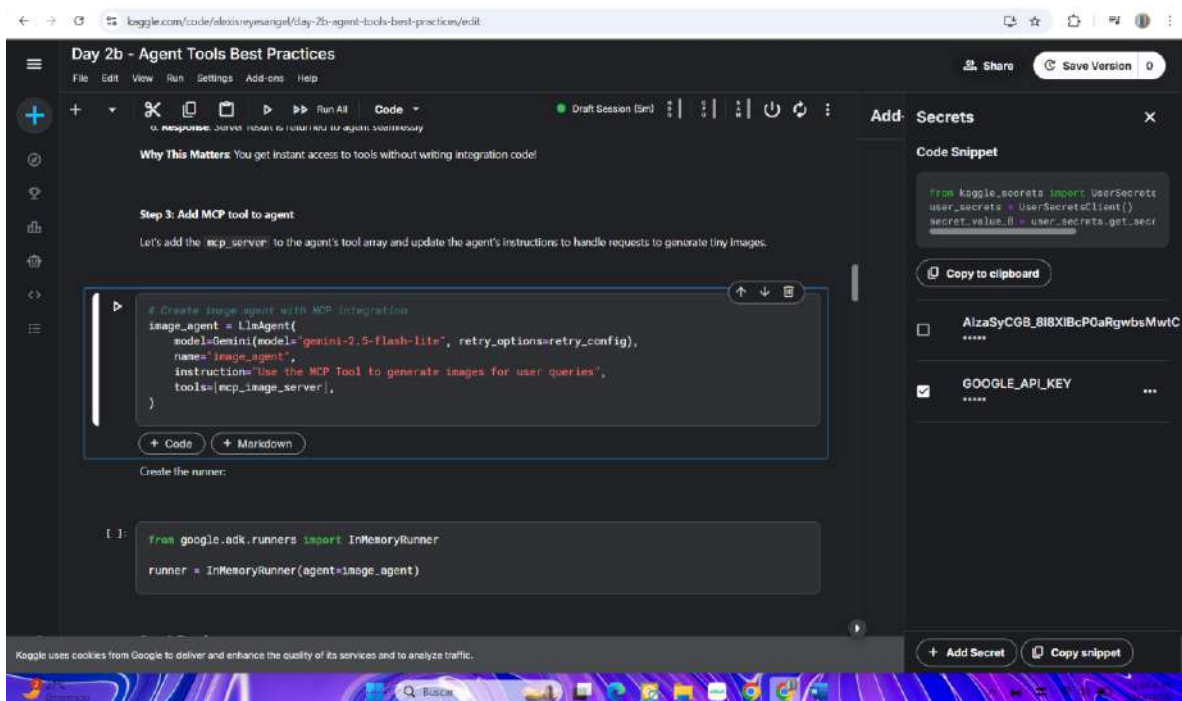
print("MCP Tool created")
```

The right sidebar shows the "Add Secrets" panel with a code snippet for retrieving secrets from Kaggle:

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_8 = user_secrets.get_secret_value_8
```

Below the code snippet, there are two secrets listed: "AlzaSyCG8_8l8XlBcP0aRgwbsMwtC" and "GOOGLE_API_KEY". The "GOOGLE_API_KEY" secret is checked.

Añadimos la herramienta al agente.

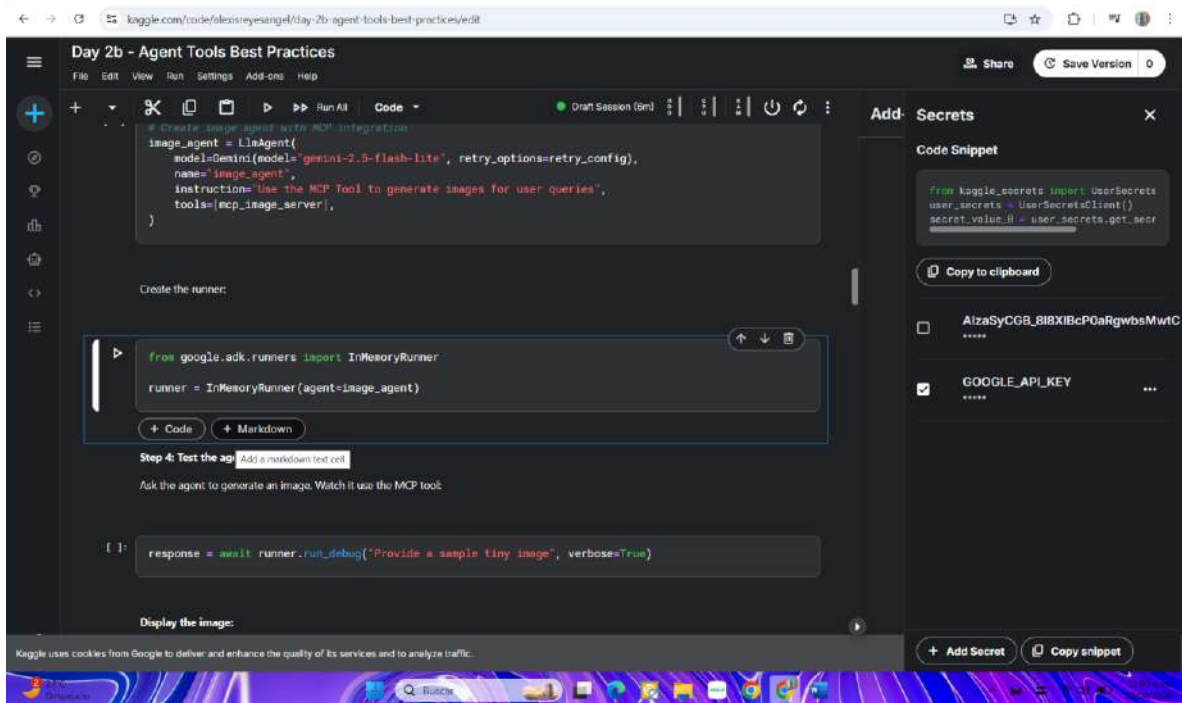


The screenshot shows the same Kaggle notebook interface. The code cell is titled "Step 3: Add MCP tool to agent". It contains a Python code snippet that creates an `LLMAgent` object. The agent is configured with the `model="gemini-2.5-flash-lite"`, the `name="image\_agent"`, and the `instructions="Use the MCP Tool to generate images for user queries"`. The `tools` list is updated to include the `mcp\_image\_server` object. Below the code snippet, there is a message "Create the runner:" followed by a code cell that defines an `InMemoryRunner` object for the agent.

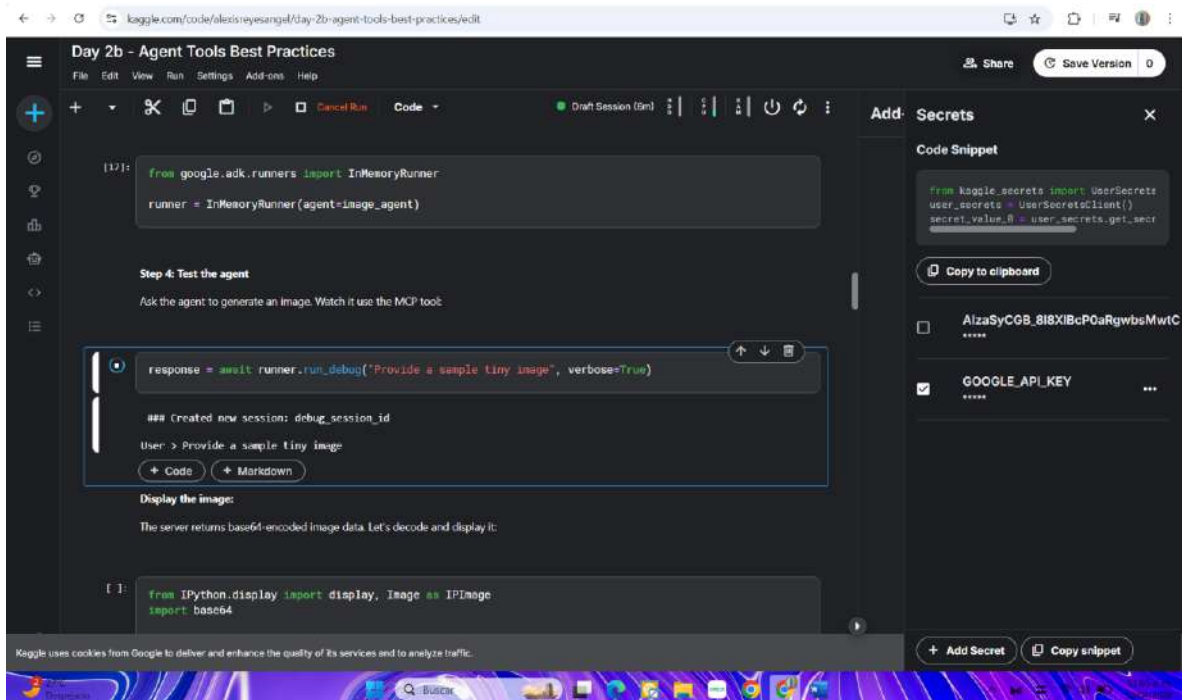
```
# Create image agent with MCP integration
image_agent = LLMAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="image_agent",
    instructions="Use the MCP Tool to generate images for user queries",
    tools=[mcp_image_server],
)

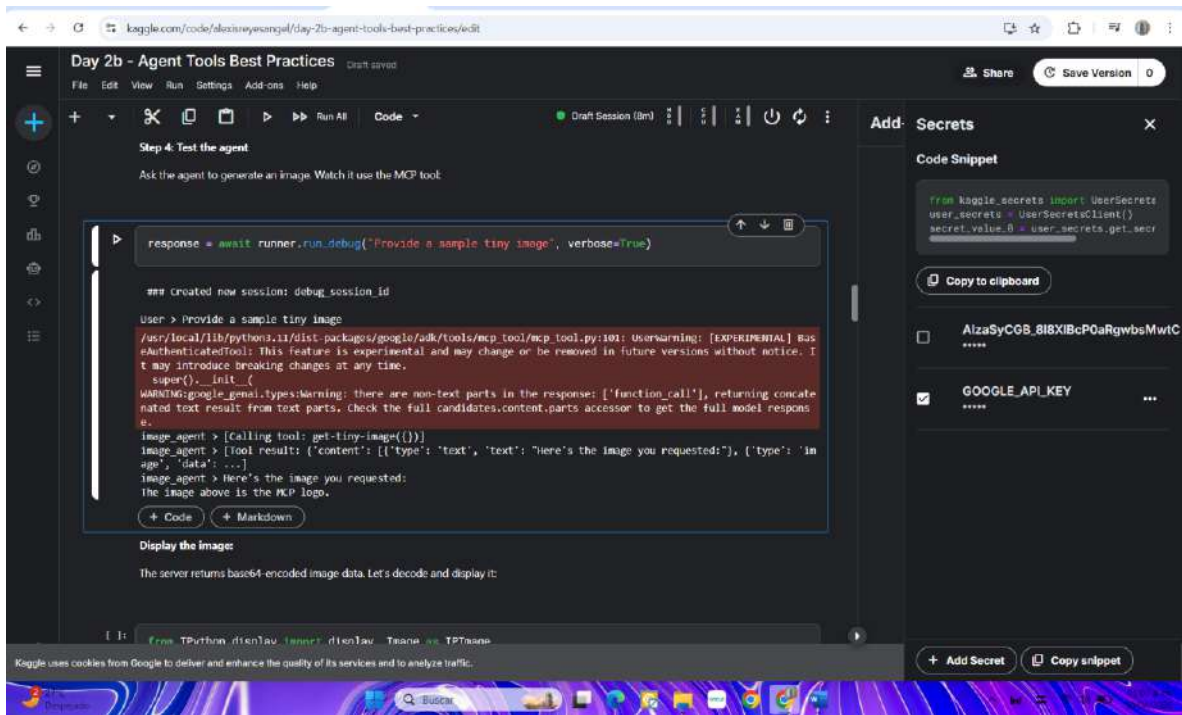
# Create the runner:
runner = InMemoryRunner(agent=image_agent)
```

The right sidebar shows the "Add Secrets" panel with the same code snippet as in the previous screenshot. The "GOOGLE_API_KEY" secret is checked.

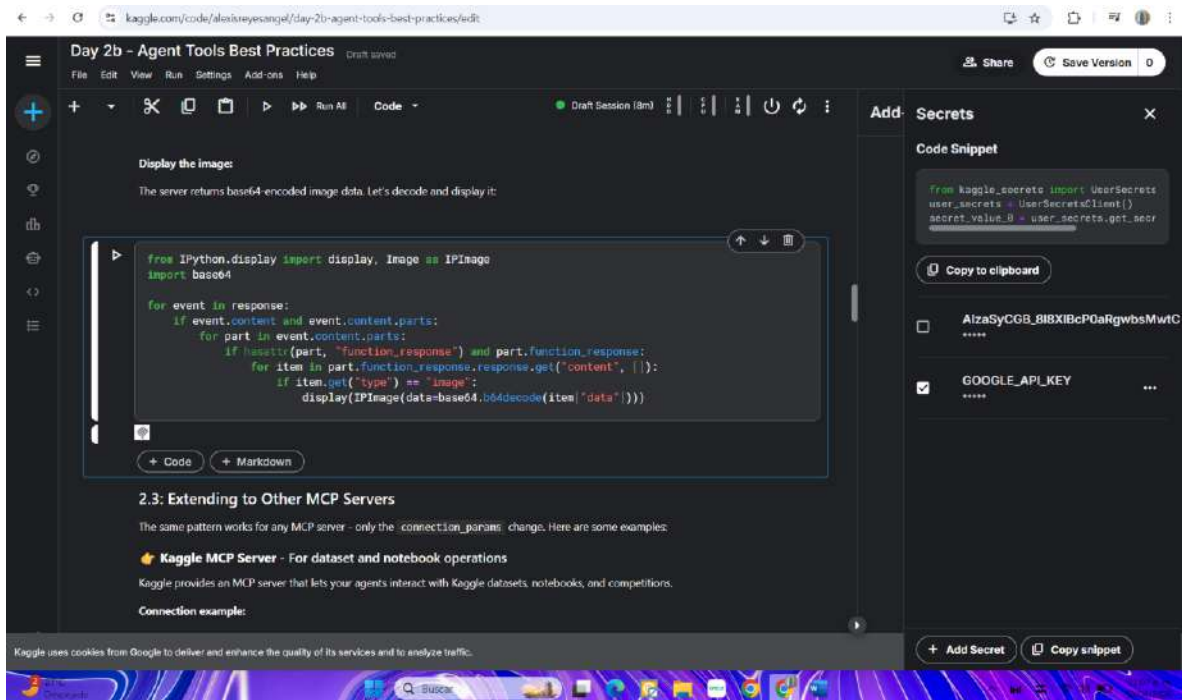


Probamos el agente para saber si genera la imagen





En esta parte nos genera la imagen que habimos ejecutado.



Day 2b - Agent Tools Best Practices

```
if tool_context.tool_confirmation.confirmed:
    return {
        "status": "approved",
        "order_id": f"ORD-{num_containers}-HUMAN",
        "num_containers": num_containers,
        "destination": destination,
        "message": f"Order approved: {num_containers} containers to {destination}",
    }
else:
    return {
        "status": "rejected",
        "message": f"Order rejected: {num_containers} containers to {destination}",
    }

print("🟢 Long-running functions created!")
```

Long-running functions created!

3.3: Understanding the Code

Now that you've seen the complete function, let's break down how it works.

if num_containers <= 5

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Creamos el agente

Day 2b - Agent Tools Best Practices

3.4: Create the Agent, App and Runner

Step 1: Create the agent

Add the tool to the Agent. The tool decides internally when to request approval based on the order size.

```
# Create shipping agent with pausable tool
shipping_agent = LLMAgent(
    name="shipping_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="You are a shipping coordinator assistant.",

    When users request to ship containers:
    1. Use the place_shipping_order tool with the number of containers and destination
    2. If the order status is 'pending', inform the user that approval is required
    3. After receiving the final result, provide a clear summary including:
       - Order status (approved/rejected)
       - Order ID (if available)
       - Number of containers and destination
    4. Keep responses concise but informative
    ...
    tools=[FunctionTool(func=place_shipping_order)],
)

print("🟢 Shipping Agent created!")
```

Shipping Agent created!

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (1m)

Share Save Version 0

- All conversation messages so far
- Which tool was called (place_shipping_order)
- Tool parameters (10 containers, Rotterdam)
- Where exactly it paused (waiting for approval)

When you resume, the App loads this saved state so the agent continues exactly where it left off - as if no time passed.

```
# Run the agent in a resumable app - THIS IS THE KEY FOR LONG-RUNNING OPERATIONS!
shipping_app = App(
    name="shipping_coordinator",
    root_agent=shipping_agent,
    resumability_config=ResumabilityConfig(is_resumable=True),
)

print('✅ Resumable app created!')
```

Resumable app created!

/tmp/ipykernel_49/362377575.py:5: UserWarning: [EXPERIMENTAL] ResumabilityConfig: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

resumability_config=ResumabilityConfig(is_resumable=True),

+ Code + Markdown

Step 3: Create Session and Runner with the App

Pass app=shipping_app instead of agent=... so the runner knows about resumability.

```
[ ]: session_service = InMemorySessionService()
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (10m)

Share Save Version 0

Resumable app created!

```
session_service = InMemorySessionService()

# Create runner with the resumable app
shipping_runner = Runner(
    app=shipping_app, # Pass the app instead of the agent
    session_service=session_service,
)

print('✅ Runner created!')
```

Runner created!

+ Code + Markdown

Step 3: Create Session and Runner with the App

Pass app=shipping_app instead of agent=... so the runner knows about resumability.

Recap: Your pausable shipping agent is now complete!

You've created:

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (10m)

check_for_approval() - Detects if the agent paused

- Loops through all events and looks for the special 'adk_request_confirmation' event
- Returns approval_id (identifies this specific request) and invocation_id (identifies which execution to resume)
- Returns None if no pause detected

```
def check_for_approval(events):  
    """Check if events contain an approval request.  
  
    Returns:  
        dict with approval details or None  
    """  
    for event in events:  
        if event.content and event.content.parts:  
            for part in event.content.parts:  
                if part.function_call and part.function_call.name == "adk_request_confirmation":  
                    return {  
                        "approval_id": part.function_call.id,  
                        "invocation_id": event.invocation_id,  
                    }  
    return None
```

+ Code + Markdown

print_agent_response() - Displays agent text

- Simple helper to extract and print text from events

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (11m)

print_agent_response() - Displays agent text

- Simple helper to extract and print text from events

```
def print_agent_response(events):  
    """Print agent's text responses from events."""  
    for event in events:  
        if event.content and event.content.parts:  
            for part in event.content.parts:  
                if part.text:  
                    print("Agent > (part.text)")
```

+ Code + Markdown

create_approval_response() - Formats the human decision

- Takes the approval info and boolean decision (True/False) from the human
- Creates a FunctionResponse that ADK understands
- Wraps it in a Content object to send back to the agent

```
[ ]: def create_approval_response(approval_info, approved):  
    """Create approval response message."""  
    confirmation_response = types.FunctionResponse(  
        id=approval_info["approval_id"],  
        name="adk_request_confirmation",  
        response={"confirmed": approved},  
    )
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (12m)

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

```
def create_approval_response(approval_info, approved):
    """Create approval response in 0.001s as title"""
    confirmation_response = types.FunctionResponse(
        id=approval_info["approval_id"],
        name="ask_request_confirmation",
        response={"confirmed": approved},
    )
    return types.Content(
        roles="user", parts=[types.Part(function_response=confirmation_response)]
    )

print("Helper functions defined")
```

Helper functions defined

+ Code + Markdown

4.4: The Workflow Function - Let's tie it all together!

The `run_shipping_workflow()` function orchestrates the entire approval flow.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (12m)

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

```
def run_shipping_workflow():
    """Run shipping workflow"""
    user_id = "test_user"
    session_id = session_id
    new_message = create_approval_response(
        approval_info, auto_approve
    )
    # Send human decision here
    invocation_id = approval_info["invocation_id"]
    # Critical! Same invocation_id tells ADK to RESUME
    if event.content and event.content.parts:
        for part in event.content.parts:
            if part.text:
                print(f"Agent > {part.text}")
    else:
        # PATH B: If the "ask_request_confirmation" is not present - no approval needed - order completed
        print_agent_response(events)

    print(f"{' ' * 68} \n")

    print("Workflow function ready")
```

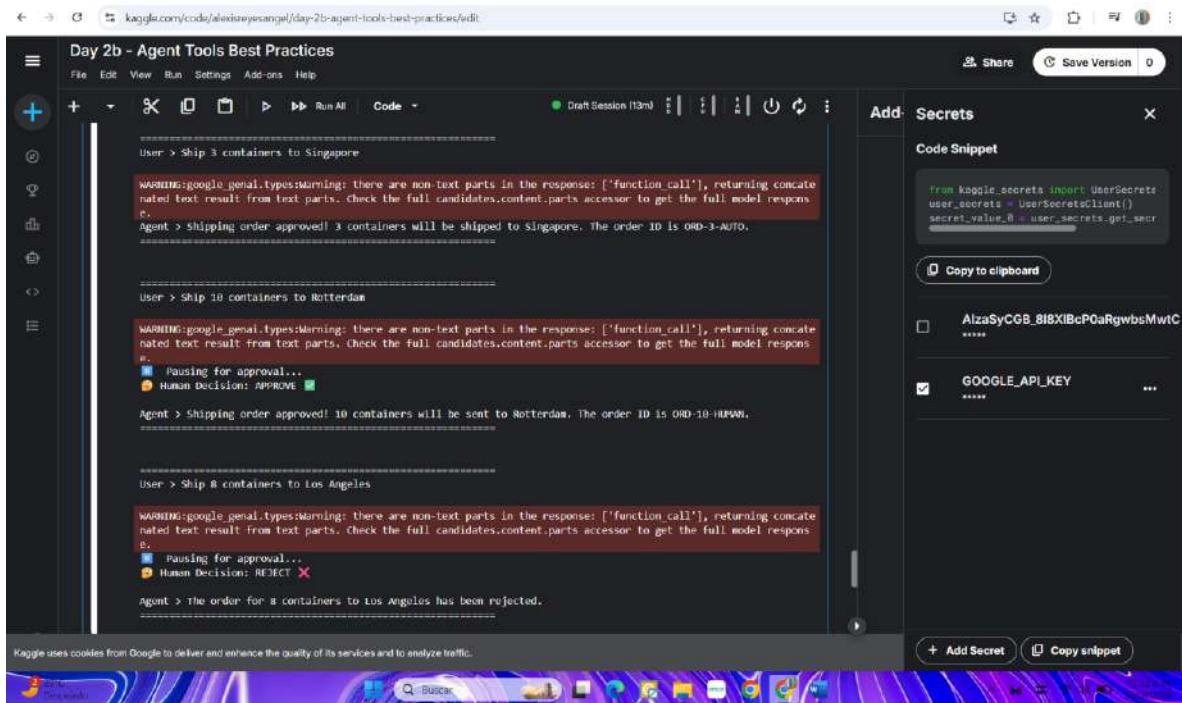
Workflow function ready

+ Code + Markdown

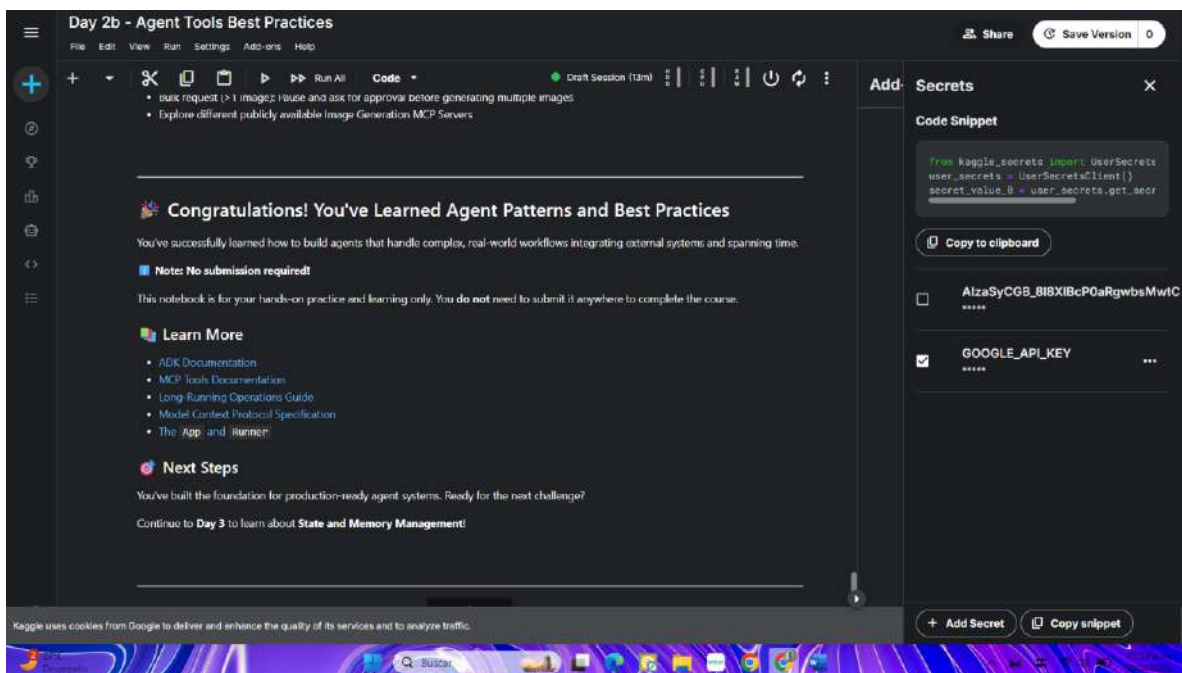
Code breakdown

Step 1: Send initial request to the Agent

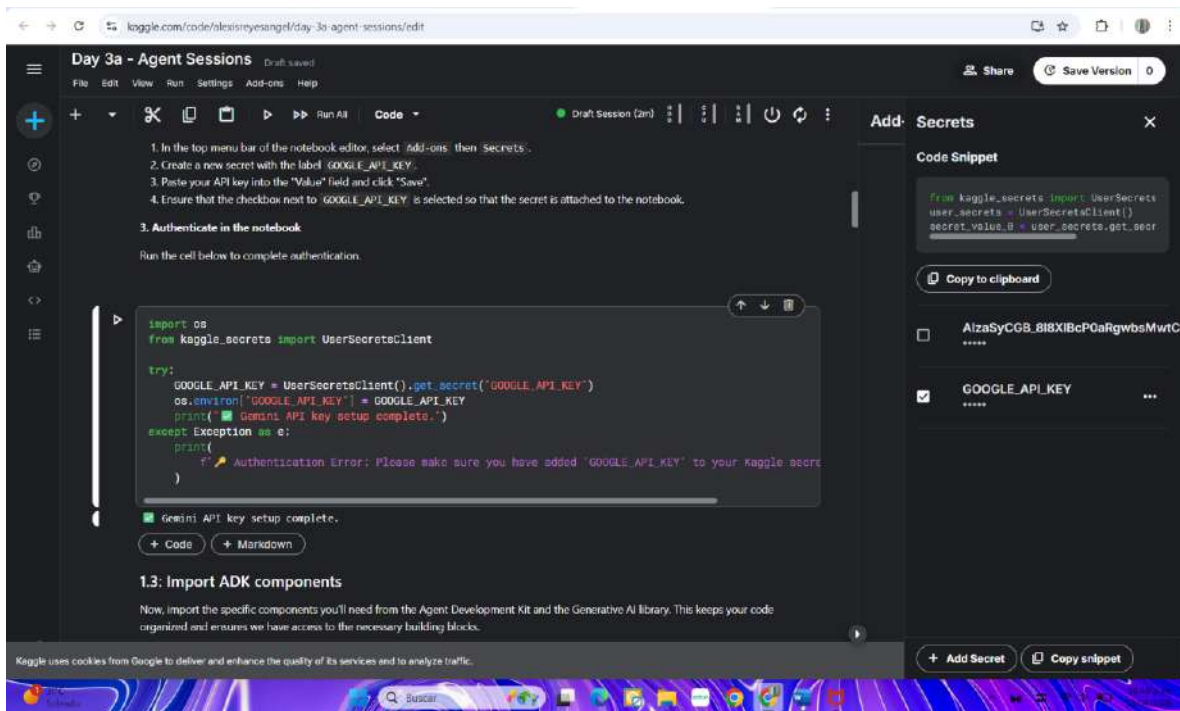
Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.



DIA 2 TERMINADO.



Curso KAGGLE – DÍA 3



The screenshot shows the Kaggle notebook interface for 'Day 3a - Agent Sessions'. The notebook is in 'Draft' mode. The left sidebar contains a table of contents with sections: 1. In the top menu bar of the notebook editor, select 'Add-ons', then 'Secrets'. 2. Create a new secret with the label 'GOOGLE_API_KEY'. 3. Paste your API key into the 'Value' field and click 'Save'. 4. Ensure that the checkbox next to 'GOOGLE_API_KEY' is selected so that the secret is attached to the notebook. 3. Authenticate in the notebook. Run the cell below to complete authentication.

The main code cell contains the following Python code:

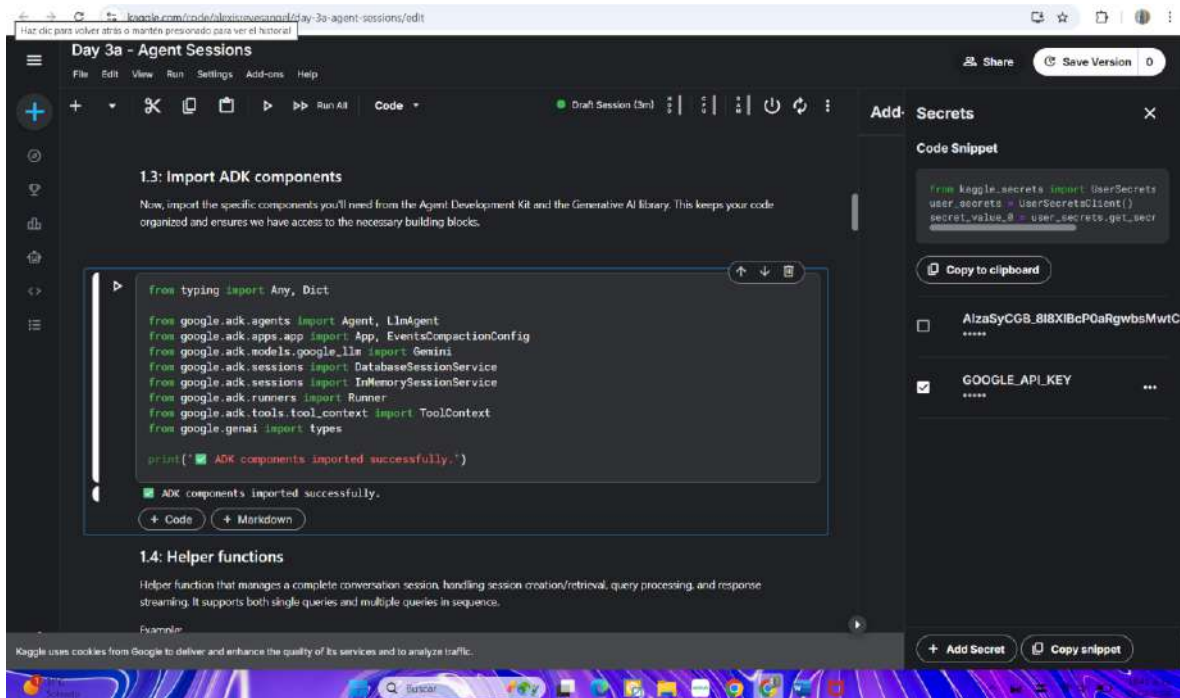
```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print('✅ Gemini API key setup complete.')
except Exception as e:
    print(f'❌ Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets')

# Gemini API key setup complete.
```

The right sidebar shows the 'Add Secrets' panel with a 'Code Snippet' and a list of secrets. The 'GOOGLE_API_KEY' secret is checked.

Ahora importamos los componentes ADK



The screenshot shows the Kaggle notebook interface for 'Day 3a - Agent Sessions'. The notebook is in 'Draft' mode. The left sidebar contains a table of contents with sections: 1.3: Import ADK components. Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

The main code cell contains the following Python code:

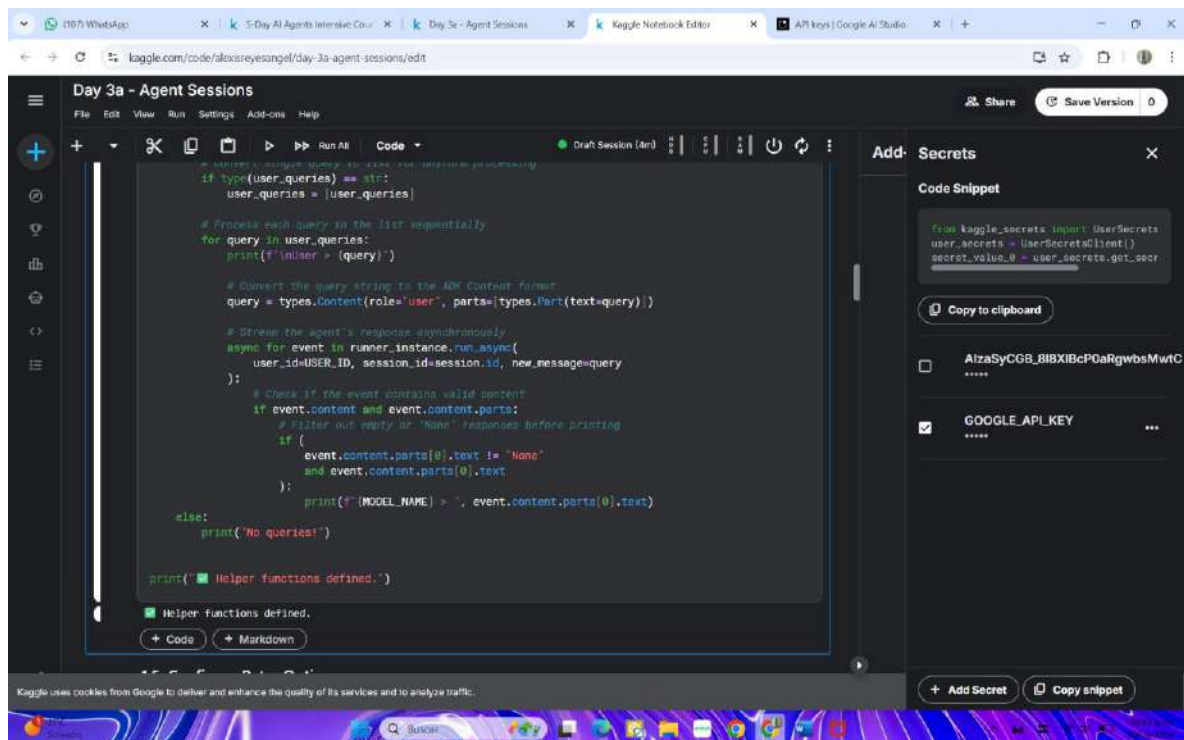
```
from typing import Any, Dict

from google.adk.agents import Agent, LinkAgent
from google.adk.apps.app import App, EventsCompactionConfig
from google.adk.models.google_llm import Gemini
from google.adk.sessions import DatabaseSessionService
from google.adk.sessions import InMemorySessionService
from google.adk.runners import Runner
from google.adk.tools.tool_context import ToolContext
from google.genai import types

print('✅ ADK components imported successfully.')
```

The right sidebar shows the 'Add Secrets' panel with a 'Code Snippet' and a list of secrets. The 'GOOGLE_API_KEY' secret is checked.

Definimos funciones.



The screenshot shows a Kaggle Notebook interface with the title "Day 3a - Agent Sessions". The code in the cell is as follows:

```
# Helper functions defined.
def process_queries(user_queries):
    if type(user_queries) == str:
        user_queries = [user_queries]

    # Process each query in the list sequentially
    for query in user_queries:
        print(f'User > {query}')

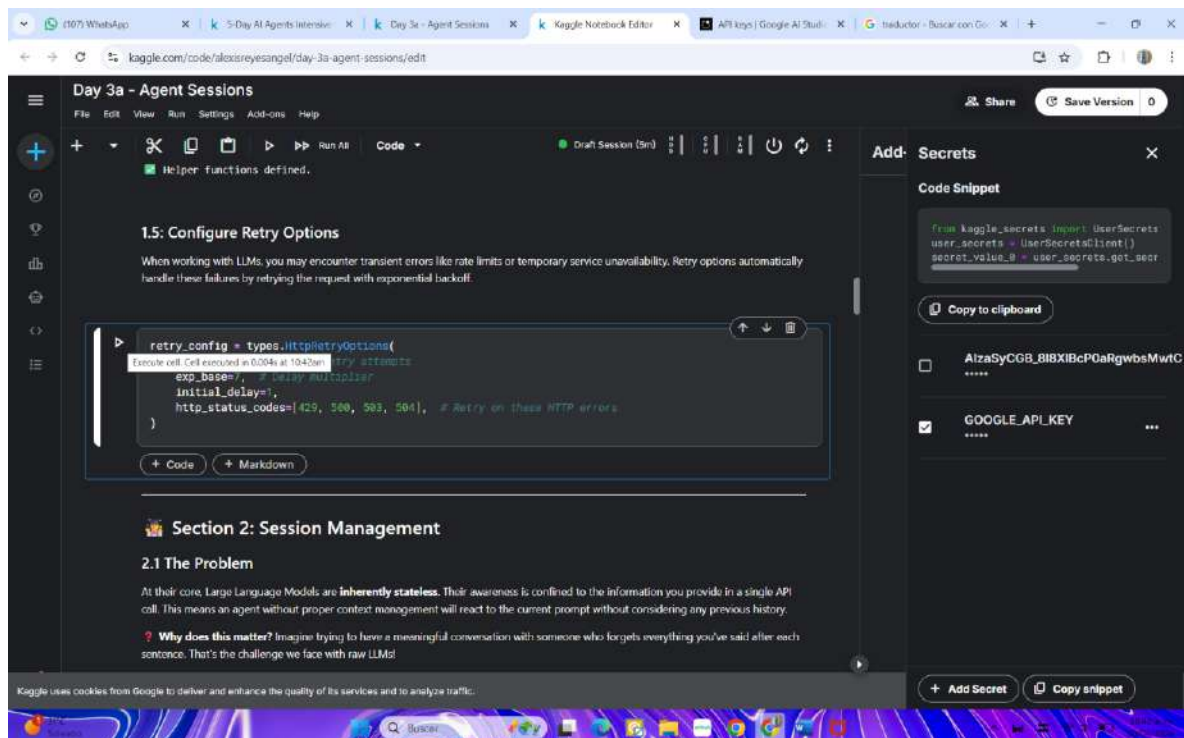
        # Convert the query string to the AIM Content format
        query = types.Content(role='user', parts=[types.Part(text=query)])

        # Stream the agent's response asynchronously
        async for event in runner_instance.run_async(
            user_id=USER_ID, session_id=session_id, new_message=query
        ):
            # Check if the event contains valid content
            if event.content and event.content.parts:
                # Filter out empty or 'None' responses before printing
                if (
                    event.content.parts[0].text != "None"
                    and event.content.parts[0].text
                ):
                    print(f'{MODEL_NAME} > ', event.content.parts[0].text)
            else:
                print("No queries!")

print('Helper functions defined.')
```

The output of the code is "Helper functions defined.".

Configuramos las opciones.



The screenshot shows a Kaggle Notebook interface with the title "Day 3a - Agent Sessions". The code in the cell is as follows:

```
retry_config = types.HttpRetryOptions(
    exp_base=2, # retry multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

The output of the code is "Example cell. Cell executed at 0.004s at 19:43:01".

1.5: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

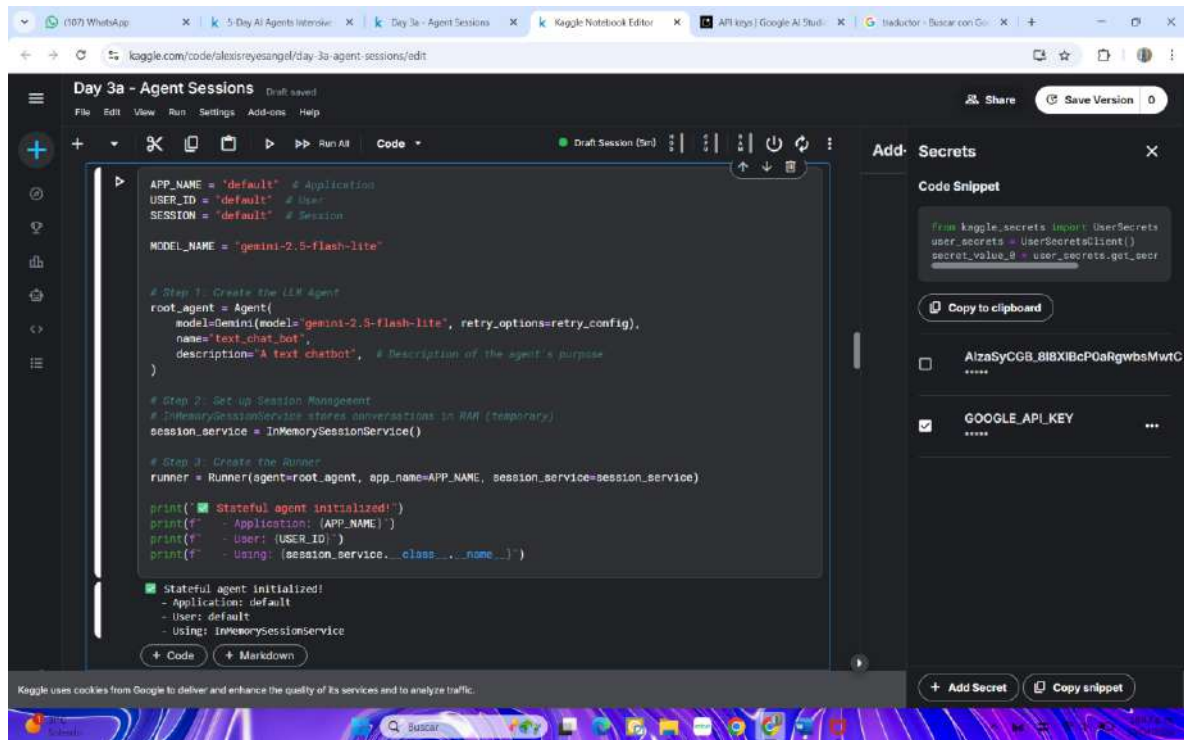
Section 2: Session Management

2.1 The Problem

At their core, Large Language Models are **inherently stateless**. Their awareness is confined to the information you provide in a single API call. This means an agent without proper context management will react to the current prompt without considering any previous history.

Why does this matter? Imagine trying to have a meaningful conversation with someone who forgets everything you've said after each sentence. That's the challenge we face with raw LLMs!

Implementamos nuestro primer agente.



The screenshot shows a Kaggle Notebook interface with the title "Day 3a - Agent Sessions". The code in the cell defines a stateful agent using the Gemini API. It sets up a root agent with the model "gemini-2.5-flash-lite", a name "text_chat_bot", and a description "A text chatbot". It also sets up a session service using "InMemorySessionService" and a runner. The code prints out the agent's configuration and status.

```
APP_NAME = 'default' # Application
USER_ID = 'default' # User
SESSION = 'default' # Session

MODEL_NAME = 'gemini-2.5-flash-lite'

# Step 1: Create the LLM Agent
root_agent = Agent(
    model=Gemini(model='gemini-2.5-flash-lite', retry_options=retry_config),
    name='text_chat_bot',
    description='A text chatbot', # Description of the agent's purpose
)

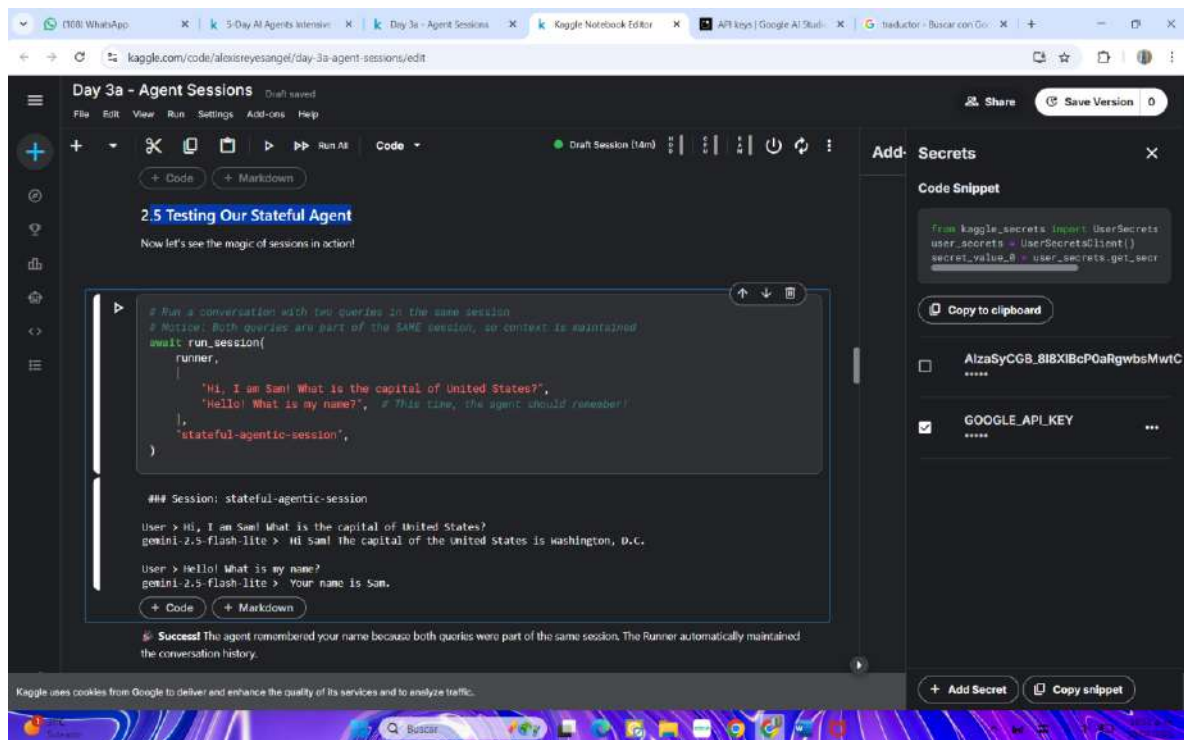
# Step 2: Set up Session Management
# InMemorySessionService stores conversations in RAM (temporary).
session_service = InMemorySessionService()

# Step 3: Create the Runner
runner = Runner(agent=root_agent, app_name=APP_NAME, session_service=session_service)

print('Stateful agent initialized!')
print(f'- Application: {APP_NAME}')
print(f'- User: {USER_ID}')
print(f'- Using: {session_service.__class__.__name__}')
```

The output shows the agent is initialized with the application name "default", user "default", and using "InMemorySessionService".

Prueba de nuestro agente con estado.



The screenshot shows the same Kaggle Notebook interface, but now with a section titled "2.5 Testing Our Stateful Agent". The code in the cell runs a conversation with two queries in the same session. The first query is "Hi, I am Sam! What is the capital of United States?", and the second query is "Hello! What is my name?". The agent's response to the second query is "Your name is Sam.", demonstrating that the agent remembers the context from the first query.

```
# Run a conversation with two queries in the same session
# Notice: Both queries are part of the SAME session, so context is maintained
await run_session(
    runner,
    [
        'Hi, I am Sam! What is the capital of United States?',
        'Hello! What is my name?', # This time, the agent should remember!
    ],
    'stateful-agentic-session',
)
```

The output shows the session name "stateful-agentic-session" and the conversation history:

```
## Session: stateful-agentic-session

User > Hi, I am Sam! What is the capital of United States?
gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.

User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam.
```

The success message at the bottom states: "Success! The agent remembered your name because both queries were part of the same session. The Runner automatically maintained the conversation history."

Probamos los nuevos agentes.

The screenshot shows a Kaggle Notebook interface for a notebook titled "Day 3a - Agent Sessions". The notebook is in a "Draft" state. The current cell is titled "(Optional) 2.6 Testing Agent's forgetfulness". The text in the cell explains that to verify if the agent forgets the conversation, the kernel should be restarted, and then all previous cells should be run, except for the `run_session` cell in 2.5. It then shows a code cell with the following content:

```
# Run this cell after restarting the kernel. All this history will be gone...
await run_session(
    runner,
    ["What did I ask you about earlier?", "And remind me, what's my name?"],
    "stateful-agent-session",
) # Note, we are using same session name

### Session: stateful-agent-session
User > What did I ask you about earlier?
```

Below the code cell, there is a section titled "The Problem" which states: "Session information is not persistent (i.e., meaningful conversations are lost). While this is advantageous in testing environments, in the real world, a user should be able to refer from past and resume conversations. To achieve this, we must persist information."

On the right side of the notebook, there is a "Secrets" panel with a "Code Snippet" showing the following code:

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Below the snippet, there are two secrets listed: "AlzaSyCGB_8l8XlBcP0aRgwbsMwtC" and "GOOGLE_API_KEY". The "GOOGLE_API_KEY" secret is checked.

The screenshot shows the same Kaggle Notebook interface, but now the code cell is titled "Let's create a 'chatbot_agent' capable of having a conversation with the user." The code cell contains the following content:

```
# Step 1: Create the agent (notice we use LLMAgent this time)
chatbot_agent = LLMAgent(
    model=Gemini(model="gemini-2.0-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f"  - Database: my_agent_data.db")
print(f"  - Sessions will survive restarts!")
```

Below the code cell, there is a section titled "3.3 Test Run 1: Verifying Persistence". The text in this section states: "In this first test run, we'll start a new conversation with the session ID 'test-db-session-01'. We will first introduce our name as 'Sam' and..."

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

3.3 Test Run 1: Verifying Persistence

In this first test run, we'll start a new conversation with the session ID 'test-db-session-01'. We will first introduce our name as 'Sam' and then ask a question. In the second turn, we will ask the agent for our name.

Since we are using `DatabaseSessionService`, the agent should remember the name.

After the conversation, we'll inspect the `my_agent_data.db` SQLite database directly to see how the conversation events (the user queries and model responses) are stored.

```
await run_session(
    runner,
    [{"Hi, I am Sam! What is the capital of the United States?", "Hello! What is my name?"},
     "test-db-session-01",
])
```

Session: test-db-session-01

User > Hi, I am Sam! What is the capital of the United States?

(Optional) 3.4 Test Run 2: Resuming a Conversation

Now, let's repeat the test again, but this time, let's stop this Kaggle Notebook's kernel and restart it again.

1. Run all the previous cells in the notebook, EXCEPT the previous Section 3.3 (`run_session` call).

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

remember that our name is 'Sam' from the first test run. This demonstrates the power of persistent sessions.

```
await run_session(
    runner,
    [{"What is the capital of India?", "Hello! What is my name?"},
     "test-db-session-01",
])
```

Session: test-db-session-01

User > What is the capital of India?

```
CancelledError
Traceback (most recent call last)
<ipykernel> 49/40420217.py in <cell line: 3>()
----> 1 await run_session(
      2     runner,
      3     [{"What is the capital of India?", "Hello! What is my name?"},
      4     "test-db-session-01",
      5 ])
      6
      7 /tmp/ipykernel_49/1727200007.py in run_session(runner_instance, user_queries, session_name)
      8
      9     # stream the agent's response asynchronously
     10     async for event in runner_instance.run_async(
     11         user_id=USER_ID, session_id=session_id, new_message=query
     12     ):
     13         pass
     14
     15 /usr/local/lib/python3.11/dist-packages/google/ai/gemini/runners.py in run_async(self, user_id, session_id, invocation_id, new_message, state_delta, run_config)
     16
     17     async with AioClosingRunWithTrace(new_message, invocation_id) as agen:
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (26m)

[28]:

```
await run_session(
    runner, ["Hello! What is my name?", "test-db-session-02"]
) # Note, we are using new session name
```

Session: test-db-session-02

User > Hello! What is my name?

CancelledError: Traceback (most recent call last)

```
File ~/ipynbkernel/ipykernel/ipykernel.py in <cell line: 1>()
----> 1 await run_session(
      runner, ["Hello! What is my name?", "test-db-session-02"]
      ) # Note, we are using new session name

File ~/ipynbkernel/ipykernel/ipykernel.py in run_session(runner_instance, user_queries, session_name)
    46
    47     # Stream the agent's response asynchronously
    48     async for event in runner_instance.run_async(
    49         user_id=USER_ID, session_id=session_id, new_message=query
    50     ):
    51         pass

File ~/ipynbkernel/ipykernel/ipykernel.py in run_async(self, user_id, session_id, invocation_id, new_message, state_delta, run_config)
    441
    442     async with AClosing(_run_with_trace.new_message, invocation_id) as agent:
    443         async for event in agent:
    444             yield event
    445         pass

File ~/ipynbkernel/ipykernel/ipykernel.py in _run_with_trace(new_message, invocation_id)
    446
    447     ) as agent:
```

Copy to clipboard

Alza5yCGB_8B8XBcPGaRgwb8Mwfc

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (26m)

Since we are using a sqlite DB to store information, let's have a quick peek to see how information is stored.

```
import sqlite3

def check_data_in_db():
    with sqlite3.connect("my_agent_data.db") as connection:
        cursor = connection.cursor()
        result = cursor.execute(
            "select app_name, session_id, author, content from events"
        )
        print([_["description"] for _ in result.fetchall()])
        for each in result.fetchall():
            print(each)

check_data_in_db()
```

[{"app_name": "stateful-agent-session", "session_id": "test-db-session-01", "author": "user", "content": [{"text": "Hi, I am Sam! What is the capital of the United States?", "role": "user"}]}, {"app_name": "stateful-agent-session", "session_id": "test-db-session-02", "author": "user", "content": [{"text": "Hello! What is my name?", "role": "user"}]}, {"app_name": "stateful-agent-session", "session_id": "test-db-session-01", "author": "user", "content": [{"text": "What is the capital of India?", "role": "user"}]}, {"app_name": "stateful-agent-session", "session_id": "test-db-session-01", "author": "user", "content": [{"text": "What did I ask you about earlier?", "role": "user"}]}, {"app_name": "stateful-agent-session", "session_id": "test-db-session-01", "author": "user", "content": [{"text": "What is the capital of India?", "role": "user"}]}]

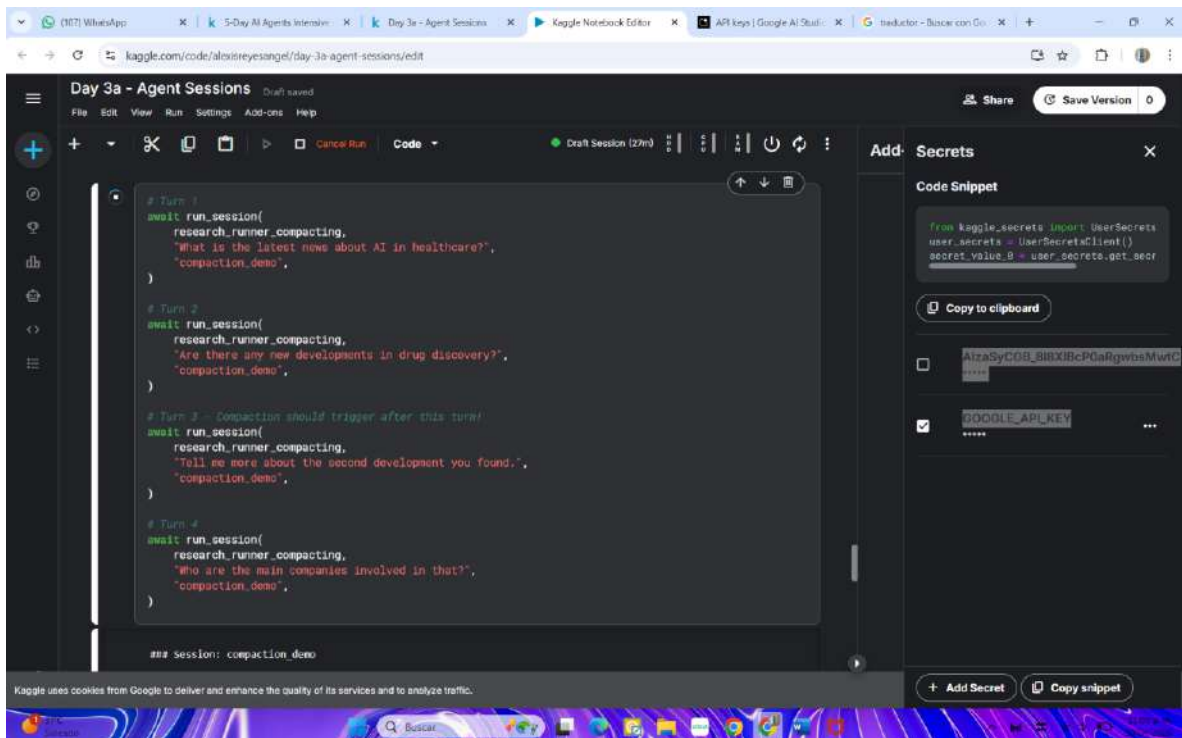
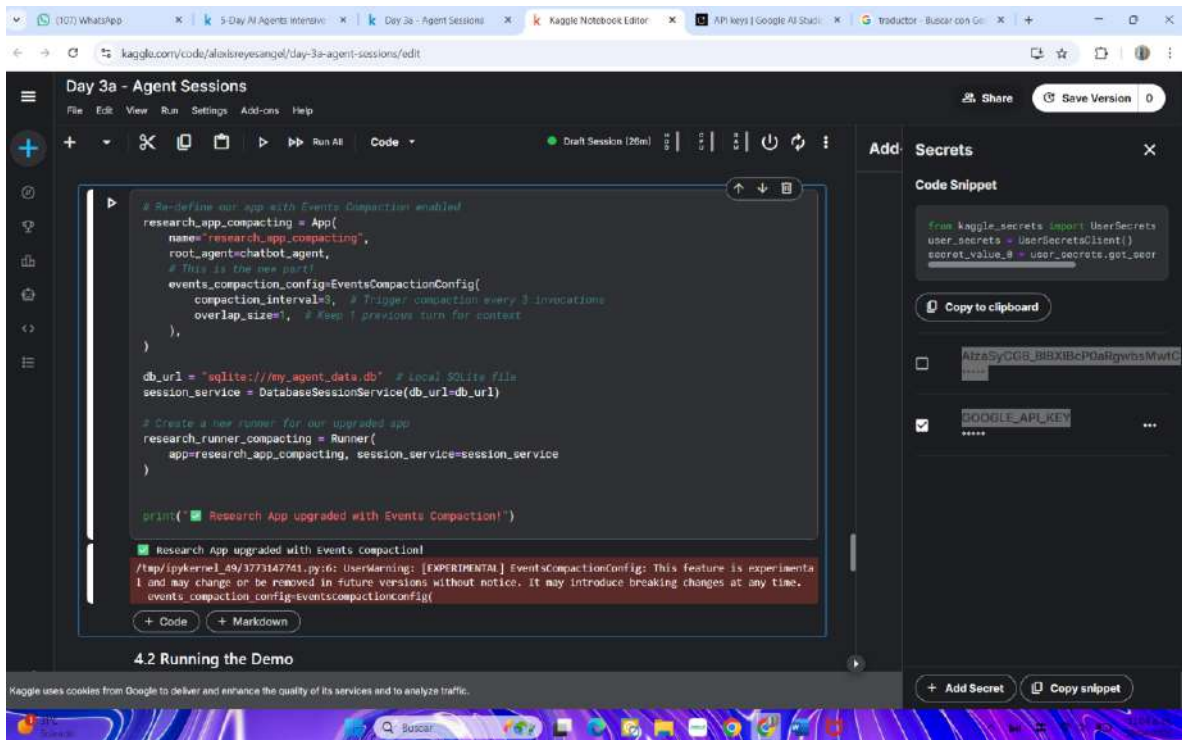
+ Code + Markdown

Copy to clipboard

Alza5yCGB_8B8XBcPGaRgwb8Mwfc

GOOGLE_API_KEY

+ Add Secret Copy snippet



Day 3a - Agent Sessions

We can inspect the `events` list from our session. The compaction process **doesn't delete old events**; it replaces them with a single, new `Event` that contains the summary. Let's find it.

```
# Get the final session state
final_session = await session_service.get_session(
    app_name=research_runner_compacting.app_name,
    user_id=USER_ID,
    session_id="compaction_demo",
)

print("--- Searching for Compaction Summary Event ---")
found_summary = False
for event in final_session.events:
    # Compaction events have a 'compaction' attribute
    if event.actions and event.actions.compaction:
        print(f"🟢 SUCCESS! Found the Compaction Event!")
        print(f"  Author: {event.author}")
        print(f"  In Compacted information: {event}")
        found_summary = True
        break

if not found_summary:
    print(f"🔴 No compaction event found. Try increasing the number of turns in the demo.")
```

4.4 What you've accomplished: Automatic Context Management

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Creamos un tolos.

Day 3a - Agent Sessions

```
def retrieve_userinfo(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Tool to record and save user name and country in session state.

    Args:
        user_name: The username to store in session state
        country: The name of the user's country
    """
    # Write to session state using the 'user:' prefix for user data
    tool_context.state["user:name"] = user_name
    tool_context.state["user:country"] = country

    return {"status": "success"}

# This demonstrates how tools can read from session state
def retrieve_userinfo(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Tool to retrieve user name and country from session state.
    """
    # Read from session state
    user_name = tool_context.state.get("user:name", "Username not found")
    country = tool_context.state.get("user:country", "Country not found")

    return {"status": "success", "user_name": user_name, "country": country}

print("🟢 Tools created.")
```

Tools created.

+ Code + Markdown

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3a - Agent Sessions

Now let's create a new agent that has access to our session state management tools:

```
# Configuration
APP_NAME = 'default'
USER_ID = 'default'
MODEL_NAME = 'gemini-2.5-flash-lite'

# Create an agent with session state tools
root_agent = LLMAgent(
    model=Gemini(model='gemini-2.5-flash-lite', retry_options=retry_config),
    name='text_chat_bot',
    description='A text chatbot.
    Tools for managing user context:
    * To record username and country when provided use "save_userinfo" tool.
    * To fetch username and country when required use "retrieve_userinfo" tool.
    tools=[save_userinfo, retrieve_userinfo], # Provide the tools to the agent
)

# Set up session service and runner
session_service = InMemorySessionService()
runner = Runner(agent=root_agent, session_service=session_service, app_name="default")

print("Agent with session state tools initialized")
```

Agent with session state tools initialized

5.3 Testing Session State in Action

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Day 3a - Agent Sessions

Let's test how the agent uses session state to remember information across conversation turns:

```
# Test conversation demonstrating session state
await run_session(
    runner,
    [
        "Hi there, how are you doing today? What is my name?", # Agent shouldn't know the name yet
        "My name is Sam. I'm from Poland.", # Provide name - agent should save it
        "What is my name? Which country am I from?", # Agent should recall from session state
    ],
    "state-demo-session",
)

### Session: state-demo-session
User > Hi there, how are you doing today? What is my name?
```

5.4 Inspecting Session State

Let's directly inspect the session state to see what's stored:

```
# Retrieve the session and inspect its state
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

+++ Draft Session (30m)

+++ Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+++ Add Secret Copy snippet

+++ Session: state-demo-session

User > Hi there, how are you doing today? What is my name?

5.4 Inspecting Session State

Let's directly inspect the session state to see what's stored:

```
# Retrieve the session and inspect its state
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="state-demo-session"
)

print("Session State Contents:")
print(session.state)
print("\n Notice the 'username' and 'user:country' keys storing our data!")
```

+ Code + Markdown

5.5 Session State Isolation

As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

+++ Draft Session (30m)

+++ Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+++ Add Secret Copy snippet

+++ Session: state-demo-session

User > Hi there, how are you doing today? What is my name?

5.4 Inspecting Session State

Let's directly inspect the session state to see what's stored:

```
# Retrieve the session and inspect its state
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="state-demo-session"
)

print("Session State Contents:")
print(session.state)
print("\n Notice the 'username' and 'user:country' keys storing our data!")
```

+ Code + Markdown

Session State Contents:

()

Notice the 'username' and 'user:country' keys storing our data!

5.5 Session State Isolation

As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Cancel Run Code Draft Session (53m)

As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:

```
# Start a completely new session ~ the agent won't know our name
await run_session(
    runner,
    ["Hi there, how are you doing today? What is my name?"],
    "new-isolated-session",
)

# Expected: The agent won't know the name because this is a different session
```

Session: new-isolated-session

User > Hi there, how are you doing today? What is my name?

+ Code + Markdown

5.6 Cross-Session State Sharing

While sessions are isolated by default, you might notice something interesting. Let's check the state of our new session (new-isolated-session):

```
[ ]: # Check the state of the new session
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="new-isolated-session"
)
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secret Copy snippet

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_8 = user_secrets.get_secret_value_8
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (4m)

While sessions are isolated by default, you might notice something interesting. Let's check the state of our new session (new-isolated-session):

Type Markdown and LaTeX α^3

```
# Check the state of the new session
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="new-isolated-session"
)

print("New Session State:")
print(session.state)

# Note: Depending on implementation, you might see shared state here.
# This is where the distinction between session-specific and user-specific state becomes important.
```

New Session State:

```
{}
```

+ Code + Markdown

Cleanup

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secret Copy snippet

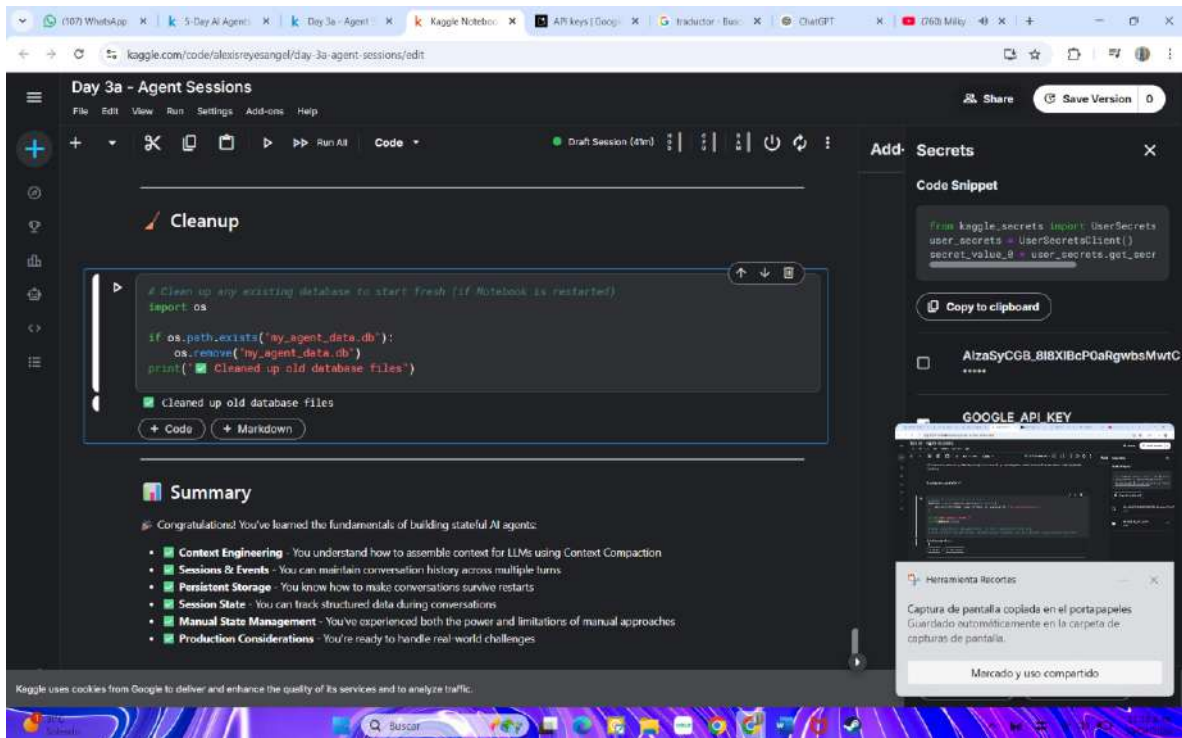
Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_8 = user_secrets.get_secret_value_8
```

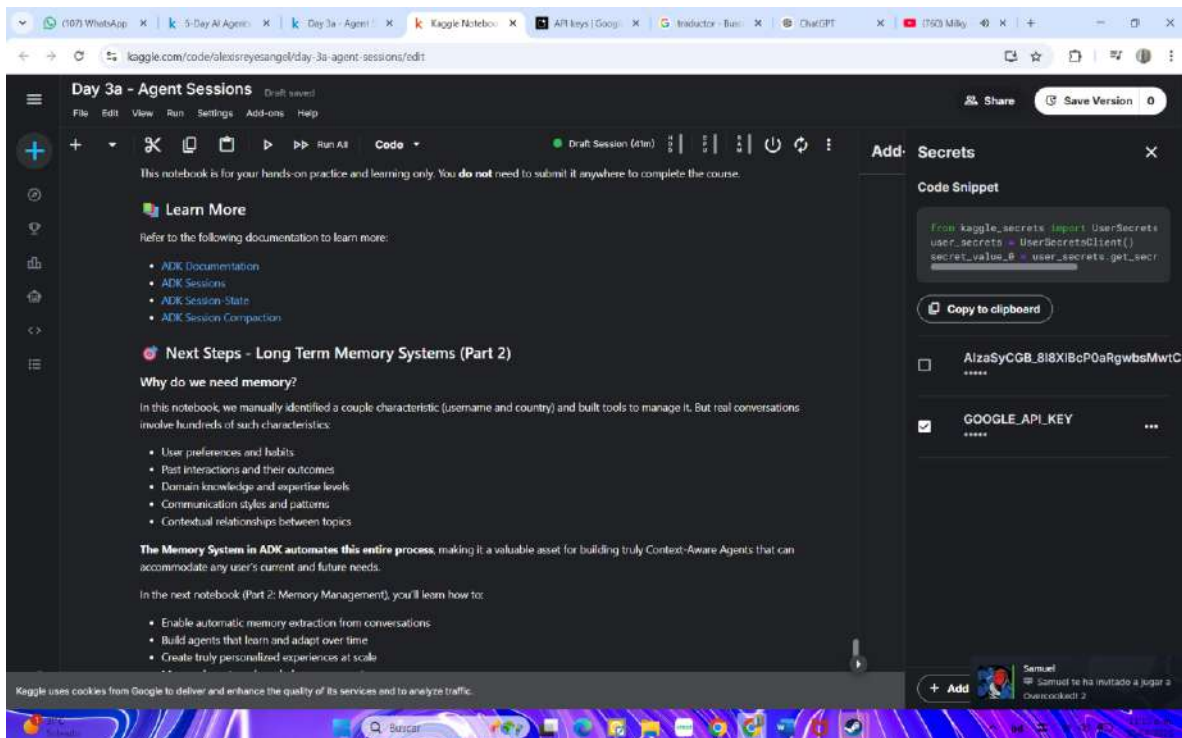
Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY



Finalización del día.



Day 3b - Agent Memory

Run the cell below to complete authentication.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("Gemini API key setup complete.")
except Exception as e:
    print(f"Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secret")
```

Gemini API key setup complete.

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import LlmAgent
from google.adk.models.google.llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.generativeai import types
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Samuel

add jupyter to Overcooked 2

Day 3b - Agent Memory

Cancel Run

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import LlmAgent
from google.adk.models.google.llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.generativeai import types

print("ADK components imported successfully.")
```

1.4: Helper functions

This helper function manages a complete conversation session, handling session creation/retrieval, query processing, and response streaming.

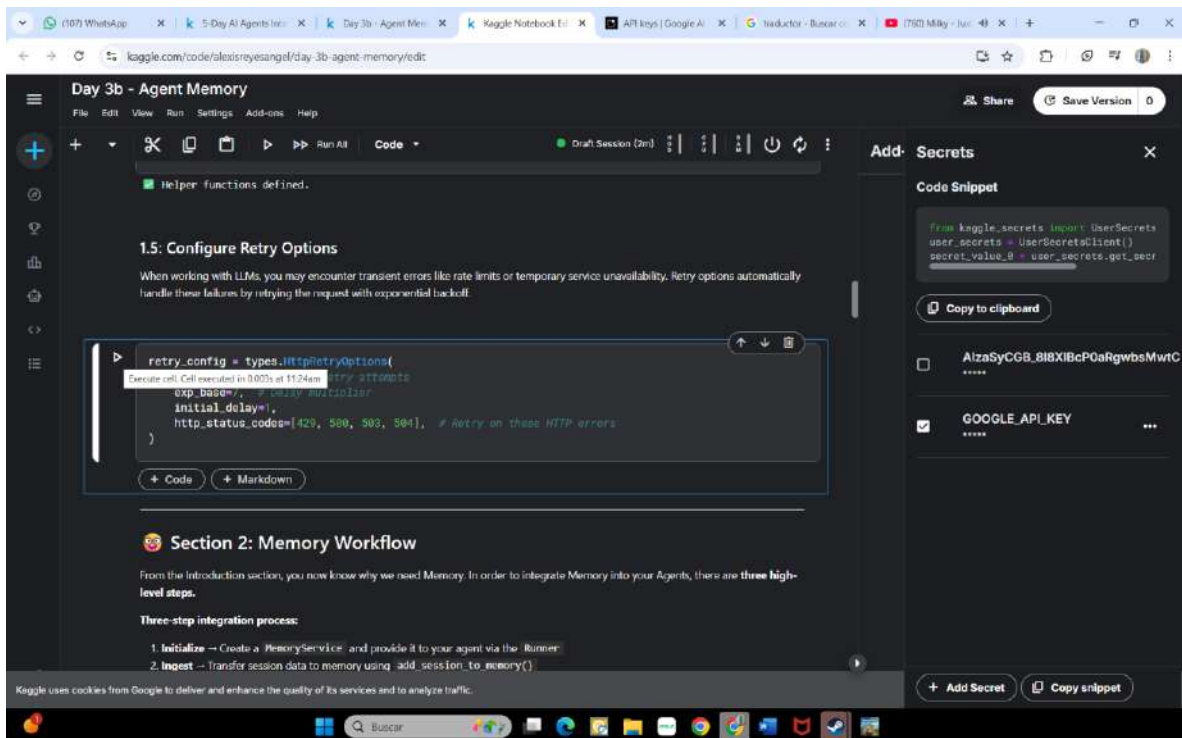
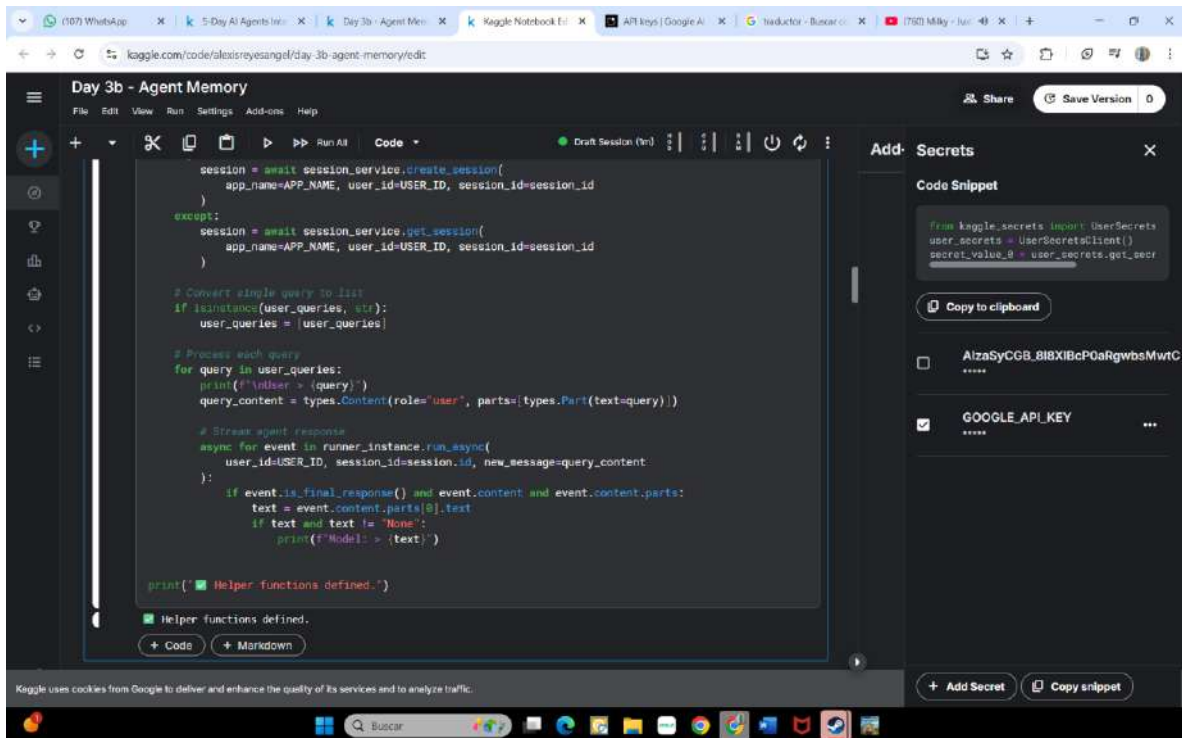
```
async def run_session(
    runner_instance: Runner, user_queries: list[str] | str, session_id: str = "default"
):
    """Helper function to run queries in a session and display responses."""
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add ?

aloxis.goma

add jupyter to Overcooked 2



Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Draft Session (2m)

3.1 Initialize Memory

ADK provides multiple `MemoryService` implementations through the `BaseMemoryService` interface:

- InMemoryMemoryService** - Built-in service for prototyping and testing (keyword matching, no persistence)
- VertexAIMemoryBankService** - Managed cloud service with LLM-powered consolidation and semantic search
- Custom implementations** - You can build your own using databases, though managed services are recommended

For this notebook, we'll use `InMemoryMemoryService` to learn the core mechanics. The same methods work identically with production-ready services like Vertex AI Memory Bank.

```
[23]: memory_service = (
    InMemoryMemoryService()
) # ADK's built-in Memory Service for development and testing
```

+ Code + Markdown

3.2 Add Memory to Agent

Next, create a simple agent to answer user queries.

```
# Define constants used throughout the notebook
APP_NAME = "MemoryDemoApp"
USER_ID = "demo_user"
```

+ Add Secret + Copy snippet

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze its usage.

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Draft Session (2m)

3.2 Add Memory to Agent

Next, create a simple agent to answer user queries.

```
# Define constants used throughout the notebook
APP_NAME = "MemoryDemoApp"
USER_ID = "demo_user"

# Create Agent
user_agent = LLMAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words.",
)

print("Agent created")
```

Agent created

+ Code + Markdown

Create Runner

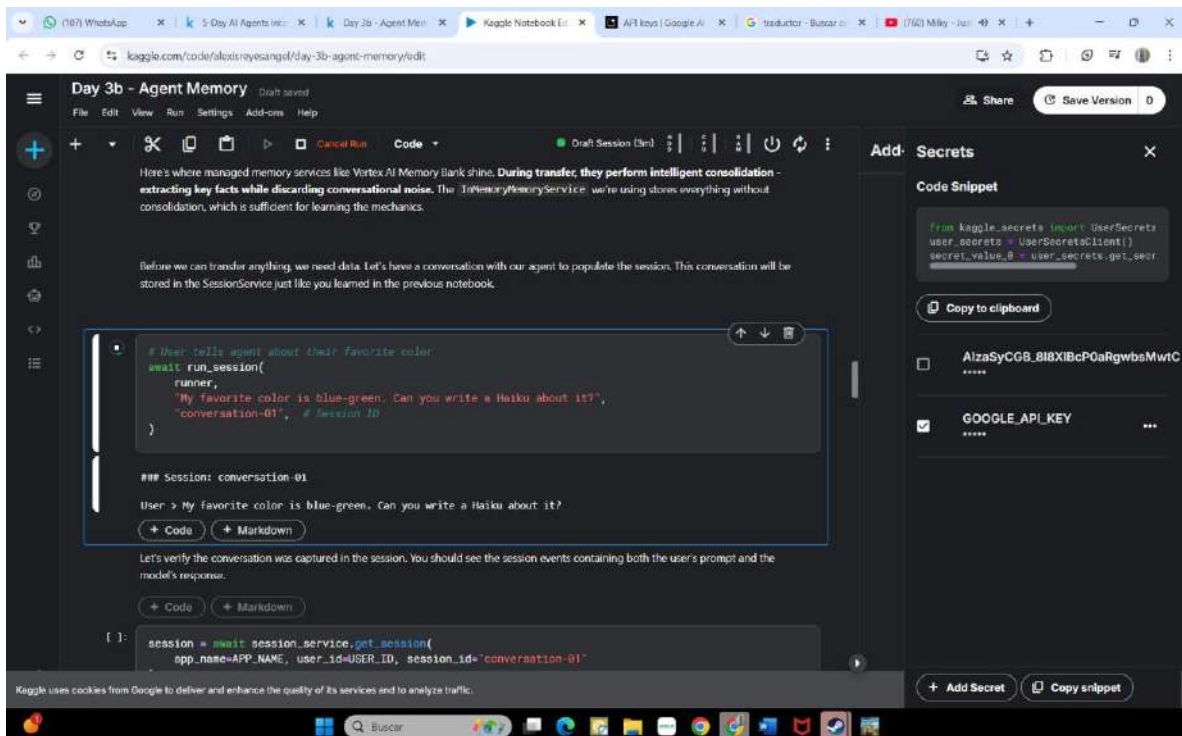
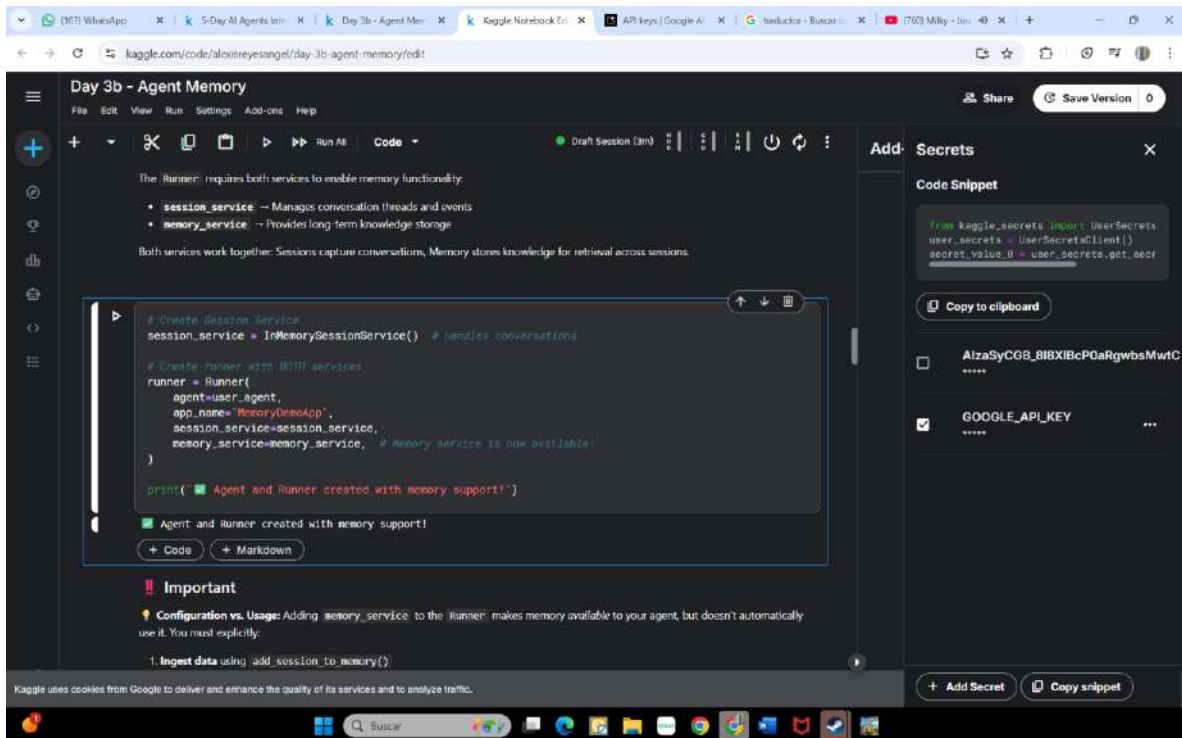
Now provide both Session and Memory services to the `Runner`.

Key configuration:

The `Runner` requires both services to enable memory functionality.

+ Add Secret + Copy snippet

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze its usage.



Añadimos la sesión a memoria

The screenshot shows a Kaggle Notebook interface for 'Day 3b - Agent Memory'. The notebook has a dark theme. At the top, there's a toolbar with icons for file operations, running, and settings. The main content area shows a code cell with the following code:

```
# This is the key method!  
await memory_service.add_session_to_memory(session)  
  
print("Session added to memory!")
```

Below the code cell, there's a green status bar indicating 'Session added to memory!' and buttons for '+ Code' and '+ Markdown'. To the right of the notebook, there's a 'Secrets' panel with a 'Code Snippet' section containing the following code:

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Below the code snippet, there's a 'Copy to clipboard' button. The 'Secrets' panel also shows two secrets: 'AlzaSyCG8_8IBXIBcP0aRgwbsMwtC' and 'GOOGLE_API_KEY'. At the bottom of the notebook, there's a section titled 'Section 5: Enable Memory Retrieval in Your Agent' with a paragraph explaining that agents can't directly access the MemoryService and need tools to search it.

The screenshot shows a Kaggle Notebook interface for 'Day 3b - Agent Memory'. The notebook has a dark theme. At the top, there's a toolbar with icons for file operations, running, and settings. The main content area shows a code cell with the following code:

```
# Create agent  
user_agent = LlmAgent(  
    model=Gemini(model='gemini-2.0-flash-lite', retry_options=retry_config),  
    name='MemoryDemoAgent',  
    instruction='Answer user questions in simple words. Use load_memory tool if you need to recall past tools',  
    tools=[  
        load_memory  
    ],  
    # Agent now has access to Memory and can search it whenever it decides to!  
)  
  
print("Agent with load_memory tool created.")
```

Below the code cell, there's a green status bar indicating 'Agent with load_memory tool created.' and buttons for '+ Code' and '+ Markdown'. To the right of the notebook, there's a 'Secrets' panel with a 'Code Snippet' section containing the following code:

```
from kaggle_secrets import UserSecrets  
user_secrets = UserSecretsClient()  
secret_value_0 = user_secrets.get_secret_value_0
```

Below the code snippet, there's a 'Copy to clipboard' button. The 'Secrets' panel also shows two secrets: 'AlzaSyCG8_8IBXIBcP0aRgwbsMwtC' and 'GOOGLE_API_KEY'. At the bottom of the notebook, there's a section titled '5.3 Update the Runner and Test' with a paragraph explaining that sessions don't share conversation history and the only way the agent can answer correctly is by using the load_memory tool.

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

5.3 Update the Runner and Test

Let's now update the Runner to use our new `user_agent`, that has the `load_memory` tool. And we'll ask the Agent about the favorite color which we had stored previously in another session.

🌟 Since sessions don't share conversation history, the only way the agent can answer correctly is by using the `load_memory` tool to retrieve the information from long-term memory that we manually stored.

```
# Create a new runner with the updated agent
runner = Runner(
    agent=user_agent,
    app_name=APP_NAME,
    session_service=session_service,
    memory_service=memory_service,
)

await run_session(runner, "What is my favorite color?", "color-test")
```

Session: color-test

User > What is my favorite color?

5.4 Complete Manual Workflow Test

Let's see the complete workflow in action. We'll have a conversation about a birthday, manually save it to memory, then test retrieval in a new session. This demonstrates the full cycle: **ingest** → **store** → **retrieve**.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

to retrieve the information from long-term memory that we manually stored.

```
# Create a new runner with the updated agent
runner = Runner(
    agent=user_agent,
    app_name=APP_NAME,
    session_service=session_service,
    memory_service=memory_service,
)

await run_session(runner, "What is my favorite color?", "color-test")
```

Session: color-test

User > What is my favorite color?

WARNING:google.generativeai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

5.4 Complete Manual Workflow Test

Let's see the complete workflow in action. We'll have a conversation about a birthday, manually save it to memory, then test retrieval in a new session. This demonstrates the full cycle: **ingest** → **store** → **retrieve**.

```
await run_session(runner, "My birthday is on March 15th.", "birthday-session-01")
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Share Save Version 0

Code

Draft Session (1m)

```
h.cancel()
CancelledError
```

Now manually save this session to memory. This is the crucial step that transfers the conversation from short-term session storage to long-term memory storage.

```
# Manually save the session to memory
birthday_session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="birthday-session-01"
)

await memory_service.add_session_to_memory(birthday_session)

print("🟢 Birthday session saved to memory!")
```

🟢 Birthday session saved to memory!

+ Code + Markdown

Here's the crucial test: we'll start a completely new session with a different session ID and ask the agent to recall the birthday.

```
[ ]: # [last retrieval] in a NEW session
await run_session(
    runner, "When is my birthday?", "birthday-session-02" # Different session ID
)
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secret Copy snippet

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Share Save Version 0

Code

Draft Session (13m)

```
# Search for color preferences
search_response = await memory_service.search_memory(
    app_name=APP_NAME, user_id=USER_ID, query="What is the user's favorite color?"
)

print("🔍 Search Results:")
print(f"Found {len(search_response.memories)} relevant memories")
print()

for memory in search_response.memories:
    if memory.content and memory.content.parts:
        text = memory.content.parts[0].text[:100]
        print(f"📄 {memory.author}: {text}...")
```

🔍 Search Results:
Found 2 relevant memories

[user]: My favorite color is blue-green. Can you write a Haiku about it?...
[user]: My birthday is on March 15th....

+ Code + Markdown

Your Turn: Test Different Queries

Try these searches to understand how keyword matching works with `TrMemoryMemoryService`:

1. "what color does the user like"
2. "haiku"
3. "age"
4. "preferred hue"

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Add Secret Copy snippet

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (14m)

But here's the challenge: how does our callback function actually access the memory service and current session? That's where `callback_context` comes in.

When you define a callback function, ADK automatically passes a special parameter called `callback_context` to it. The `callback_context` provides access to the Memory Service and other runtime components.

How we'll use it: in our callback, we'll access the memory service and current session to automatically save conversation data after each turn.

Important: You don't create this context - ADK creates it and passes it to your callback automatically when the callback runs.

```
async def auto_save_to_memory(callback_context):
    """Automatically save session to memory after each agent turn."""
    await callback_context.invocation_context.memory_service.add_session_to_memory(
        callback_context.invocation_context.session
    )

print("✅ Callback created.")
```

✅ Callback created.

+ Code + Markdown

6.3 Create an Agent: Callback and PreLoad Memory Tool

Now create an agent that combines:

- **Automatic storage:** `after_agent_callback` saves conversations
- **Automatic retrieval:** `preload_memory` loads memories

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (14m)

✅ callback created.

6.3 Create an Agent: Callback and PreLoad Memory Tool

Now create an agent that combines:

- **Automatic storage:** `after_agent_callback` saves conversations
- **Automatic retrieval:** `preload_memory` loads memories

This creates a fully automated memory system with zero manual intervention.

```
# Agent with automatic memory saving
auto_memory_agent = LlmAgent(
    model=Gemini(model="gemini-1.5-flash-lite", retry_options=retry_config),
    name="AutoMemoryAgent",
    instruction="Answer user questions.",
    tools=[preload_memory],
    after_agent_callback=auto_save_to_memory, # Saves after each turn!
)

print("✅ Agent created with automatic memory saving!")
```

✅ Agent created with automatic memory saving!

+ Code + Markdown

What happens automatically:

- After every agent response → callback triggers

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

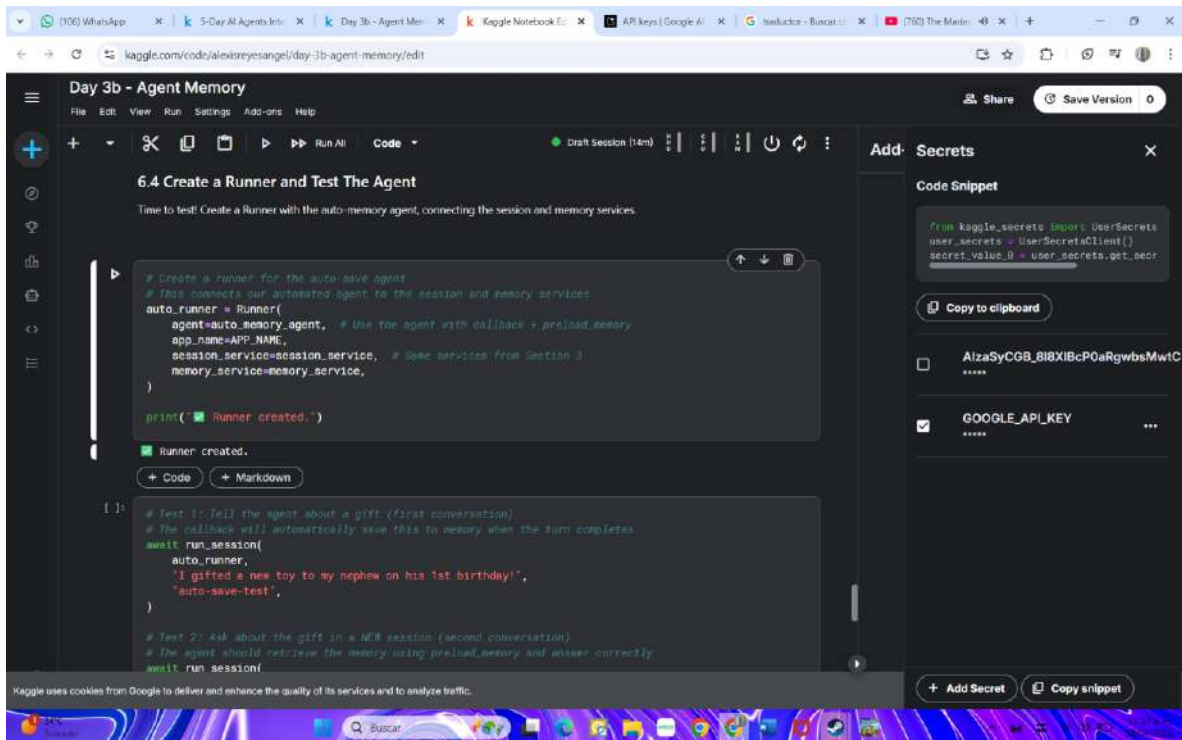
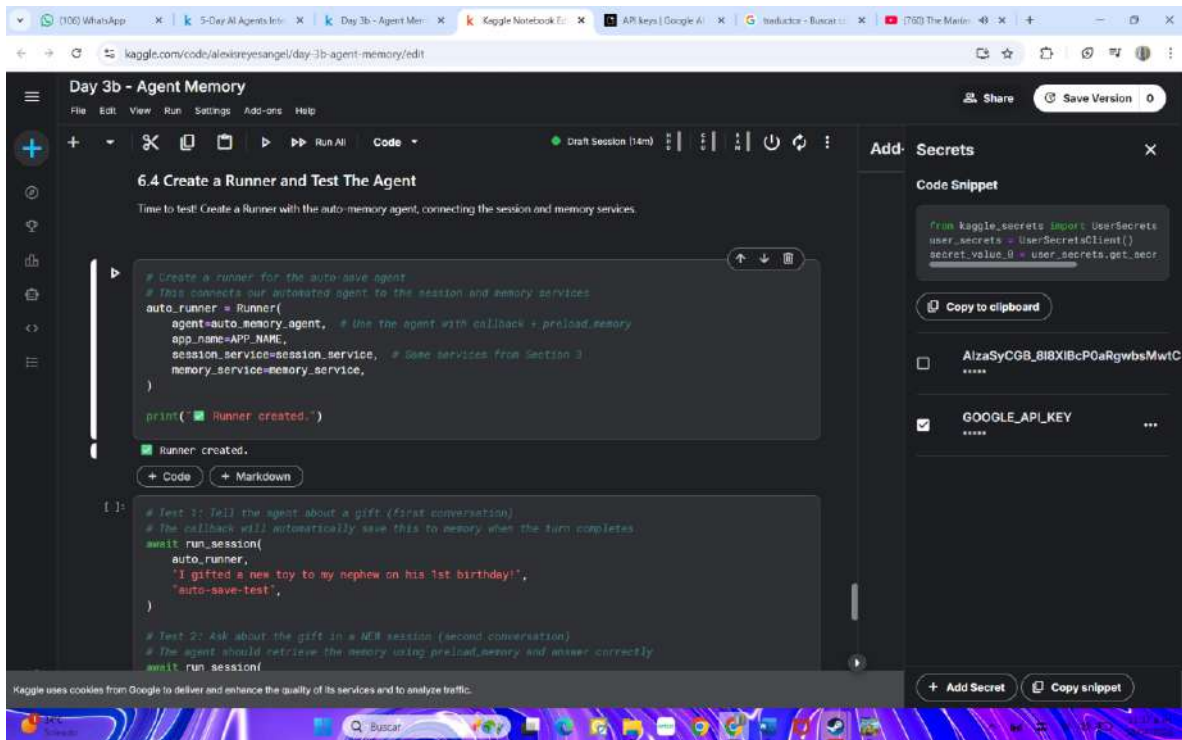
```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

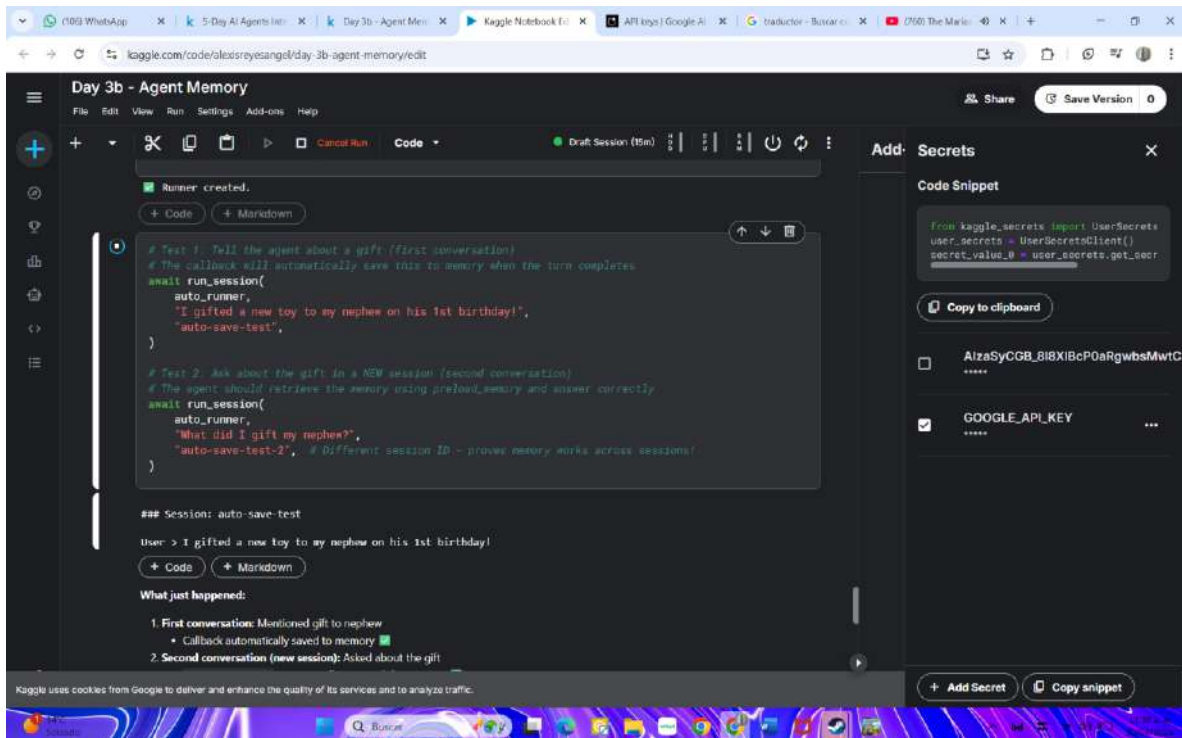
Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

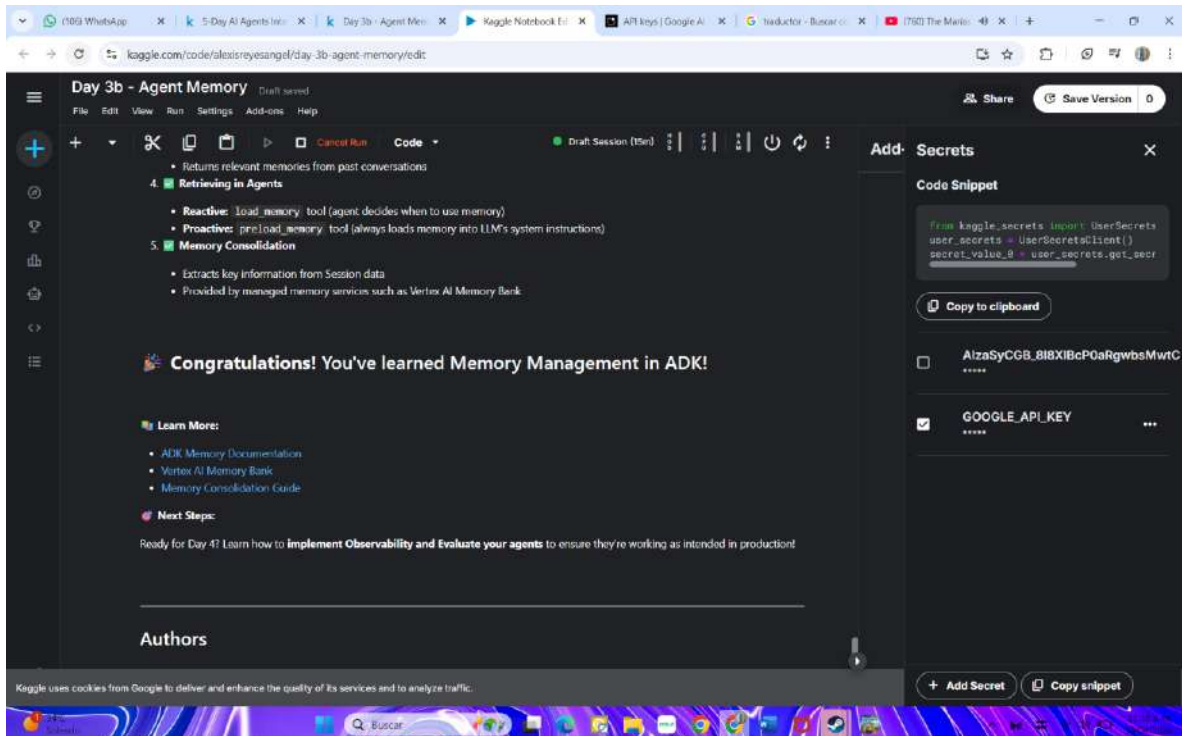
GOOGLE_API_KEY

+ Add Secret Copy snippet



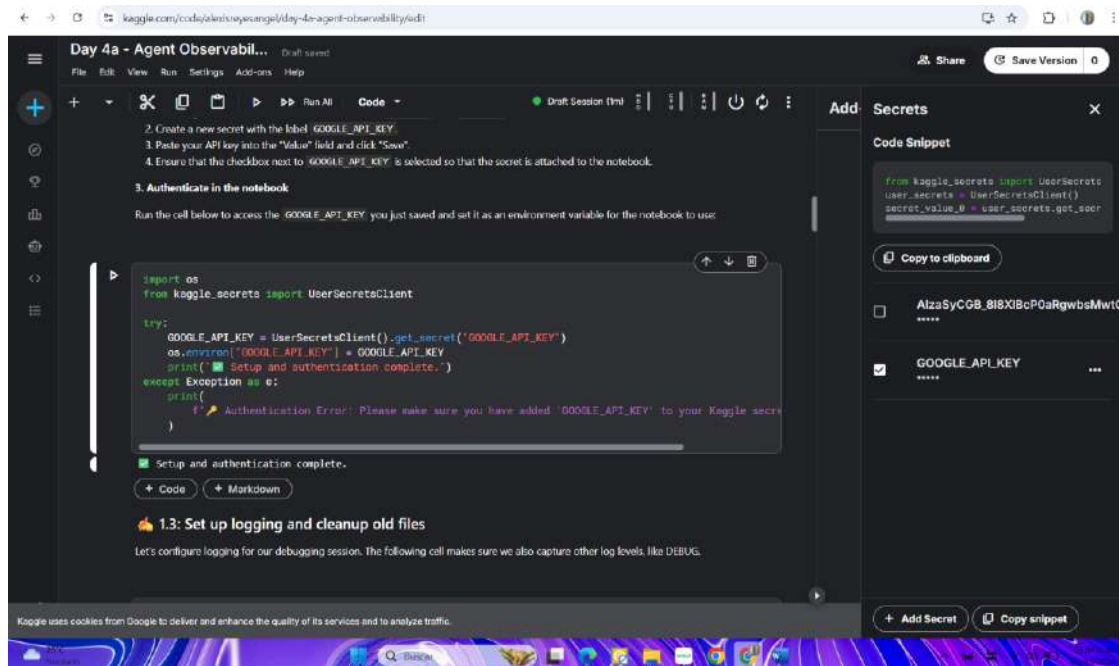


Finalización del día 3.



Curso KAGGLE – DÍA 4

En este código importamos las librerías necesarias, obtengo mi clave ****GOOGLE_API_KEY**** desde los secretos de Kaggle y la guardo como variable de entorno para poder usarla en el proyecto; si todo funciona correctamente muestro un mensaje de confirmación y, si ocurre un error, indica que falta configurar la clave.



The screenshot shows a Kaggle Notebook interface. The main code cell contains the following Python code:

```
import os
from kaggle_secrets import UserSecretsClient

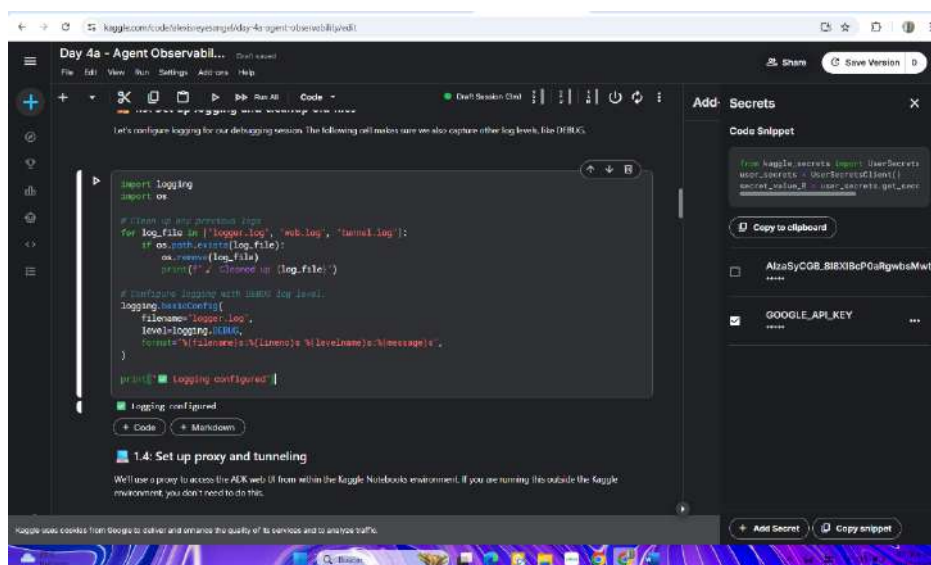
try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print('Setup and authentication complete.')
except Exception as e:
    print(f'Authentication Error! Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets')
    
```

Below the code cell, there is a message: "Setup and authentication complete." and buttons for "Code" and "Markdown".

On the right side, there is a "Secrets" panel showing a list of secrets. The secret "GOOGLE_API_KEY" is selected and its value is displayed as "*****".

Below the code cell, there is a section titled "1.3: Set up logging and cleanup old files" with the text: "Let's configure logging for our debugging session. The following cell makes sure we also capture other log levels like DEBUG."

En este código importo las librerías necesarias, elimino archivos de logs anteriores si existen para limpiar el entorno, luego configuro un sistema de registro (logging) en nivel DEBUG para guardar información detallada en un archivo llamado `logger.log`, y finalmente se muestra un mensaje indicando que la configuración se realizó correctamente.



The screenshot shows a Kaggle Notebook interface. The main code cell contains the following Python code:

```
import logging
import os

# Clean up any previous logs
for log_file in ["logger.log", "web.log", "tunnel.log"]:
    if os.path.exists(log_file):
        os.remove(log_file)
    print(f"Cleaned up {log_file}")

# Configure logging with DEBUG log level
logging.basicConfig(
    filename="logger.log",
    level=logging.DEBUG,
    format='%(asctime)s: %(levelname)s: %(message)s'
)

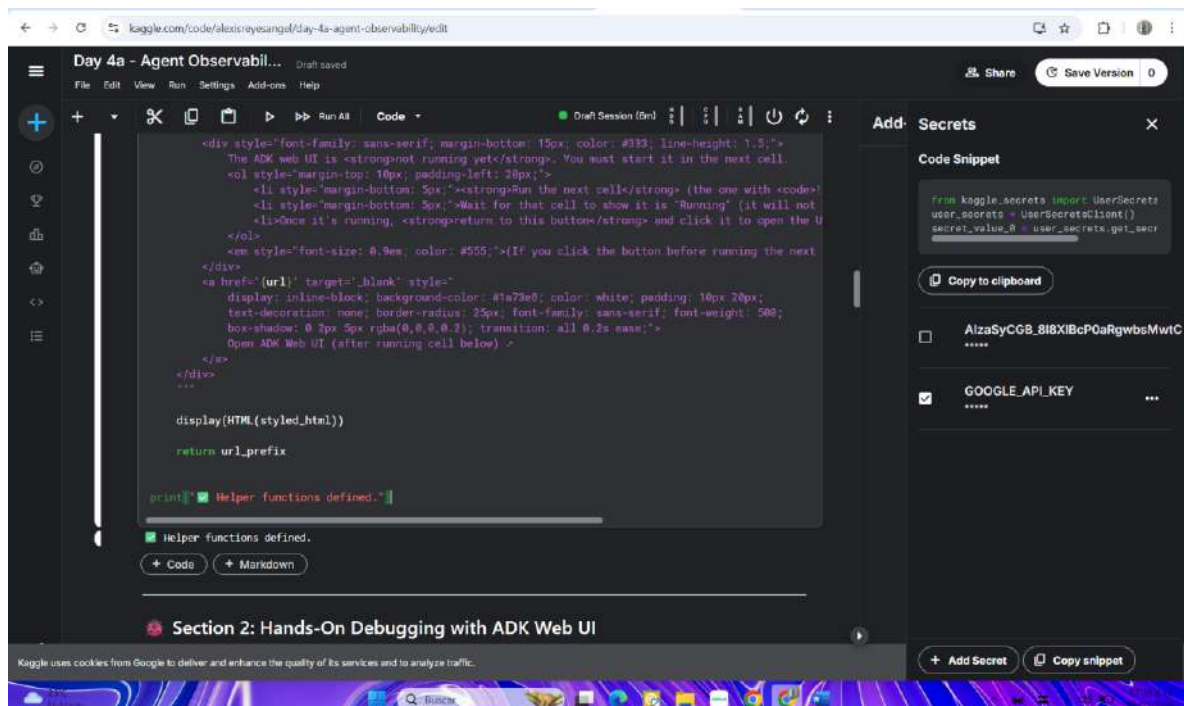
print('Logging configured')
```

Below the code cell, there is a message: "Logging configured" and buttons for "Code" and "Markdown".

On the right side, there is a "Secrets" panel showing a list of secrets. The secret "GOOGLE_API_KEY" is selected and its value is displayed as "*****".

Below the code cell, there is a section titled "1.4: Set up proxy and tunneling" with the text: "We'll use a proxy to access the AER web UI from within the Kaggle Notebooks environment. If you are running this outside the Kaggle environment, you don't need to do this."

En este código se obtiene la URL de acceso a un servicio en Kaggle usando el servidor activo, construye el enlace con el puerto correspondiente y se muestra un mensaje en pantalla con instrucciones para acceder a la interfaz web.



The screenshot shows a Kaggle notebook interface. The main code cell contains HTML and Python code. The HTML part creates a button with the text "Open ADK Web UI (after running cell below)". The Python part uses `display` to show the HTML and `print` to show a message "Helper functions defined.". The output of the cell shows the button and the message.

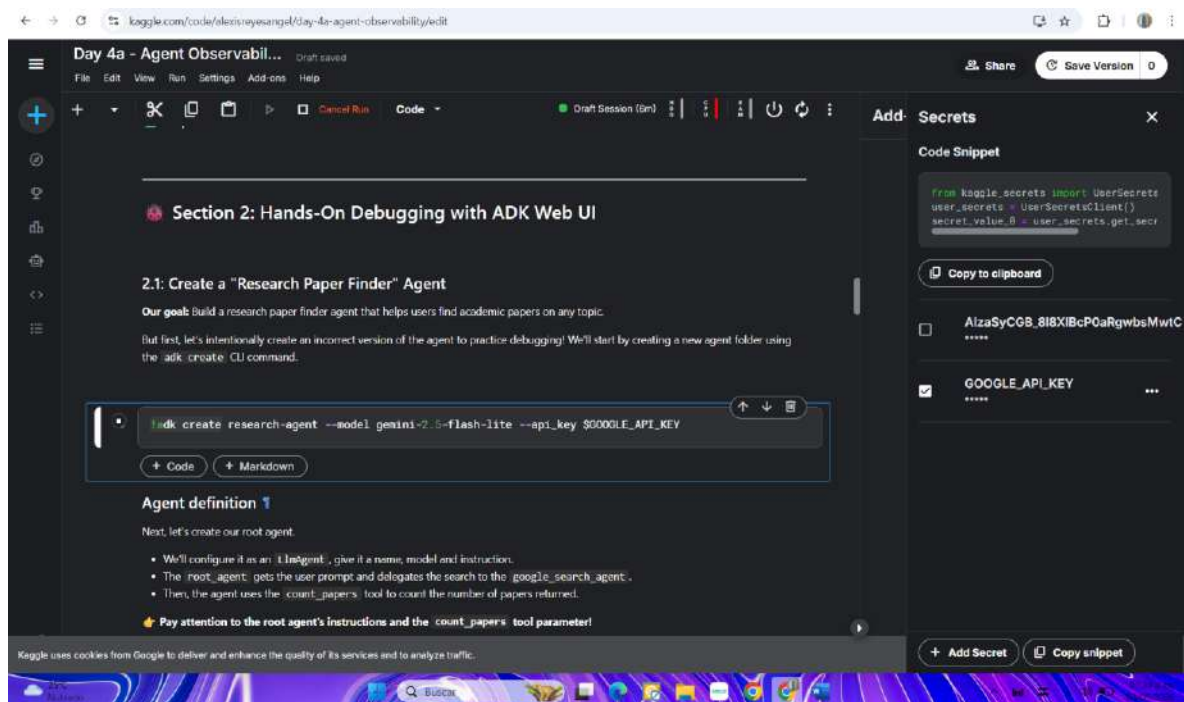
```
<div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
  The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
</div>
<ol style="margin-top: 10px; padding-left: 20px;">
  <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code></code>)
  <li style="margin-bottom: 5px;"><strong>Wait for that cell to show it is "Running" (it will not
  <li><strong>Once it's running, <strong>return to this button</strong> and click it to open the U
  </li>
</ol>
<div style="font-size: 0.9em; color: #555;"><strong>(If you click the button before running the next
</div>
<a href="{url}" target="_blank" style="
  display: inline-block; background-color: #1a73e0; color: white; padding: 10px 20px;
  text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
  box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
  Open ADK Web UI (after running cell below) </a>
</div>
</div>
</div>

display(HTML(styled_html))

return url_prefix

print("Helper functions defined.")
```

Helper functions defined.



The screenshot shows the same Kaggle notebook interface, but the code cell is now empty, and the output shows the start of a new section. The section title is "Section 2: Hands-On Debugging with ADK Web UI". Below the title, there is a sub-section title "2.1: Create a 'Research Paper Finder' Agent". The text below the sub-section title describes the goal of the agent and the steps to create it. A terminal command is shown in a box, which is `!adk create research-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY`.

Section 2: Hands-On Debugging with ADK Web UI

2.1: Create a "Research Paper Finder" Agent

Our goal: Build a research paper finder agent that helps users find academic papers on any topic.

But first, let's intentionally create an incorrect version of the agent to practice debugging! We'll start by creating a new agent folder using the `adk create` CLI command.

```
!adk create research-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

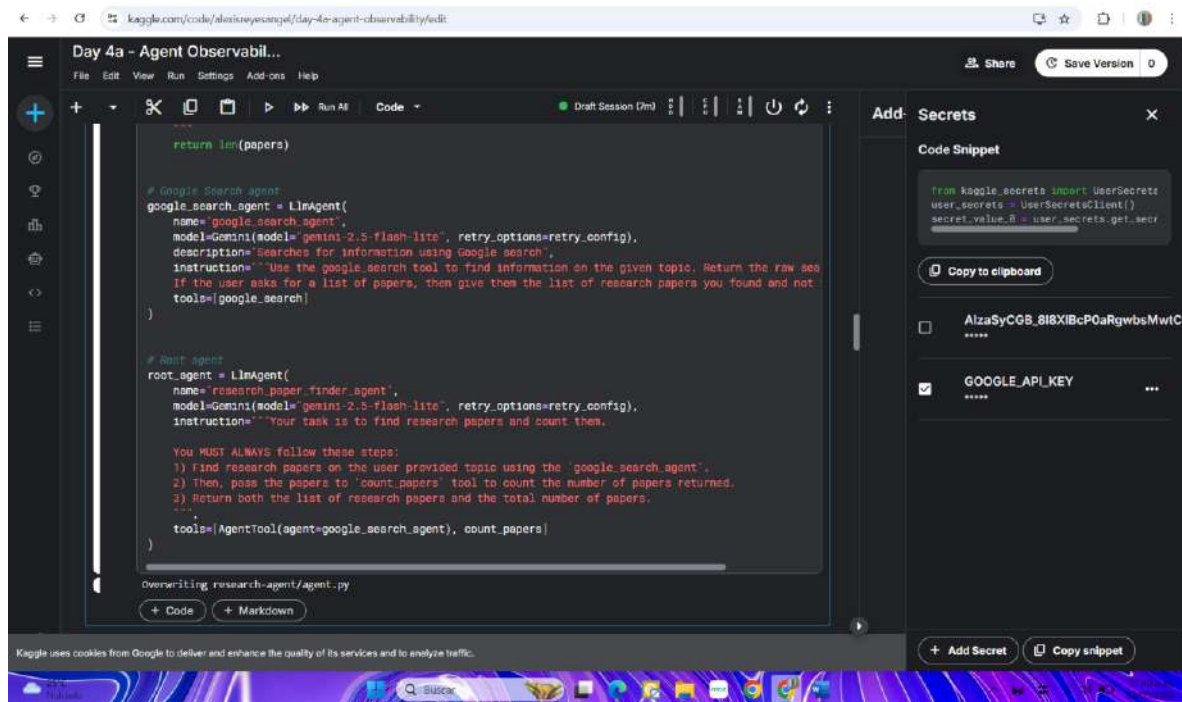
Agent definition

Next, let's create our root agent.

- We'll configure it as an `LLMAgent`, give it a name, model and instruction.
- The `root_agent` gets the user prompt and delegates the search to the `google_search_agent`.
- Then, the agent uses the `count_papers` tool to count the number of papers returned.

Pay attention to the root agent's instructions and the `count_papers` tool parameter!

En este código se creó un archivo donde se configuró un agente que usa un modelo de Google, definió opciones de reintento para errores de conexión y creó una función que cuenta la cantidad de elementos en una lista de textos.



The screenshot shows a Kaggle notebook interface with the title "Day 4a - Agent Observability...". The code in the cell is as follows:

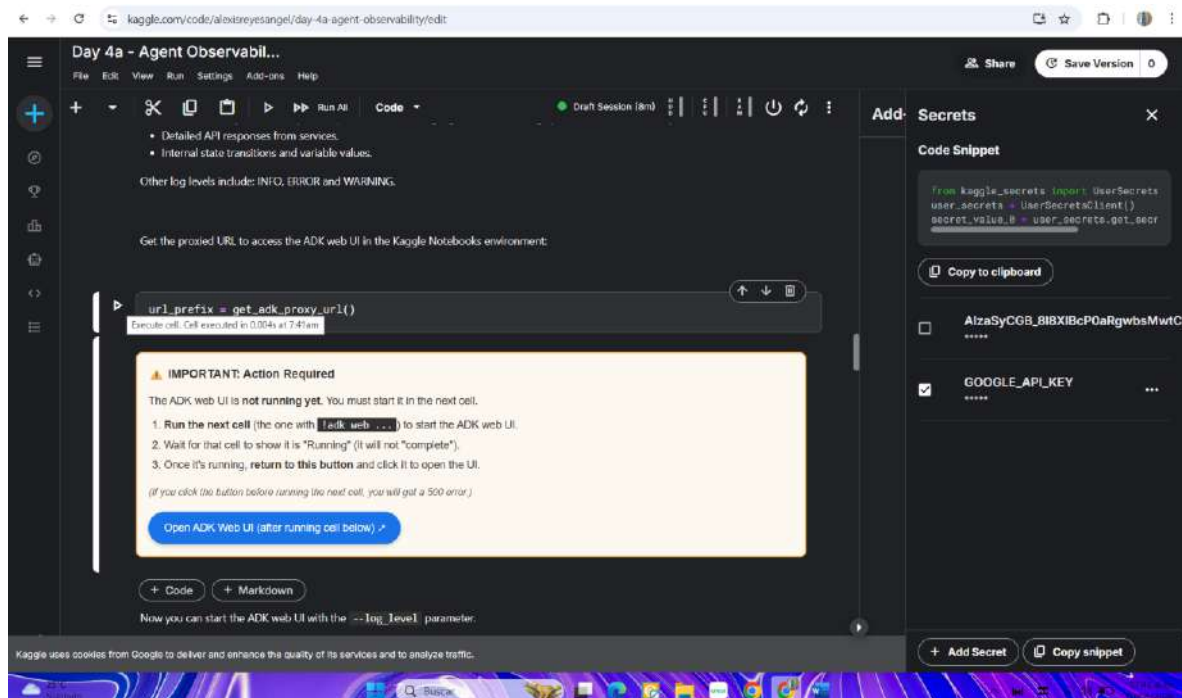
```
return len(papers)

# Google Search agent
google_search_agent = LLMAgent(
    name="google_search_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    description="searches for information using Google search",
    instruction="Use the google search tool to find information on the given topic. Return the raw sea
If the user asks for a list of papers, then give them the list of research papers you found and not
tools=[google_search]
)

# Root agent
root_agent = LLMAgent(
    name="research_paper_finder_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="Your task is to find research papers and count them.

You MUST ALWAYS follow these steps:
1) Find research papers on the user provided topic using the 'google_search_agent'.
2) Then, pass the papers to 'count_papers' tool to count the number of papers returned.
3) Return both the list of research papers and the total number of papers.
...
tools=[AgentTool(agent=google_search_agent), count_papers]
)
```

Below the code, it says "Overwriting research-agent/agent.py" with buttons for "+ Code" and "+ Markdown". On the right, there is a "Secrets" panel with a "Code Snippet" showing the import of `UserSecrets` and `UserSecretClient` from `kaggle\_secrets`, and a list of secrets including `AlzaSyCGB\_818XIBcP0aRgwbsMwtC` and `GOOGLE\_API\_KEY`.



The screenshot shows the same Kaggle notebook interface, but now the code cell has been executed. The output area shows the following:

```
url_prefix = get_adk_proxy_url()
Execute cell. Cell executed in 0.004s at 7:41am
```

Below the output, there is a yellow box with the title "IMPORTANT: Action Required". The text inside the box says:

The ADK web UI is **not running yet**. You must start it in the next cell.

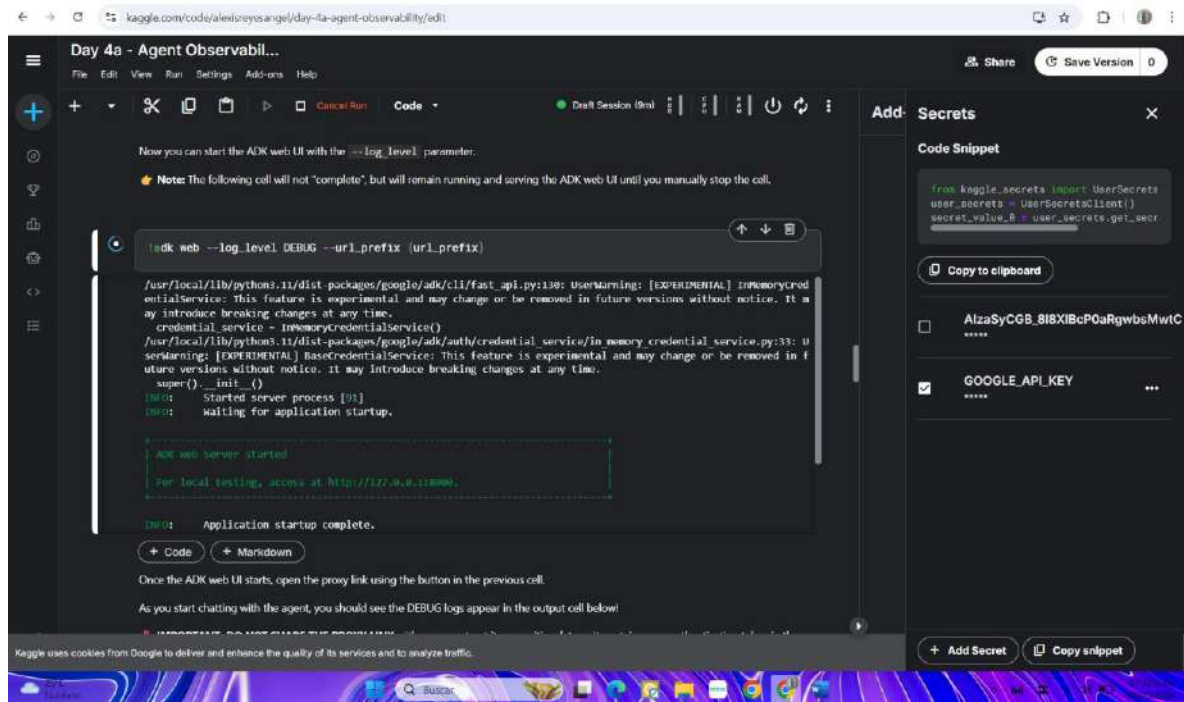
1. Run the next cell (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, return to this button and click it to open the UI.

(If you click this button before running the next cell, you will get a 500 error.)

Below the text, there is a blue button that says "Open ADK Web UI (after running cell below)".

At the bottom of the output area, it says "Now you can start the ADK web UI with the `--log_level` parameter."

En este código ejecuto un servidor web de ADK en modo DEBUG que levanta una interfaz accesible en un puerto local; la celda queda corriendo para mantener activo el servicio hasta que la detenga manualmente.



Day 4a - Agent Observabil...

File Edit View Run Settings Add-ons Help

Now you can start the ADK web UI with the `--log_level` parameter.

Note: The following cell will not "complete", but will remain running and serving the ADK web UI until you manually stop the cell.

```
adk web --log_level DEBUG --url_prefix (url_prefix)
```

```
./usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:110: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
./usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [91]
INFO: waiting for application startup.

ADK web server started
Per local testing, access at http://127.0.0.1:11000.

INFO: Application startup complete.
```

+ Code + Markdown

Once the ADK web UI starts, open the proxy link using the button in the previous cell.

As you start chatting with the agent, you should see the DEBUG logs appear in the output cell below!

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze usage.

Secrets

Code Snippet

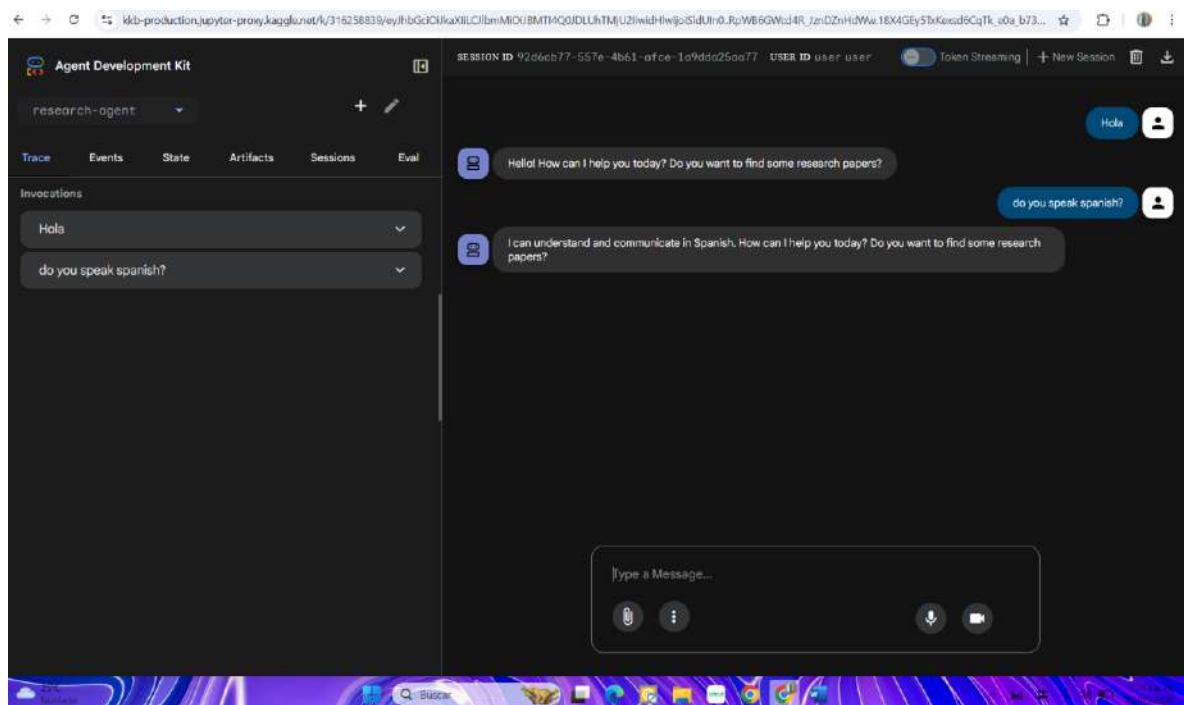
```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_8 = user_secrets.get_secret_value_8
```

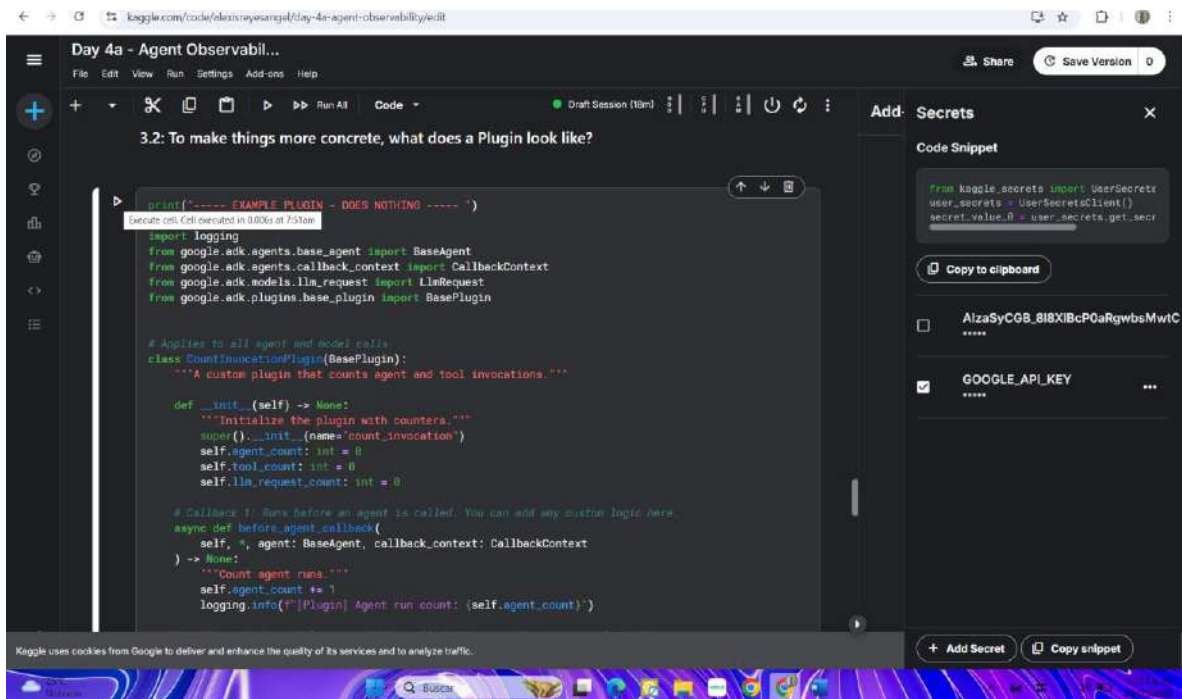
Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

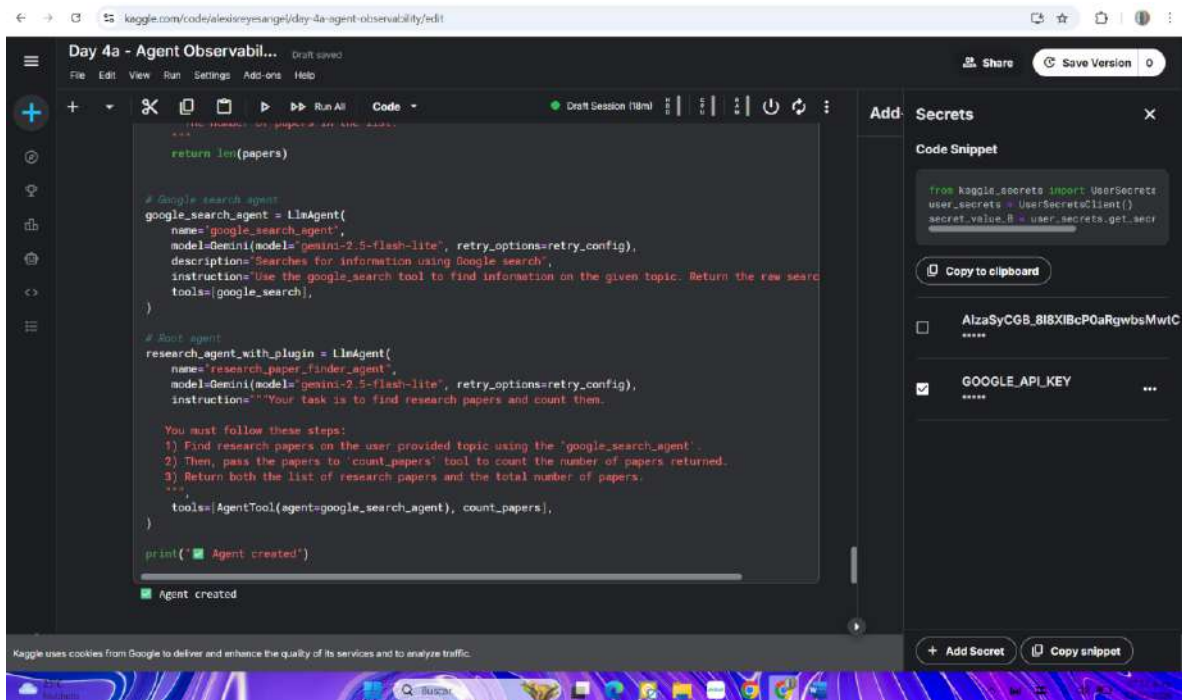
GOOGLE_API_KEY

+ Add Secret + Copy snippet

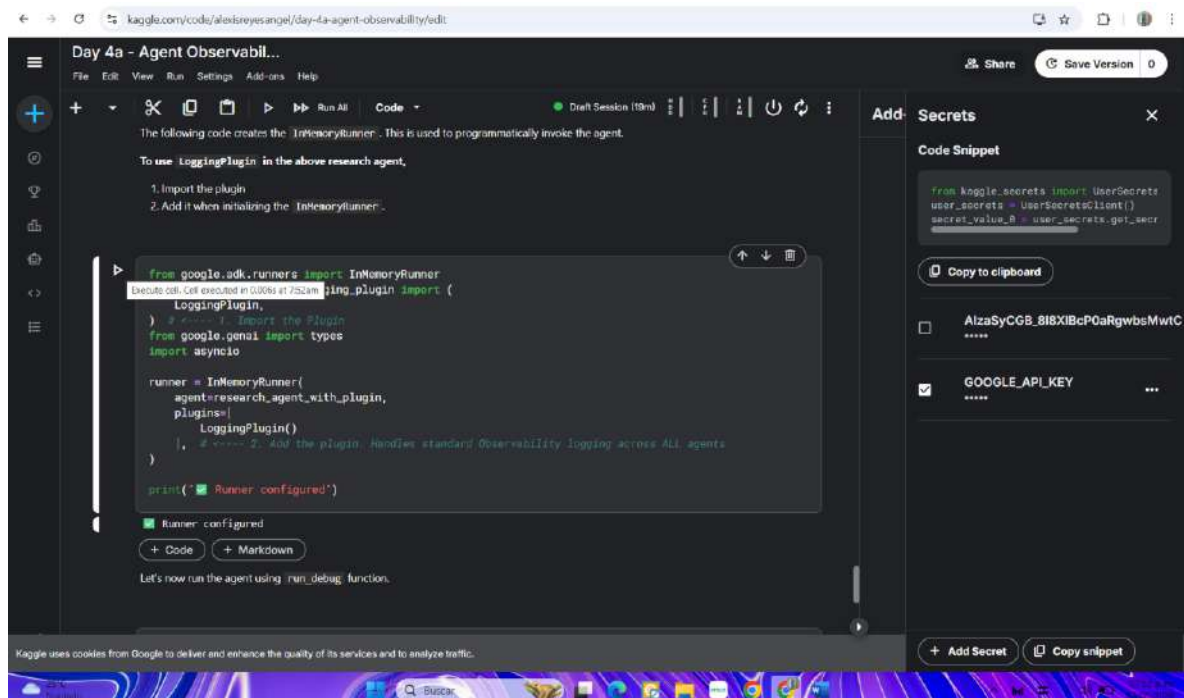




Creamos un agente



En este código configuro un ejecutor en memoria para mi agente, le agrego un plugin de logging para registrar la actividad automáticamente y muestro un mensaje confirmando que todo quedó listo.



The screenshot shows a Kaggle notebook interface. The code cell contains the following Python code:

```
from google.adk.runners import InMemoryRunner
from google.adk.plugins import logging_plugin import (
    LoggingPlugin,
)
# <--- 1. Import the Plugin
from google.genai import types
import asyncio

runner = InMemoryRunner(
    agent=research_agent_with_plugin,
    plugins=[
        LoggingPlugin()
    ],
    # <--- 2. Add the plugin. Handles standard Observability logging across ALL agents
)

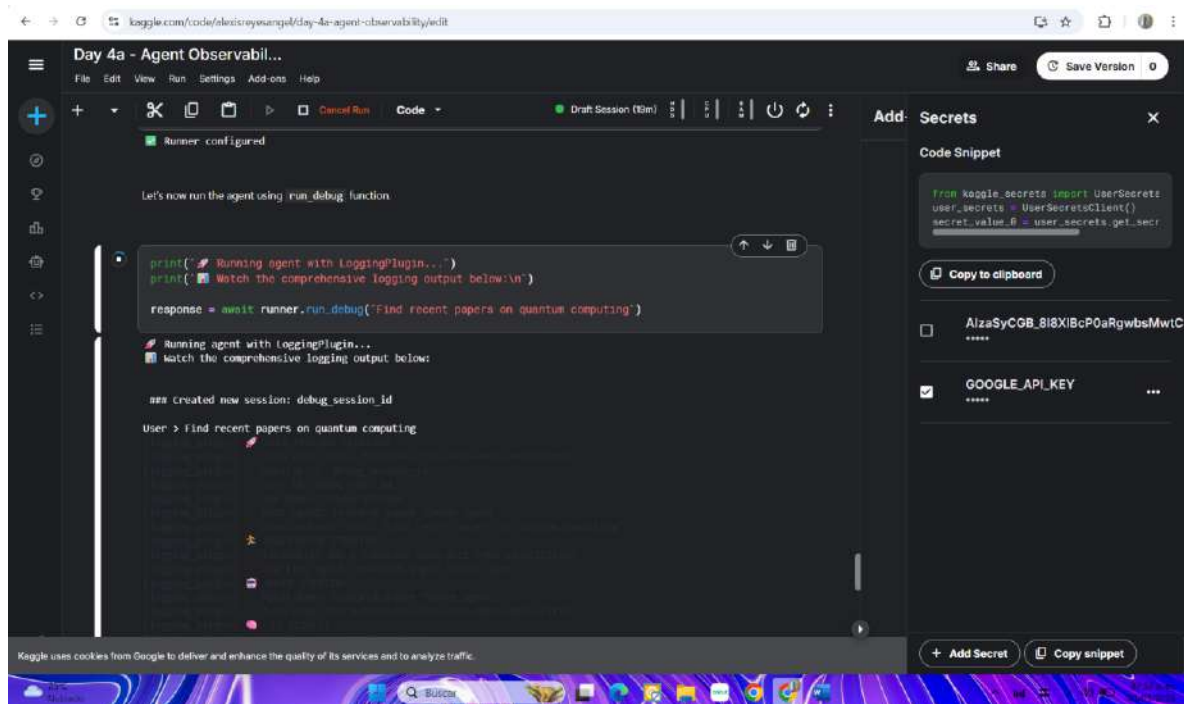
print("✅ Runner configured")
```

Below the code, there is a green status bar that says "Runner configured". To the right of the code editor, there is a "Secrets" panel with a "Code Snippet" section containing the following code:

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_B = user_secrets.get_secret_value("B")
```

Below the code snippet, there is a "Copy to clipboard" button. In the "Secrets" panel, there are two secrets listed: "AlzaSyCGB_8l8XlBcP0aRgwbsMwtC" and "GOOGLE_API_KEY". The "GOOGLE_API_KEY" secret is checked.

En este código muestro mensajes indicando que ejecutaré el agente con logging activado y luego lanzo la consulta para buscar información, obteniendo la respuesta mientras se registran los detalles del proceso.



The screenshot shows the same Kaggle notebook interface. The code cell contains the following Python code:

```
print("🔥 Running agent with LoggingPlugin...")
print("🔥 Watch the comprehensive logging output below:")

response = await runner.run_debug("Find recent papers on quantum computing")

# Running agent with LoggingPlugin...
# Watch the comprehensive logging output below:

### Created new session: debug_session_id

User > Find recent papers on quantum computing
```

Below the code, there is a green status bar that says "Runner configured". To the right of the code editor, there is a "Secrets" panel with a "Code Snippet" section containing the following code:

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_B = user_secrets.get_secret_value("B")
```

Below the code snippet, there is a "Copy to clipboard" button. In the "Secrets" panel, there are two secrets listed: "AlzaSyCGB_8l8XlBcP0aRgwbsMwtC" and "GOOGLE_API_KEY". The "GOOGLE_API_KEY" secret is checked.

Y con esto finaliza el día 4ª

The screenshot shows a Kaggle notebook interface for 'Day 4a - Agent Observability'. The notebook is in 'Draft' mode. The main content area includes a 'Congratulations!' message, a list of tasks to learn how to (debug agent failures, use core debugging patterns, scale observability with logging plugin, understand logging types), a note that no submission is required, and a 'Resources' section with links to ADK documentation, custom plugins, and external integrations. A 'Next Steps' section is also present. On the right, the 'Add Secrets' panel is open, showing a 'Code Snippet' for importing secrets and a list of secrets including 'AlzaSyCGB_8l8XlBcP0aRgwbsMwtC' and 'GOOGLE_API_KEY'.

Day 4a - Agent Observability... Draft saved

File Edit View Run Settings Add-ons Help

Try it out!

Extend the agent's observability by implementing a custom ADK plugin that tracks and reports the total number of tool calls made during a session.

Congratulations!

You now know how to:

- Debug agent failures through DEBUG logs and the ADK web UI
- Use the core debugging pattern: symptom → logs → root cause → fix
- Scale observability with `loggingPlugin` for production systems
- Understand when to use the different logging types

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Resources

Refer to the ADK documentation to learn more about observability:

- ADK Observability Documentation - Complete guide to logging in ADK
- Custom Plugin - Build your own Plugins
- External Integrations - Explore external third-party observability integrations with ADK

Next Steps

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

The screenshot shows a Kaggle notebook interface for 'Day 4b - Agent Evaluation'. The notebook is in 'Draft' mode. The main content area includes a code cell for setting up the environment, a section titled '1.3: Set up proxy and tunneling' with instructions on how to use a proxy to access the ADK web UI, and a code cell for importing IPython and Jupyter server modules. On the right, the 'Add Secrets' panel is open, showing a 'Code Snippet' for importing secrets and a list of secrets including 'AlzaSyCGB_8l8XlBcP0aRgwbsMwtC' and 'GOOGLE_API_KEY'.

Day 4b - Agent Evaluation

File Edit View Run Settings Add-ons Help

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("Setup and authentication complete.")
except Exception as e:
    print(f"Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secret")
```

Setup and authentication complete.

1.3: Set up proxy and tunneling

We'll use a proxy to access the ADK web UI from within the Kaggle Notebooks environment. If you are running this outside the Kaggle environment, you don't need to do this.

```
from IPython.core.display import display, HTML
from jupyter_server.serverapp import list_running_servers
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 4b - Agent Evaluation

1.3: Set up proxy and tunneling

We'll use a proxy to access the ADK web UI from within the Kaggle Notebooks environment. If you are running this outside the Kaggle environment, you don't need to do this.

```
from IPython.core.display import display, HTML
%run cell_Call_executing_in_3.02s at 7:50am
%%import list_running_servers

# Gets the proxied URL in the Kaggle Notebooks environment.
def get_adk_proxy_url():
    PROXY_HOST = "https://kbb-production.jupyter-proxy.kaggle.net"
    ADK_PORT = "8080"

    servers = list(list_running_servers())
    if not servers:
        raise Exception("No running Jupyter servers found.")

    base_url = servers[0]["base_url"]

    try:
        path_parts = base_url.split("/")
        kernel = path_parts[2]
        token = path_parts[3]
    except IndexError:
        raise Exception("Could not parse kernel/token from base URL: {base_url}")

    url_prefix = f"/{kernel}/{token}/proxy/proxy/{ADK_PORT}"
    url = f"{PROXY_HOST}{url_prefix}"
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Day 4b - Agent Evaluation

Helper functions defined.

Section 2: Create a Home Automation Agent

Let's create the agent that will be the center of our evaluation story. This home automation agent seems perfect in basic tests but has hidden flaws we'll discover through comprehensive evaluation. Run the `adk create` command to set up the project scaffolding.

```
adk create home_automation_agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

Agent created in `/kaggle/working/home_automation_agent`:

- `__init__.py`
- `agent.py`

Run the below cell to create the home automation agent.

This agent uses a single `set_device_status` tool to control smart home devices. A device's status can only be ON or OFF. **The agent's instruction is deliberately overconfident** - it claims to control "ALL smart devices" and "any device the user mentions" - setting up the evaluation problems we'll discover.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Day 4b - Agent Evaluation

```
device_id: The unique identifier for the device.
status: The desired status, either 'ON' or 'OFF'.

Returns:
A dictionary confirming the action.
---
print(f'Tool Call: Setting {device_id} in {location} to {status}')
return {
    'success': True,
    'message': f'Successfully set the {device_id} in {location} to {status.lower()}.'
}

# This agent has COLLATERAL FLAWS that we'll discover through evaluation:
root_agent = LlmAgent(
    model=Gemini(model='gemini-2.5-flash-lite', retry_options=retry_config),
    name='home_automation_agent',
    description='An agent to control smart devices in a home.',
    instruction='''You are a home automation assistant. You control ALL smart devices in the house.

    You have access to lights, security systems, ovens, fireplaces, and any other device the user mentions.
    Always try to be helpful and control whatever device the user asks for.

    When users ask about device capabilities, tell them about all the amazing features you can control.'''
    tools=[set_device_status],
)

Overwriting home_automation_agent/agent.py
+ Code + Markdown
```

Section 3: Interactive Evaluation with ADK Web UI

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 4b - Agent Evaluation

Section 3: Interactive Evaluation with ADK Web UI

3.1: Launch ADK Web UI

Get the proxied URL to access the ADK web UI in the Kaggle Notebooks environment:

```
url_prefix = get_adk_proxy_url()
```

IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. Run the **next cell** (the one with **adk web** ...) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, return to **this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

Open ADK Web UI (after running cell below) ↗

+ Code + Markdown

Now you can start the ADK web UI using the following command:

Notes: The following cell will not "complete", but will remain running and serving the ADK web UI until you manually stop the cell.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.



500 : Internal Server Error



Day 4b - Agent Evaluation

File Edit View Run Settings Add-ons Help

Cancel Run Code

Notes: The following cell was run, complete, but was return running and serving the ADK web UI until you manually stop the cell.

+ Code + Markdown

```
!adk web --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: this feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: this feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [88]
INFO: Waiting for application startup.

-----
ADK Web Server started
For local testing, access at http://127.0.0.1:8080.
-----

INFO: Application startup complete.
```

+ Code + Markdown

Once the ADK web UI starts, open the proxy link using the button in the previous cell.

IMPORTANT: DO NOT SHARE THE PROXY LINK with anyone - treat it as sensitive data as it contains your authentication token in the URL.

Share Save Version 0

Add Secrets

Code Snippet

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

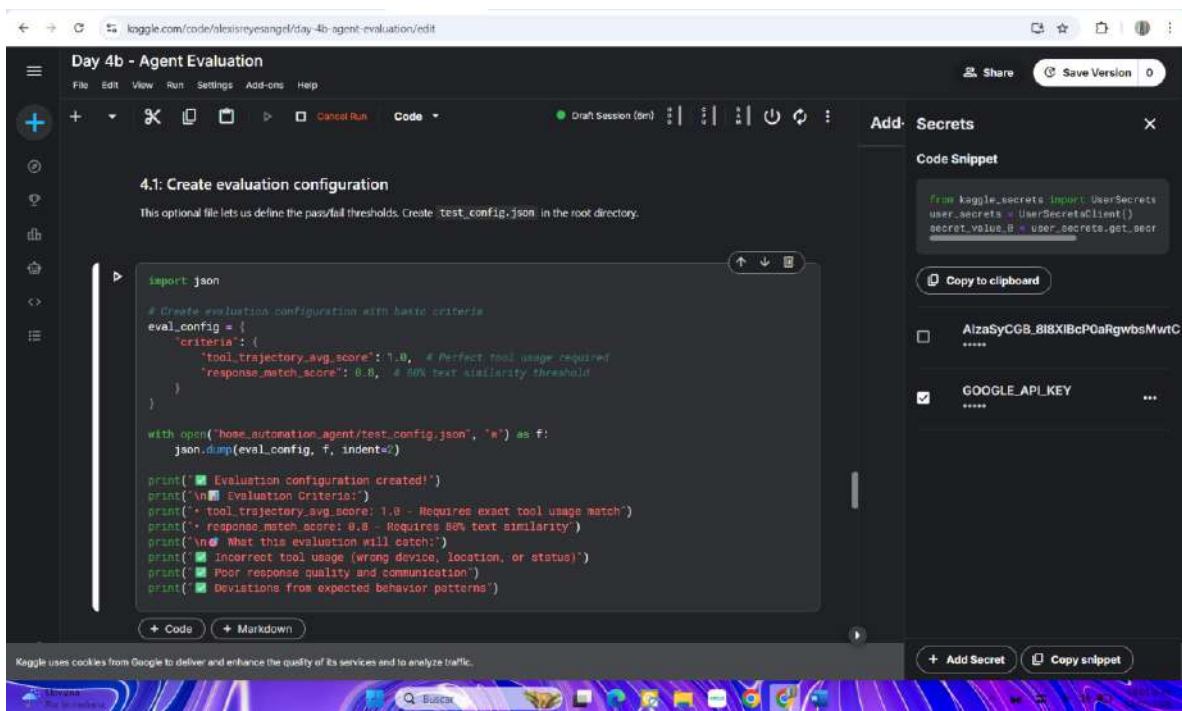
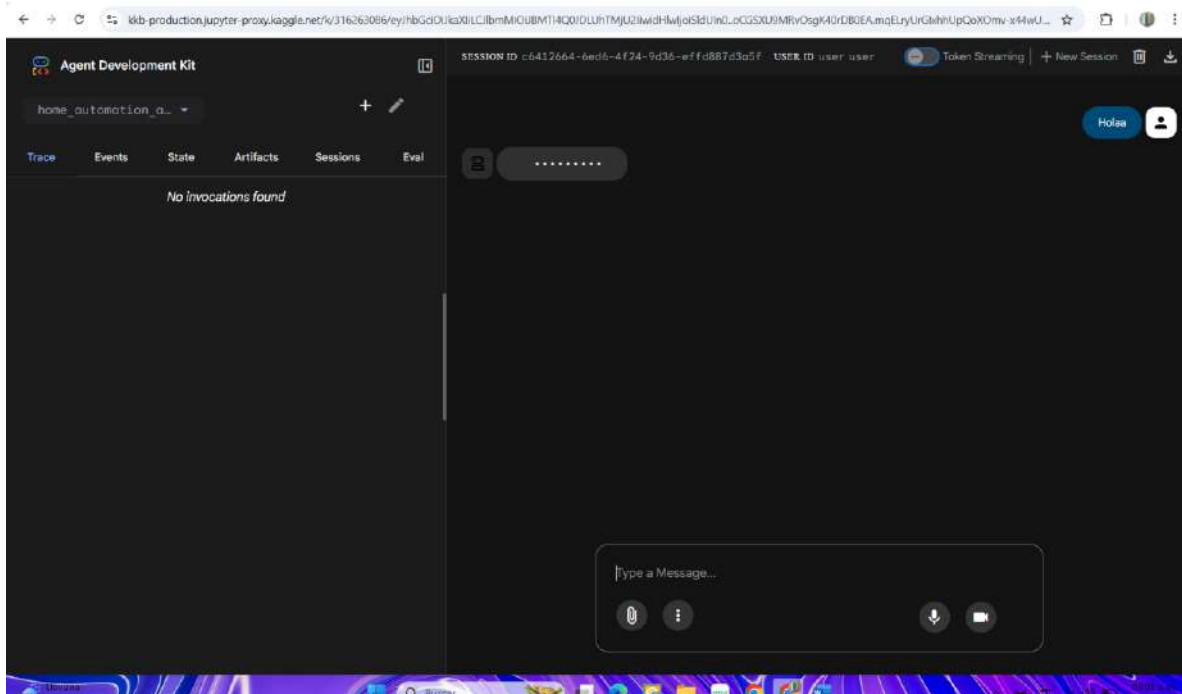
Copy to clipboard

☐ AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.



Day 4b - Agent Evaluation Draft saved

File Edit View Run Settings Add-ons Help

Cancel Run Code Draft Session (6m)

```
    "final_response": {
      "parts": [
        {
          "text": "Successfully set the main light in the kitchen to on."
        }
      ]
    },
    "intermediate_data": {
      "tool_uses": [
        {
          "name": "set_device_status",
          "args": {
            "location": "kitchen",
            "device_id": "main light",
            "status": "ON"
          }
        }
      ]
    }
  },
  ...
}
```

+ Code + Markdown

Let's write the test cases to the `integration_evalset.json` in our agent's root directory.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Share Save Version 0

Add Secrets

Code Snippet

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Day 4b - Agent Evaluation Draft saved

File Edit View Run Settings Add-ons Help

Cancel Run Code Draft Session (6m)

```
import json

with open("home_automation_agent/integration_evalset.json", "w") as f:
    json.dump(test_cases, f, indent=2)

print("✅ Evaluation test cases created")
print("🔪 Test scenarios:")
for case in test_cases["eval_cases"]:
    user_msg = case["conversation"][0]["user_content"]
    print(f"Case {case['eval_id']}: {user_msg}")

print("\n📋 Expected results:")
print("• basic_device_control: Should pass both criteria")
print("• wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters")
print("• poor_response_quality_test: May fail response_match if response differs too much")
```

+ Code + Markdown

4.3: Run CLI Evaluation

Execute the `adk eval` command, pointing it to your agent directory, the evalset, and the config file.

```
print("🔪 Run this command to execute evaluation:")
```

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Share Save Version 0

Add Secrets

Code Snippet

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCG8_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

Add Secret Copy snippet

Day 4b - Agent Evaluation Draft saved

File Edit View Run Settings Add-ons Help

Code

```
print('Evaluation test cases created')
print('\n Test scenarios:')
for case in test_cases['eval_cases']:
    user_msg = case['conversation'][0]['user_content'] + ['parts'][0]['text']
    print(f'Case {case['eval_id']}: {user_msg}')

print('\n Expected results:')
print('* basic_device_control: Should pass both criteria')
print('* wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters')
print('* poor_response_quality_test: May fail response_match if response differs too much')

# Evaluation test cases created

# Test scenarios:
# living_room_light_on: Please turn on the floor lamp in the living room
# kitchen_on_off_sequence: Switch on the main light in the kitchen.

# Expected results:
# basic_device_control: Should pass both criteria
# wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters
# poor_response_quality_test: May fail response_match if response differs too much
```

+ Code + Markdown

4.3: Run CLI Evaluation

Execute the `adk eval` command, pointing it to your agent directory, the evalset, and the config file.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Secrets

Code Snippet

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 4b - Agent Evaluation Draft saved

File Edit View Run Settings Add-ons Help

Code

4.3: Run CLI Evaluation

Execute the `adk eval` command, pointing it to your agent directory, the evalset, and the config file.

```
adk eval home_automation_agent home_automation_agent/integration/evalset.json --config_file_path=home_automation_agent/integration/config.yaml
```

+ Code + Markdown

4.4: Analyzing sample evaluation results

The command will run all test cases and print a summary. The `--print_detailed_results` flag provides a turn-by-turn breakdown of each test, showing scores and a diff for any failures.

```
# Analyzing evaluation results - the data science approach
print('Understanding Evaluation Results:')
print()
print('EXAMPLE ANALYSIS:')
print()
print('Test Case: living_room_light_on')
print('response_match_score: 0.45/0.88')
```


Curso KAGGLE – DÍA 5

Básicamente, en este código estoy configurando mi entorno para usar una API. Importo librerías, obtengo mi clave de Google desde los *secrets* de Kaggle y la guardo como variable de entorno. Si todo sale bien, confirmo que quedó listo; si no, muestro un error indicando que falta la clave.

```
import os
from kaggle.secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print('✅ Setup and authentication complete.')
except Exception as e:
    print(f'❌ Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets')

print('✅ Setup and authentication complete.')
```

Importo todo lo necesario para usar agentes de Google ADK, manejo de requests y utilidades. También oculto warnings para que no estorben y al final solo confirmo que todo se cargó bien.

```
import json
import requests
import subprocess
import time
import uuid

from google.adk.agents import LLMAgent
from google.adk.agents.remote_a2a_agent import (
    RemoteA2Agent,
    AGENT_CARD_WELL_KNOWN_PATH,
)

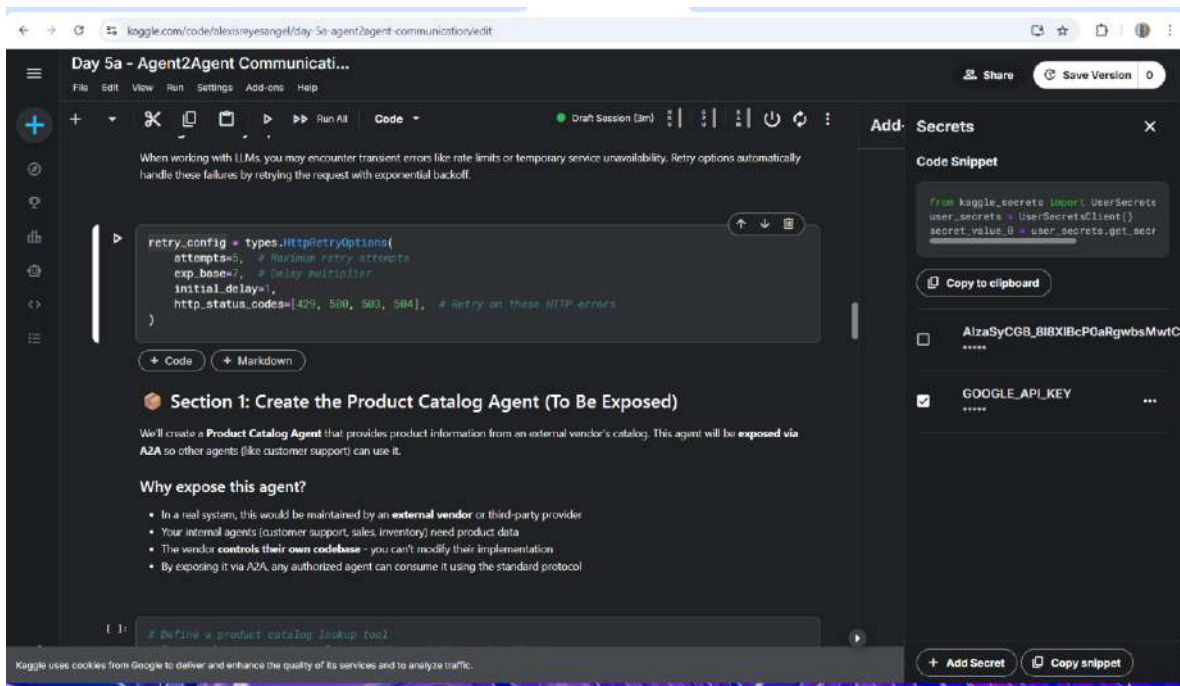
from google.adk.a2a.utils.agent_to_a2a import to_a2a
from google.adk.models.google_llm import Genini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types

# Hide additional warnings in the notebook
import warnings

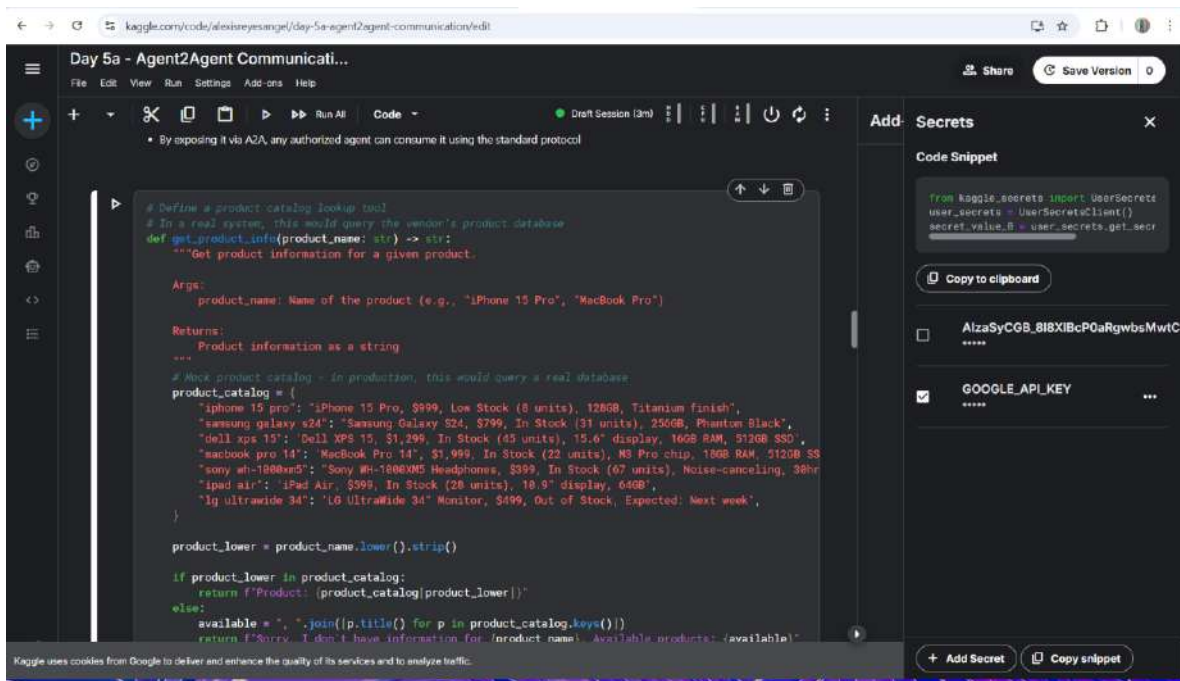
warnings.filterwarnings('ignore')

print('✅ ADK components imported successfully.')
```

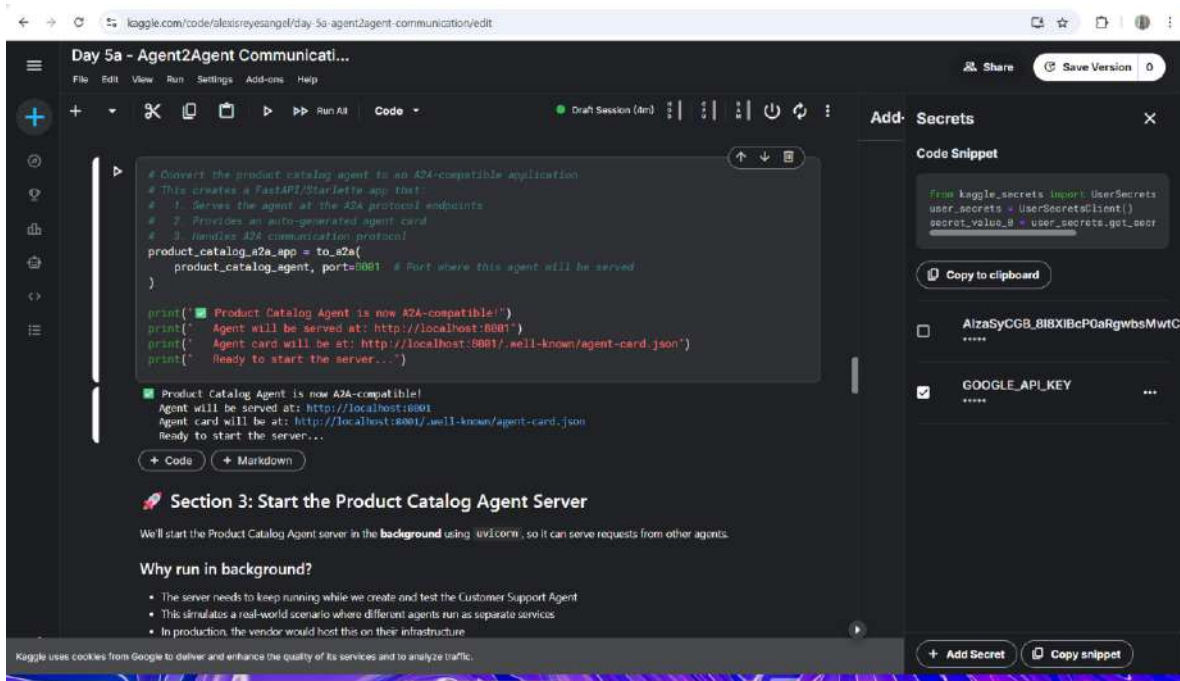
Defino la configuración de reintentos: cuántas veces intentar, el tiempo de espera entre intentos y en qué errores HTTP volver a intentar.



Defino una función que busca info de productos en un catálogo simulado; si lo encuentra devuelve sus datos y si no, muestra opciones disponibles.



Convierto el agente en una app compatible con A2A, lo levanto en un puerto y muestro las rutas donde se puede usar y consultar su info.



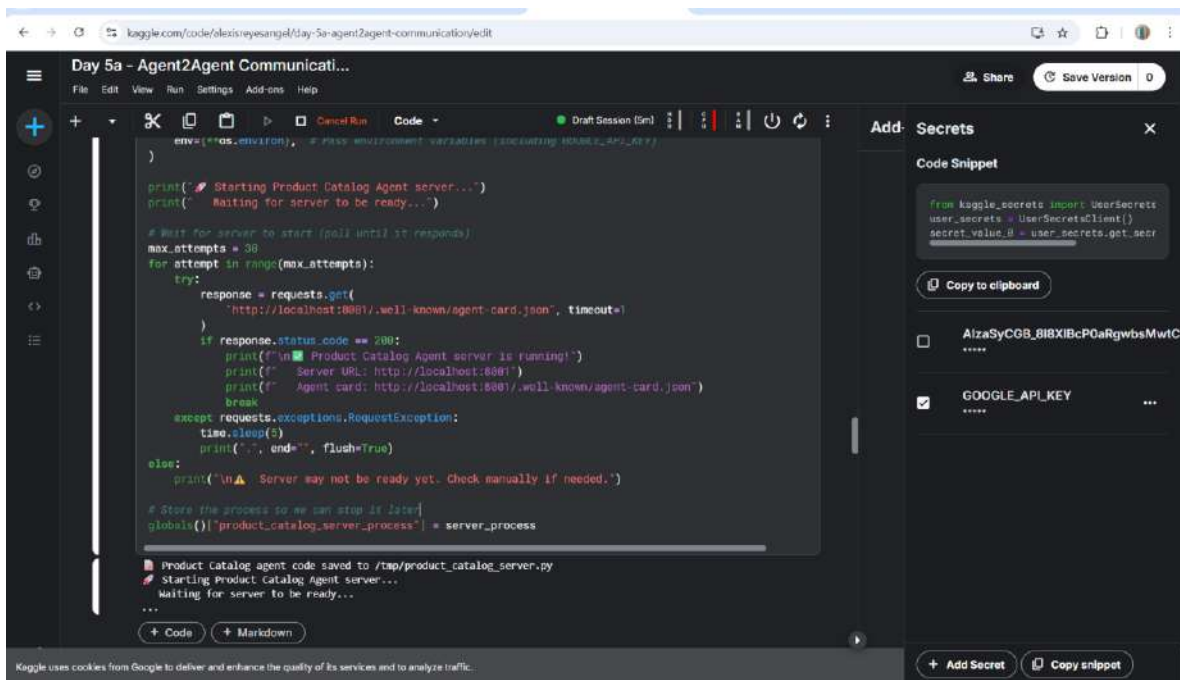
The screenshot shows a Kaggle notebook titled "Day 5a - Agent2Agent Communicati...". The code in the cell is as follows:

```
# Convert the product catalog agent to an A2A-compatible application
# This creates a FastAPI/Starlette app that:
# 1. Serves the agent at the A2A protocol endpoints
# 2. Provides an auto-generated agent card
# 3. Handles A2A communication protocol
product_catalog_a2a_app = to_a2a(
    product_catalog_agent, port=8081 # Port where this agent will be served
)

print("✅ Product Catalog Agent is now A2A-compatible!")
print("  Agent will be served at: http://localhost:8081")
print("  Agent card will be at: http://localhost:8081/.well-known/agent-card.json")
print("  Ready to start the server...")
```

Below the code, there is a section titled "Section 3: Start the Product Catalog Agent Server". It explains that the server will be run in the background using uvicorn. A list of bullet points states: "The server needs to keep running while we create and test the Customer Support Agent", "This simulates a real-world scenario where different agents run as separate services", and "In production, the vendor would host this on their infrastructure".

Guardo el agente en un archivo, levanto un servidor con uvicorn en segundo plano y verifico que esté funcionando antes de continuar.



The screenshot shows the same Kaggle notebook, but with updated code to start the server and verify its status:

```
envs = {"OS_ENVIRON": "PASS_ENVIRONMENT_VARIABLES (Omitting GOOGLE_API_KEY)"
}

print("🚀 Starting Product Catalog Agent server...")
print("  Waiting for server to be ready...")

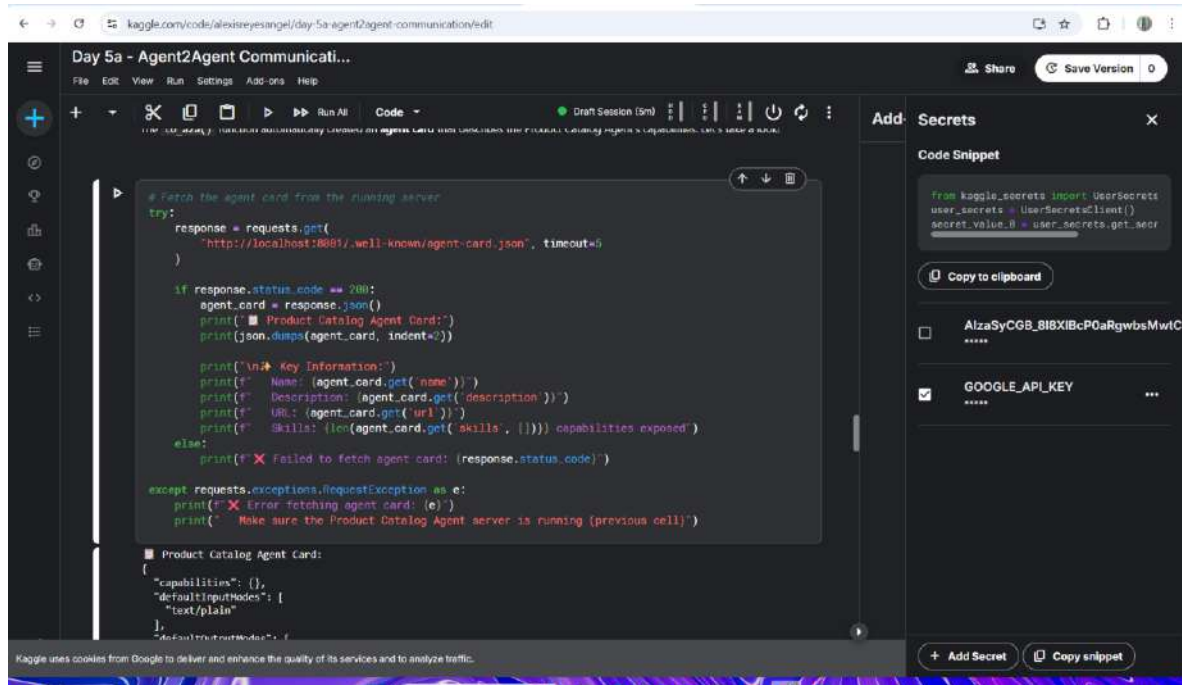
# Wait for server to start (poll until it responds)
max_attempts = 30
for attempt in range(max_attempts):
    try:
        response = requests.get(
            "http://localhost:8081/.well-known/agent-card.json", timeout=1
        )
        if response.status_code == 200:
            print(f"✅ Product Catalog Agent server is running!")
            print(f"  Server URL: http://localhost:8081")
            print(f"  Agent card: http://localhost:8081/.well-known/agent-card.json")
            break
    except requests.exceptions.RequestException:
        time.sleep(5)
        print(".", end="", flush=True)
    else:
        print("\n⚠️ Server may not be ready yet. Check manually if needed.")

# Store the process so we can stop it later
globals()["product_catalog_server_process"] = server_process
```

Below the code, there is a section titled "Product Catalog agent code saved to /tmp/product_catalog_server.py". It shows the command to start the server and the output of the command:

```
🚀 Starting Product Catalog Agent server...
Waiting for server to be ready...
...
```

Hago una petición al servidor para obtener la info del agente; si responde bien, muestro sus datos principales, y si no, marco error.



```
# Fetch the agent card from the running server
try:
    response = requests.get(
        "http://localhost:8081/well-known/agent-card.json", timeout=5
    )

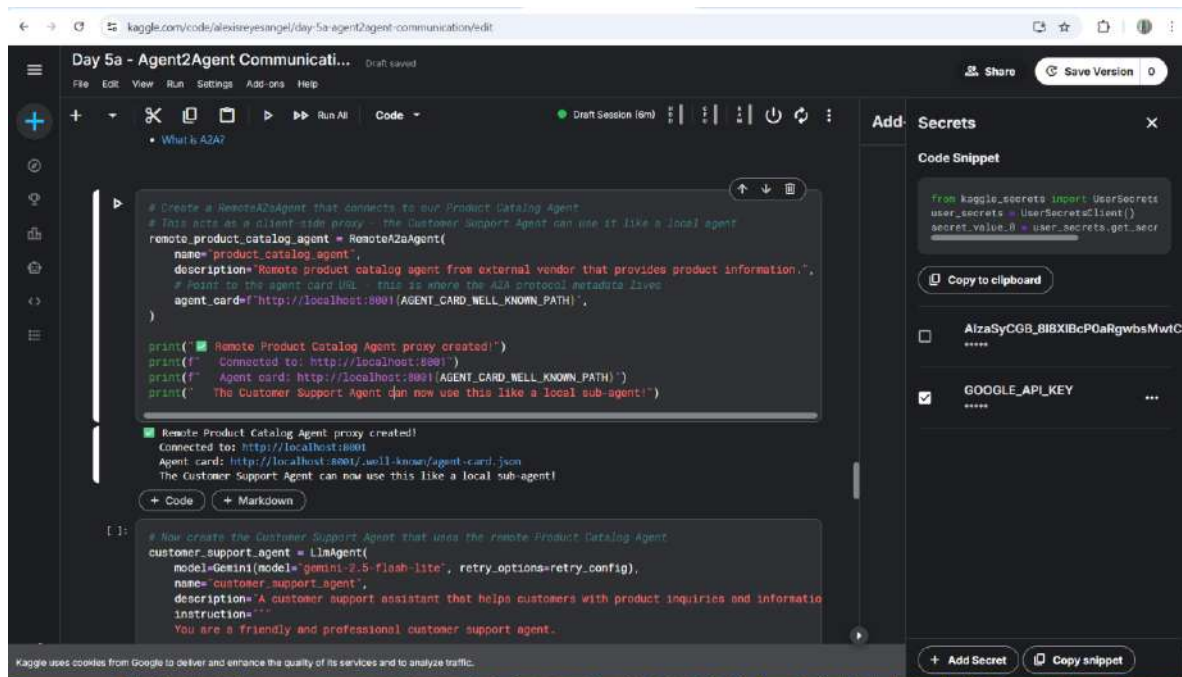
    if response.status_code == 200:
        agent_card = response.json()
        print("■ Product Catalog Agent Card:")
        print(json.dumps(agent_card, indent=4))

        print("\n➡ Key Information:")
        print(f"  Name: {agent_card.get('name')}")
        print(f"  Description: {agent_card.get('description')}")
        print(f"  URL: {agent_card.get('url')}")
        print(f"  Skills: {len(agent_card.get('skills', []))} capabilities exposed")
    else:
        print(f"❌ Failed to fetch agent card: {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"❌ Error fetching agent card: {e}")
    print("  Make sure the Product Catalog Agent server is running (previous cell)")

# Product Catalog Agent Card:
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ]
}
```

Creo un agente remoto que se conecta al servidor del catálogo y lo puedo usar como si fuera un agente local.

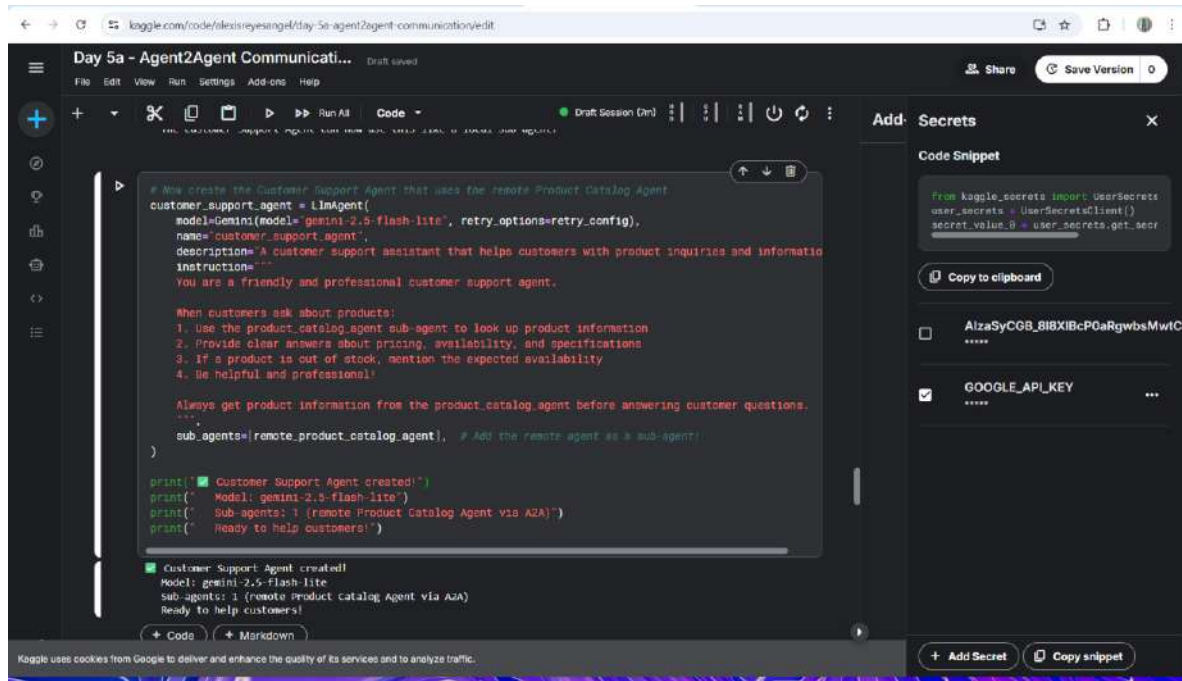


```
# Create a RemoteAgent that connects to our Product Catalog Agent
# This acts as a client-side proxy - the Customer Support Agent can use it like a local agent
remote_product_catalog_agent = RemoteAgent(
    name="product_catalog_agent",
    description="Remote product catalog agent from external vendor that provides product information.",
    # Point to the agent card URL - this is where the Agent2Agent metadata lives
    agent_card="http://localhost:8081/AGENT_CARD_WELL_KNOWN_PATH",
)

print("■ Remote Product Catalog Agent proxy created!")
print(f"  Connected to: http://localhost:8081")
print(f"  Agent card: http://localhost:8081/AGENT_CARD_WELL_KNOWN_PATH")
print("  The Customer Support Agent can now use this like a local sub-agent!")

# Now create the Customer Support Agent that uses the remote Product Catalog Agent
customer_support_agent = LlmAgent(
    model=Gemini(model="gemini-1.5-flash-lite", retry_options=retry_config),
    name="customer_support_agent",
    description="A customer support assistant that helps customers with product inquiries and information.",
    instruction="""
    You are a friendly and professional customer support agent.
    """
)
```

Creo un agente de atención al cliente que usa el agente remoto del catálogo para consultar productos y responder de forma clara y profesional.



```
# Now create the Customer Support Agent that uses the remote Product Catalog Agent
customer_support_agent = LLMAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="customer_support_agent",
    description="A customer support assistant that helps customers with product inquiries and information",
    instructions="""
    You are a friendly and professional customer support agent.

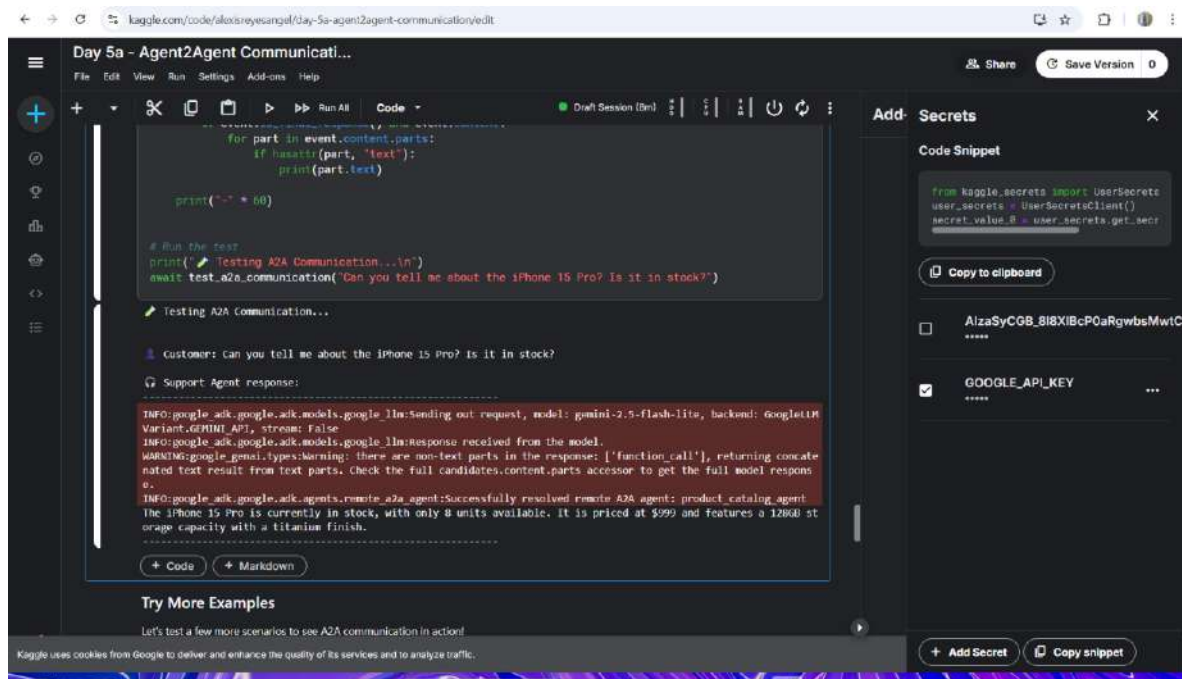
    When customers ask about products:
    1. use the product_catalog_agent sub-agent to look up product information
    2. Provide clear answers about pricing, availability, and specifications
    3. If a product is out of stock, mention the expected availability
    4. Be helpful and professional!

    Always get product information from the product_catalog_agent before answering customer questions.
    """,
    sub_agents=[remote_product_catalog_agent], # Add the remote agent as a sub-agent!
)

print("✅ Customer Support Agent created!")
print("Model: gemini-2.5-flash-lite")
print("Sub-agents: 1 (remote Product Catalog Agent via AZA)")
print("Ready to help customers!")
```

Customer Support Agent created!
Model: gemini-2.5-flash-lite
Sub-agents: 1 (remote Product Catalog Agent via AZA)
Ready to help customers!

Defino una función async que crea una sesión y prepara el runner para probar la comunicación entre el agente de soporte y el de catálogo.



```
for part in event.content.parts:
    if hasattr(part, "text"):
        print(part.text)

print("\n" * 60)

# Run the test
print("🚀 Testing AZA Communication...\n")
await test_aza_communication("Can you tell me about the iPhone 15 Pro? Is it in stock?")
```

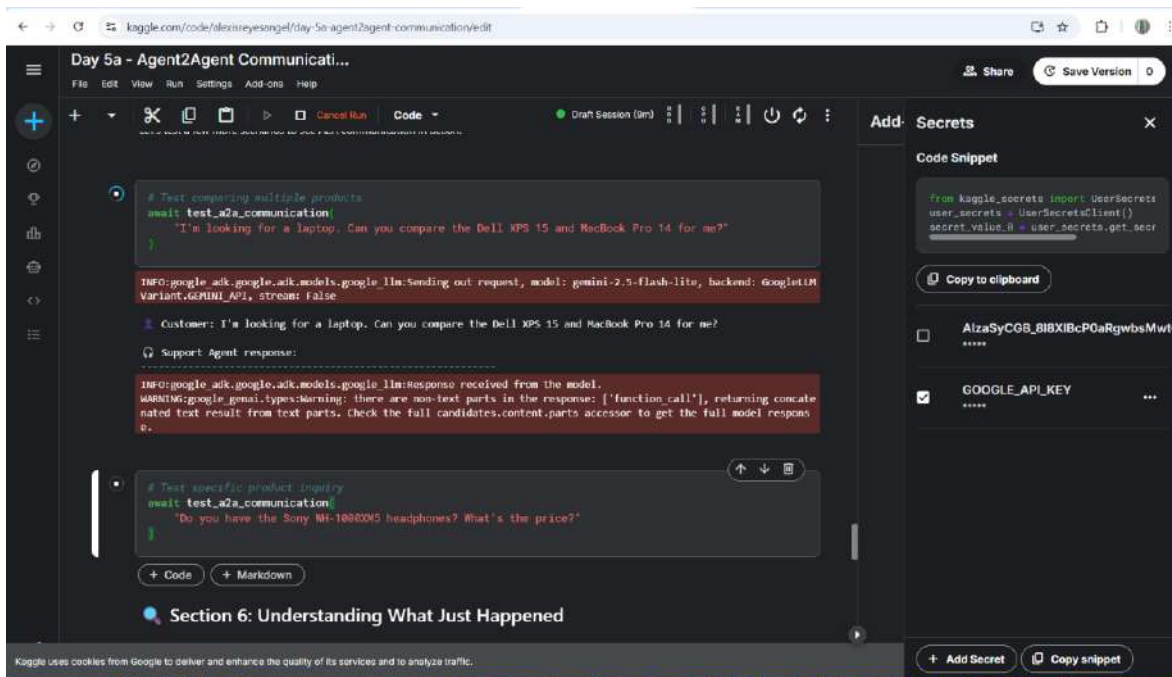
Testing AZA Communication...

Customer: Can you tell me about the iPhone 15 Pro? Is it in stock?

Support Agent response:

```
INFO:google.ai.gemini.models.google_llm:sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
INFO:google.ai.gemini.models.google_llm:response received from the model.
WARNING:google.ai.gemini.types:warning: there are non-text parts in the response: ['function_call'], returning concatenate text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
INFO:google.ai.gemini.agents.remote_aza_agent:Successfully resolved remote AZA agent: product catalog agent
The iPhone 15 Pro is currently in stock, with only 8 units available. It is priced at $999 and features a 128GB storage capacity with a titanium finish.
```

Ejecuto pruebas enviando preguntas al agente de soporte para ver cómo consulta al agente de catálogo y responde con comparaciones o precios.



```
# Test comparing multiple products
await test_a2a_communication(
    "I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?"
)

INFO:google.ai.gemini-2.0-flash-lite:Sending out request, model: gemini-2.0-flash-lite, backend: GoogleLM
Variant:GEMINI_API, stream: False

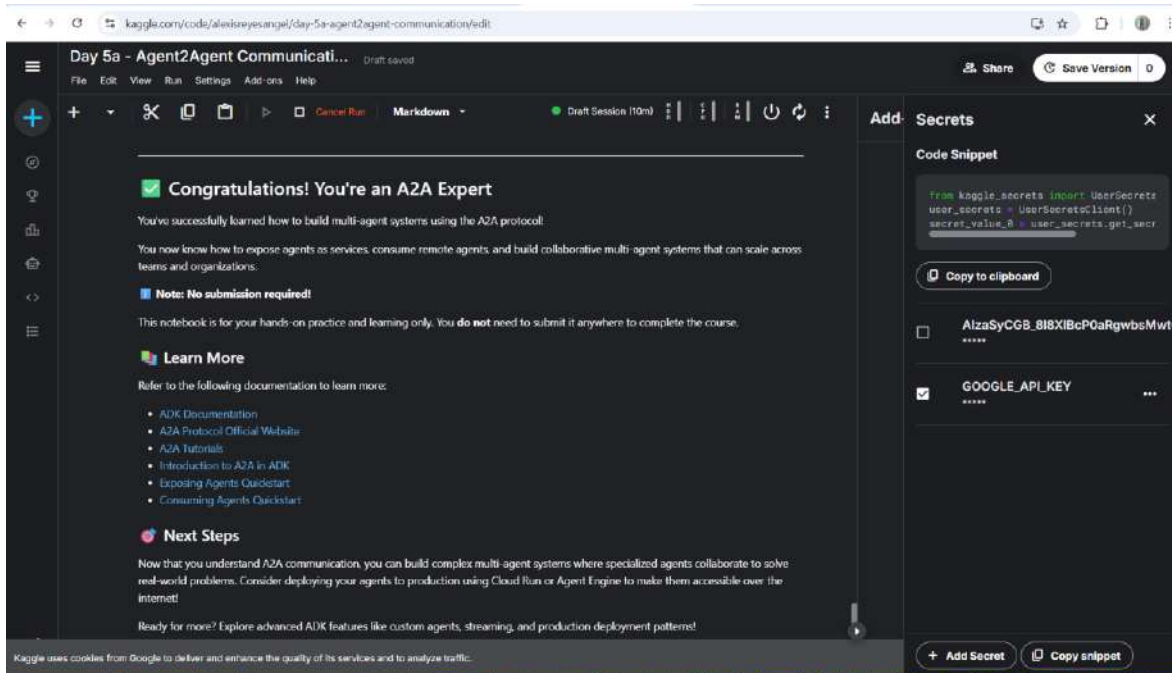
Customer: I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?

Support Agent response:
-----
INFO:google.ai.gemini-2.0-flash-lite:response received from the model.
WARNING:google.gemini.types.warning: there are non-text parts in the response: ['function call'], returning concatenate
d text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

# Test specific product inquiry
await test_a2a_communication(
    "Do you have the Sony WH-1000XM5 headphones? What's the price?"
)
```

Section 6: Understanding What Just Happened

Finalizamos



```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Congratulations! You're an A2A Expert

You've successfully learned how to build multi-agent systems using the A2A protocol.

You now know how to expose agents as services, consume remote agents, and build collaborative multi-agent systems that can scale across teams and organizations.

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Learn More

Refer to the following documentation to learn more:

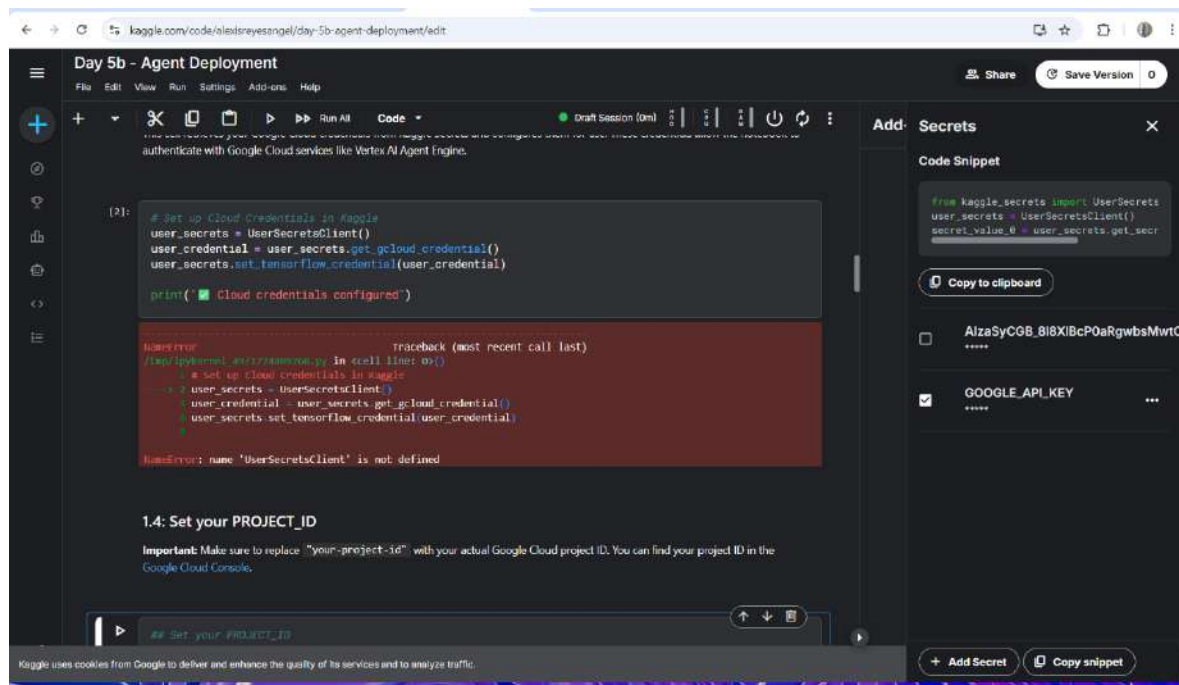
- [ADK Documentation](#)
- [A2A Protocol Official Website](#)
- [A2A Tutorials](#)
- [Introduction to A2A in ADK](#)
- [Exposing Agents Quickstart](#)
- [Consuming Agents Quickstart](#)

Next Steps

Now that you understand A2A communication, you can build complex multi-agent systems where specialized agents collaborate to solve real-world problems. Consider deploying your agents to production using Cloud Run or Agent Engine to make them accessible over the internet!

Ready for more? Explore advanced ADK features like custom agents, streaming, and production deployment patterns!

Configuro credenciales de Google Cloud desde Kaggle y asigno el PROJECT_ID; si no lo cambio por uno real, lanzo error.



The screenshot shows the Kaggle code editor interface for a notebook titled "Day 5b - Agent Deployment". The code cell [2] contains the following Python code:

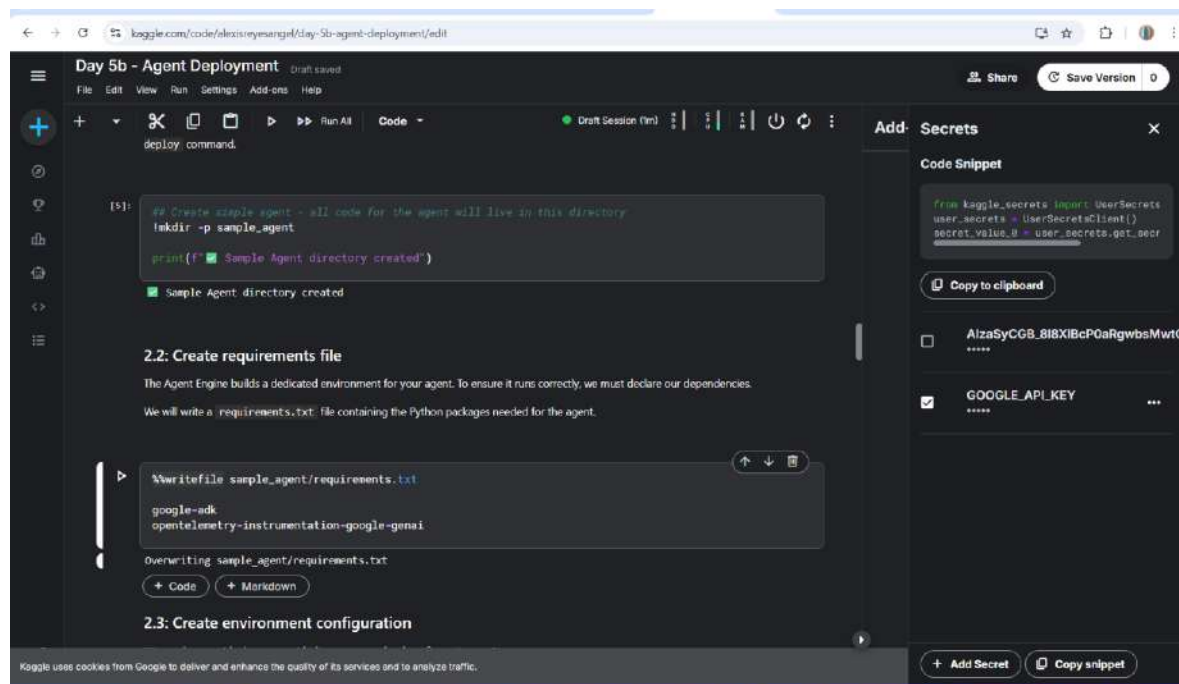
```
# Set up Cloud Credentials in Kaggle
user_secrets = UserSecretsClient()
user_credential = user_secrets.get_gcloud_credential()
user_secrets.set_tensorflow_credential(user_credential)

print('✅ Cloud credentials configured')
```

The execution output shows a `NameError` with the message: `NameError: name 'UserSecretsClient' is not defined`. The error traceback indicates it occurred in `/tmp/ipykernel_4172/4172.py` at line 0. Below the error, a section titled "1.4: Set your PROJECT_ID" provides instructions to replace "your-project-id" with the actual Google Cloud project ID.

On the right side, the "Add Secrets" panel is visible, showing a "Code Snippet" for importing `UserSecrets` and a list of secrets including `GOOGLE_API_KEY`.

Creo una carpeta para el agente y guardo un archivo de dependencias con las librerías necesarias.



The screenshot shows the Kaggle code editor for the same notebook. The code cell [3] contains the following Python code:

```
# Create sample agent - all code for the agent will live in this directory
mkdir -p sample_agent

print(f'✅ Sample Agent directory created')
```

The execution output shows the message: `Sample Agent directory created`. Below the code, a section titled "2.2: Create requirements file" explains that the Agent Engine builds a dedicated environment and that a `requirements.txt` file must be created. The code cell [4] contains the following command:

```
Writefile sample_agent/requirements.txt
google-adk
opentelemetry-instrumentation-google-genai
```

The execution output shows the message: `Overwriting sample_agent/requirements.txt`. Below the code, a section titled "2.3: Create environment configuration" is visible.

On the right side, the "Add Secrets" panel is visible, showing a "Code Snippet" for importing `UserSecrets` and a list of secrets including `GOOGLE_API_KEY`.

Day 5b - Agent Deployment

2.3: Create environment configuration

We need to provide the agent with the necessary cloud configuration settings.

We will write a `.env` file that sets the cloud location to `global` and explicitly enables the Vertex AI backend for the ADK SDK.

```
#!/bin/bash
set -e

# Write the .env file
cat <<EOF >>sample_agent/.env
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=1
EOF
```

Writing `sample_agent/.env`

[+ Code](#) [+ Markdown](#)

Configuration explained:

- `GOOGLE_CLOUD_LOCATION=global` - Uses the `global` endpoint for Gemini API calls
- `GOOGLE_GENAI_USE_VERTEXAI=1` - Configures ADK to use Vertex AI instead of Google AI Studio

2.4: Create agent code

We will now generate the `agent.py` file. This script defines the behavior of our `Weather Assistant`.

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

[Copy to clipboard](#)

☐ AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

☒ GOOGLE_API_KEY

[+ Add Secret](#) [Copy snippet](#)

Day 5b - Agent Deployment

```
return weather_data[city_lower]
else:
    available_cities = ", ".join([c.title() for c in weather_data.keys()])
    return {
        "status": "error",
        "error_message": f"Weather information for '{city}' is not available. Try: {available_cities}"
    }

root_agent = Agent(
    name="weather_assistant",
    model="gemini-2.5-flash-lite",  # Fast, cost-effective Gemini model
    description="A helpful weather assistant that provides weather information for cities.",
    instructions="""
    You are a friendly weather assistant. When users ask about the weather:

    1. Identify the city name from their question
    2. Use the get_weather tool to fetch current weather information
    3. Respond in a friendly, conversational tone
    4. If the city isn't available, suggest one of the available cities

    Be helpful and concise in your responses.
    """,
    tools=[get_weather]
)
```

Writing `sample_agent/agent.py`

[+ Code](#) [+ Markdown](#)

Section 3: Deploy to Agent Engine

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

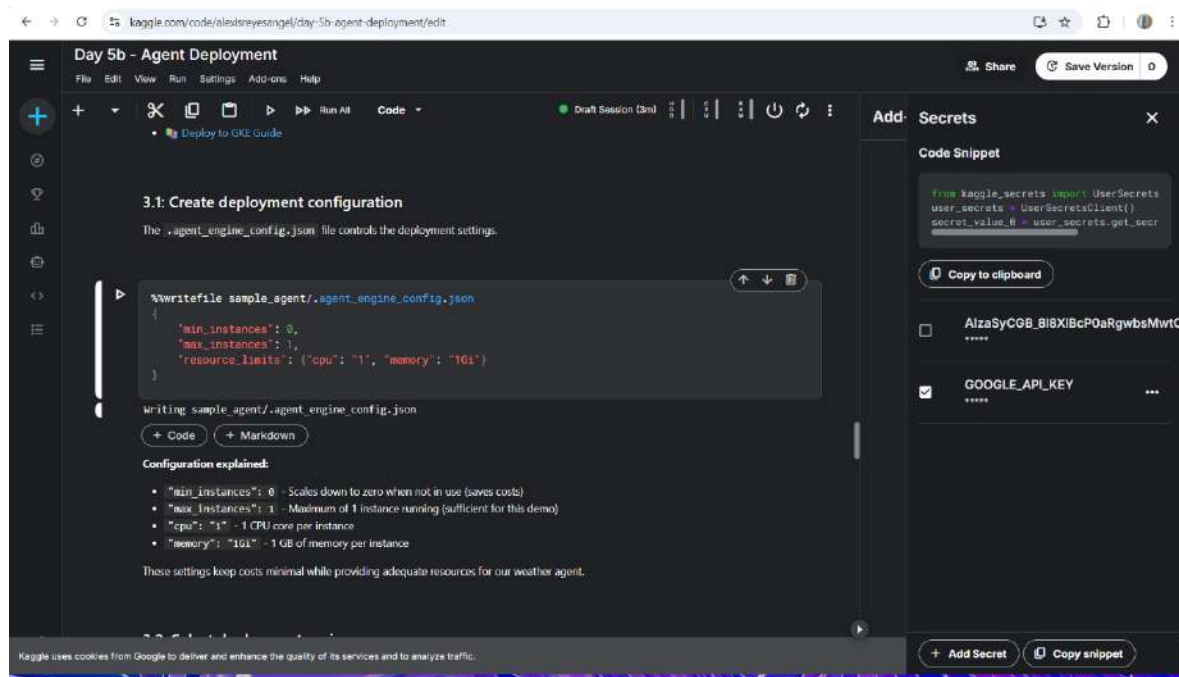
[Copy to clipboard](#)

☐ AlzaSyCGB_8l8XlBcP0aRgwbsMwtC

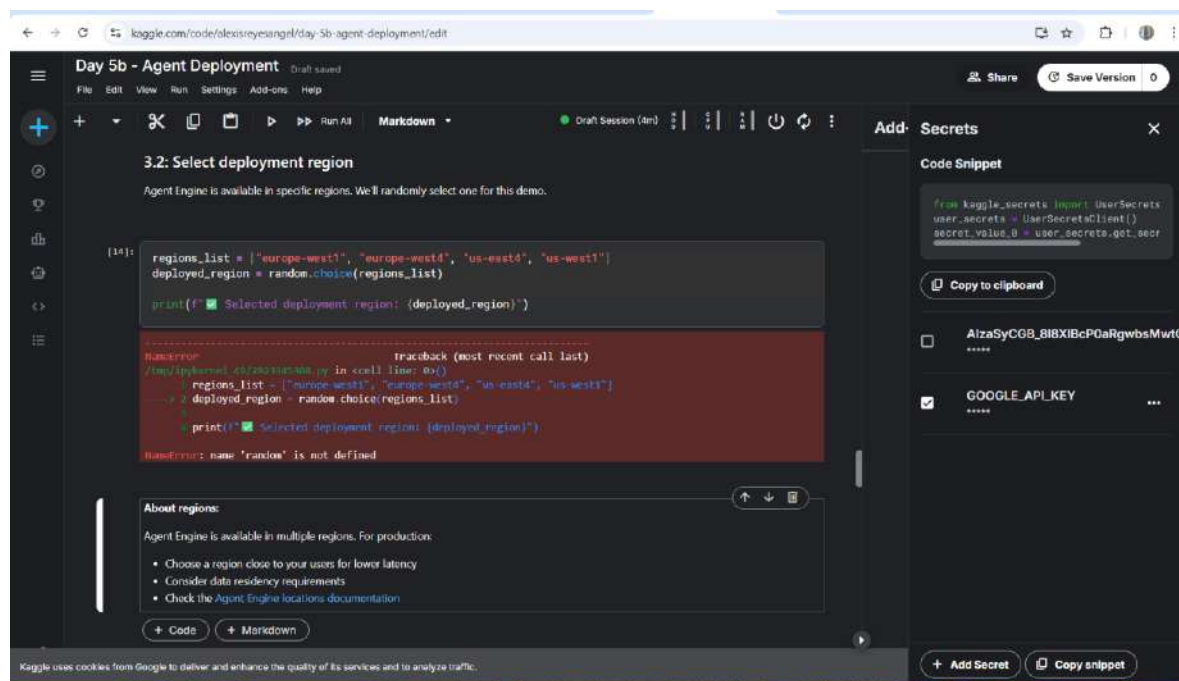
☒ GOOGLE_API_KEY

[+ Add Secret](#) [Copy snippet](#)

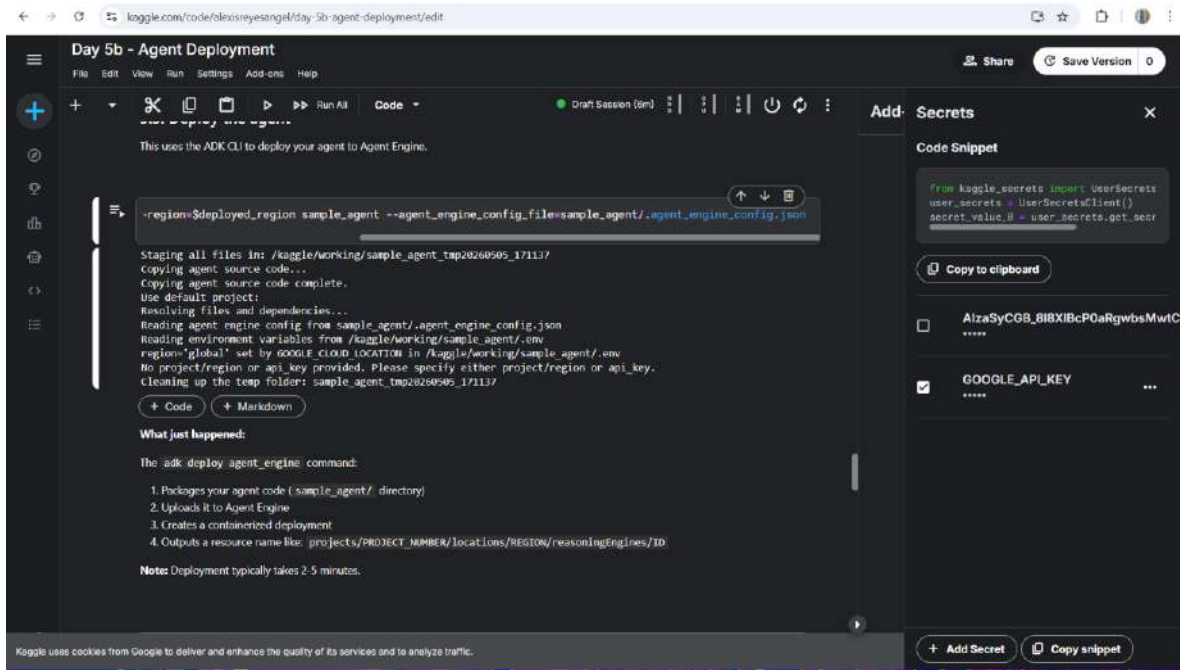
Guardo un archivo de configuración del agente donde defino recursos y cuántas instancias puede usar.



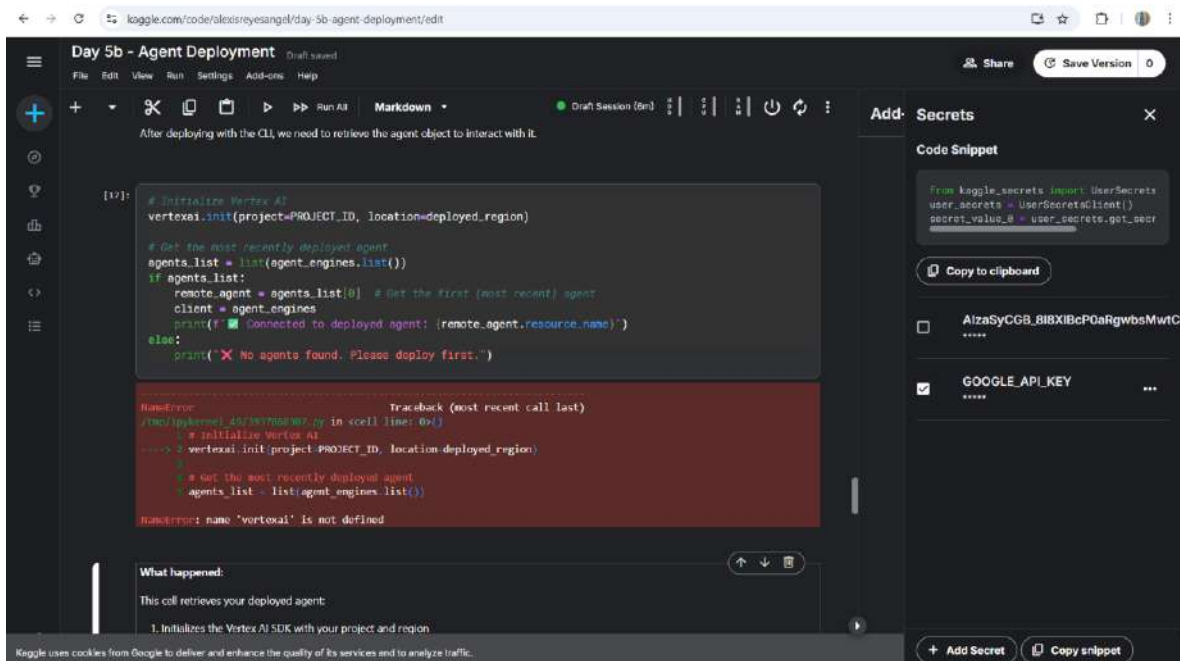
Elijo aleatoriamente una región de despliegue de una lista y la muestro.



Ejecuto el comando para desplegar el agente en la nube usando el proyecto, región y configuración definidos.



Inicializo Vertex AI, busco el agente desplegado y me conecto al más reciente; si no hay, aviso error.



Hago una consulta al agente en streaming (en tiempo real) y voy imprimiendo cada parte de la respuesta conforme llega.

The screenshot shows a Jupyter Notebook interface with the title "Day 5b - Agent Deployment". The active cell is titled "4.2: Test the deployed agent" and contains the following code:

```
async for item in remote_agent.async_stream_query(
    message="What is the weather in Tokyo?",
    user_id="user_42",
):
    print(item)
```

The code execution has failed, resulting in a `RuntimeError: name 'remote_agent' is not defined`. The error traceback shows the code was executed in `/tmp/ipykernel_40/251700001.py` at line 10. The "What happened:" section explains that the cell tests the deployed agent by sending a query and streaming the response.

On the right side, the "Add Secrets" panel is open, showing a code snippet for interacting with Kaggle secrets and a list of secrets including `AlzaSyCG8_8l8XlBcP0aRgwbsMwtC` and `GOOGLE_API_KEY`.

The screenshot shows the same Jupyter Notebook interface, but the active cell now contains code to delete the remote agent:

```
agent_engines.delete(resource_name=remote_agent.resource_name, force=True)
print("✅ Agent successfully deleted")
```

The code execution has failed with the same `RuntimeError: name 'agent_engines' is not defined`. The error traceback shows the code was executed in `/tmp/ipykernel_40/251700001.py` at line 10. The "What happened:" section explains that this cell deletes the deployed agent, identifying the resource name and forcing deletion.

The "Add Secrets" panel remains open on the right side, showing the same code snippet and secrets list.

Finalizado el dia 5

The screenshot shows the Kaggle interface for 'Day 5b - Agent Deployment'. The main content area displays a green checkmark and the text 'Congratulations! You're Ready for Production Deployment'. Below this, it states: 'You've successfully learned how to deploy ADK agents to Vertex AI Agent Engine - taking your agents from development to production! You now know how to deploy agents with enterprise-grade infrastructure, manage costs, and test production deployments.' A 'Learn More' section follows, with a link to 'Refer to the following documentation to learn more:' and a list of links: 'ADK Documentation', 'Agent Engine Documentation', and 'ADK Deployment Guide'. Below this, 'Other Deployment Options' includes 'Cloud Run Deployment' and 'GKE Deployment'. 'Production Best Practices' includes 'Delete test deployments when finished to avoid costs', 'Enable tracing (enable_tracing=True) for debugging', 'Monitor via Vertex AI Console', and 'Follow security best practices'. A 'Course Recap: Your 5-Day Journey' section is at the bottom, stating 'Over the past 5 days, you've learned:'. On the right, the 'Add Secrets' panel is open, showing a 'Code Snippet' with Python code to import secrets from Kaggle. Below the code is a 'Copy to clipboard' button. The secrets list shows two entries: 'AlzaSyCG8_8l8XlBcP0aRgwbsMwtC' (disabled) and 'GOOGLE_API_KEY' (checked).

Day 5b - Agent Deployment

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (Bit)

Share Save Version 0

✓ Congratulations! You're Ready for Production Deployment

You've successfully learned how to deploy ADK agents to Vertex AI Agent Engine - taking your agents from development to production! You now know how to deploy agents with enterprise-grade infrastructure, manage costs, and test production deployments.

Learn More

Refer to the following documentation to learn more:

- [ADK Documentation](#)
- [Agent Engine Documentation](#)
- [ADK Deployment Guide](#)

Other Deployment Options:

- [Cloud Run Deployment](#)
- [GKE Deployment](#)

Production Best Practices:

- Delete test deployments when finished to avoid costs
- Enable tracing (enable_tracing=True) for debugging
- Monitor via [Vertex AI Console](#)
- Follow [security best practices](#)

Course Recap: Your 5-Day Journey

Over the past 5 days, you've learned:

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets\nuser_secrets = UserSecretsClient()\nsecret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ AlzaSyCG8_8l8XlBcP0aRgwbsMwtC *****

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.